

INDUSTRIAL PROJECT

on

E-commerce Web App

Submitted in partial fulfillment of requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTER APPLICATION

Engineering College Bikaner

Session-2024-2026

Submitted by:

Guided by:

Submitted to:

(Signature)

Name: Sumit Gandhi

(Signature)

Name: Devendra Gahlot

(Signature)

Name: Sanjay Tejasvee

Father's Name: Rajkishor
Gandhi

MCA Department
Engineering College Bikaner
Bikaner Technical University,
Bikaner, Rajasthan.

HOD, MCA Department
Engineering College Bikaner
Bikaner Technical University,
Bikaner, Rajasthan.

Roll No.:

Semester: Fourth Sem



Engineering College Bikaner,

Pugal Road, RIICO Karni Industrial Area, Bikaner,
Rajasthan-33004

web: www.ecb.ac.in, Telephone: +91-1512253404,



Bikaner Technical University,
UCET Campus

Pugal Road, RIICO Karni Industrial Area,
Bikaner, Rajasthan-33004

web: www.btu.ac.in

F.No. /CDTS/ 2026/MCA -Internship/150

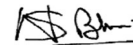
Date: 05-03-2026

TO WHOM IT MAY CONCERN

This is to certify that Mr. Sumit Gandhi S/o Sh. Raj Kishor Gandhi, student of MCA from Engineering College Bikaner, Bikaner (Rajasthan) has successfully completed an industrial training cum internship program on “**MERN Stack**” and also worked on the same projects (e-Commerce solutions). The duration of the internship was 45 days between 19.01.2026 to 05-03-2026.

During the period of his training program with us found punctual, hardworking, and inquisitive.

We wish him success in life.



Narendra Singh Bhui

(Director)



CDTS
Cloud & DevOps

Certificate of Completion

This is to certify that Mr. Sumit Gandhi S/o Sh. Raj Kishor Gandhi, student of MCA from Engineering College Bikaner, Bikaner has successfully completed an industrial training cum internship program on “**MERN Stack**” and also worked on the same project.

The duration of internship was 45 days from 19.01.2026 to 05.03.2026

During the internship session his behaviour was good.

NARENDRA SINGH BHUI
Academy Director

Certificate number: CDTS/CERT/Gen-File/250

Issued date: 05.03.2026

URL: <https://www.cdtsbikaner.in> Contact: +91-7742163889

📍 Behind Rajasthan Patrika Press, Amar Singh Pura, Bikaner PIN 334001 (Rajasthan) INDIA

Society Reg. No. COOP/2020/BIKANER/200308

TABLE OF CONTENTS

Part	Section Number	Section Title
PART I — SYSTEM FOUNDATION	01	Executive Technical Overview
	02	Product Architecture
	03	System Topology & Service Map
	04	Frontend Architecture
	05	Backend Architecture
	06	Authentication & Authorization Architecture
	07	Database Design & Schema Engineering
	08	Domain Modeling & Bounded Contexts
PART II — CORE COMMERCE ENGINE	09	Order Management System
	10	Cart & Checkout Architecture
	11	Inventory Consistency Design
	12	Caching Strategy
	13	Search Architecture
PART III — PLATFORM SERVICES	14	Realtime Systems
	15	Notification Systems
	16	Admin Platform Design
	17	Security Architecture
	18	Scalability Strategy

Part	Section Number	Section Title
	19	CI/CD & Deployment
	20	Infrastructure & Cloud Design
	21	Observability & Monitoring
PART IV — OPERATIONAL EXCELLENCE	22	Error Handling Strategy
	23	Production Readiness Assessment
	24	Future Scaling Challenges
	25	Engineering Best Practices
	26	UI Design
	27	Project Links

APPENDIX

- A. Glossary of Terms
- B. Engineering Decision Records (EDRs)
- C. Dependency Manifest & Rationale
- D. Configuration Reference

PART I — SYSTEM FOUNDATION

SECTION 01: EXECUTIVE TECHNICAL OVERVIEW

1.1 Platform Identity

Browse-Mart is a multi-tenant, multi-role e-commerce platform that supports three distinct user archetypes: Consumers, Sellers, and Administrators. The platform enables sellers to list and manage product inventories, consumers to browse, cart, checkout, and receive orders, and administrators to govern the ecosystem — managing users, products, categories, and platform integrity.

The system is designed as a monolithic API backend with a single-page application (SPA) frontend. Both tiers are deployed independently as serverless functions and static assets on Vercel's edge infrastructure.

1.2 Core Technical Decisions (Summary)

The following decisions define the platform's foundational engineering character and are explained in detail throughout this document:

- **DECISION-001: Monolithic Express API over Microservices**

The platform uses a single Express.js application as its entire backend, rather than decomposing into microservices. At the current scale, this eliminates distributed systems complexity (service discovery, inter-service authentication, distributed tracing) while still enabling future extraction if growth demands it.

- **DECISION-002: MongoDB as Primary Data Store**

A document-oriented model was chosen over a relational database. The product catalog (variable attributes per category), user carts (nested arrays), and order items (denormalized snapshots) map naturally to document structures. JOIN-heavy relational schemas would add overhead without correctness benefits at this scale.

- **DECISION-003: JWT Stateless Authentication over Sessions**
Stateless tokens eliminate the need for shared session storage across serverless function instances. Each Vercel invocation is stateless; a shared Redis/session store would introduce an additional always-on infrastructure dependency.
- **DECISION-004: Vercel Serverless over Traditional VPS/Container Hosting**
Serverless deployment means zero-ops infrastructure management, automatic horizontal scaling, global CDN for static assets, and pay-per-invocation billing at low traffic. The tradeoff is cold start latency and execution time limits, which the current request patterns (API calls < 10s) satisfy.
- **DECISION-005: React SPA with Vite over Next.js SSR**
Given that product discovery is not primarily SEO-driven at this stage, a fully client-rendered SPA was chosen. Vite provides near-instant HMR in development and fast production builds. SSR/SSG was deferred as a future concern.
- **DECISION-006: Context API over Redux for State Management**
The application's cross-component state requirements are minimal: auth token, cart count, user detail, categories, and theme. The overhead of Redux (actions, reducers, middleware, devtools) is not justified when five small React contexts suffice. This also reduces the learning curve for contributors.
- **DECISION-007: Cloudinary for Media Storage**
All product images are uploaded directly to Cloudinary via server-side streaming. This removes binary blob storage from MongoDB (which would degrade query performance), eliminates server-side file system state (incompatible with serverless), and provides a CDN-backed image delivery pipeline with transformation capabilities.
- **DECISION-008: OTP-Based Email Verification over Magic Links**
OTP verification was chosen because it works in environments where it cannot click links from email clients that strip URLs. OTPs are short-lived (10 minutes), bcrypt-hashed at rest, and validated through a dedicated middleware pipeline.

1.3 Platform Scale Context

This documentation is written for the current production state of Browse-Mart, which targets small-to-medium traffic volumes (hundreds to low thousands of concurrent users). Architectural commentary throughout this document explicitly calls out the boundaries where current patterns remain appropriate versus where scaling pressure will force architectural evolution.

Tech Stack

- Backend: Node.js with Express and Mongoose.
- Frontend: React 19 powered by Vite and Tailwind CSS.
- Auth: Stateless JWTs and bcrypt hashing.
- Hosting: Entirely on Vercel (Serverless + CDN).

Key External Services

- MongoDB Atlas: Our primary document database.
- Cloudinary: Handles all image uploads and storage.
- Gmail SMTP: Used for sending transactional emails via Nodemailer.
- Google OAuth: Supports social login flows.

SECTION 02: PRODUCT ARCHITECTURE

2.1 Architectural Style

Browse-Mart follows a Layered Monolith architecture for the backend, combined with a Single-Page Application frontend. This is sometimes called a "Full-Stack Monolith with Deployment Separation."

The backend is not microservices, but it is not undifferentiated spaghetti either. It follows a strict separation of concerns across horizontal layers:

ROUTING LAYER: Express Router definitions per domain (auth, user, product, order, seller, admin, category).

MIDDLEWARE LAYER: Authentication, Authorization, OTP Verification, Status Check, File Upload (Multer).

CONTROLLER LAYER: Request parsing, validation, business logic, response formatting — one controller per operation.

SERVICE LAYER: Email (Nodemailer), OTP generation, Cloudinary upload/delete, Order confirmation email.

MODEL LAYER: Mongoose schemas with pre-save hooks, instance methods, and index definitions.

PERSISTENCE LAYER: MongoDB via Mongoose — Atlas cluster.

2.2 Domain Decomposition

Despite being a monolith, the codebase is functionally decomposed along bounded contexts. Each domain owns its models, controllers, and routes:

DOMAIN: Identity & Access

Models: User, SellerUser

Routes: /api/auth/*, /api/user/*

Concern: Registration, login, OTP, password, profile, Google OAuth

DOMAIN: Catalog

Models: Product, Category

Routes: /api/product/*, /api/category/*

Concern: Product CRUD, image management, categorization, search

DOMAIN: Commerce

Models: Order

Routes: /api/order/*

Concern: Cart operations (on User), checkout, order lifecycle

DOMAIN: Seller Operations

Models: SellerUser, Product, Order

Routes: /api/seller/*

Concern: Seller dashboard, product management, order fulfillment

DOMAIN: Platform Administration

Models: User, Product, SellerUser, Category

Routes: /api/admin/*

Concern: User governance, platform stats, category management

DOMAIN: Engagement

Models: User (embedded wishlist), Product (embedded reviews)

Routes: /api/user/add-to-wishlist, /api/product/submit-review

Concern: Wishlist, reviews, newsletter

The domain boundaries are enforced by middleware (sellerAuthentication, adminAuthentication) rather than physical service isolation. This means a badly-written controller could cross domain lines — a risk that is acceptable at current scale but would be enforced with stricter interfaces (ports/adapters or module boundaries) as the team grows.

2.3 Request Processing Pipeline

Every request through Browse-Mart follows this processing pipeline:

Client Request

- (Initiated from Web Browser, PWA, or Mobile Browser)

Vercel Edge

- TLS termination
- HTTP/2
- Gzip compression

Express Application (server.js)

- **Global Middleware**
 - cors(): Cross-origin policy enforcement
 - express.json(): Request body parsing
 - express.urlencoded(): Form data parsing
- **Route Matching** (Express Router)
- **Route-Level Middleware** (Per endpoint)
 - userAuthentication: JWT validation
 - checkUserStatus: DB freshness check
 - sellerAuthentication / adminAuthentication
 - otpVerification: OTP-gated endpoints
 - multer: File upload endpoints
- **Controller Function**
 - Request validation
 - Business logic execution
 - Database operations (Mongoose)
 - External service calls (Cloudinary, Nodemailer)
 - Response serialization

Response → Client

2.4 Multi-Role Architecture

Browse-Mart's multi-role design is implemented through a single User document with a userType discriminator field ("consumer", "seller", "admin") rather than separate user collections. This means:

BENEFIT: A single login flow for all roles.

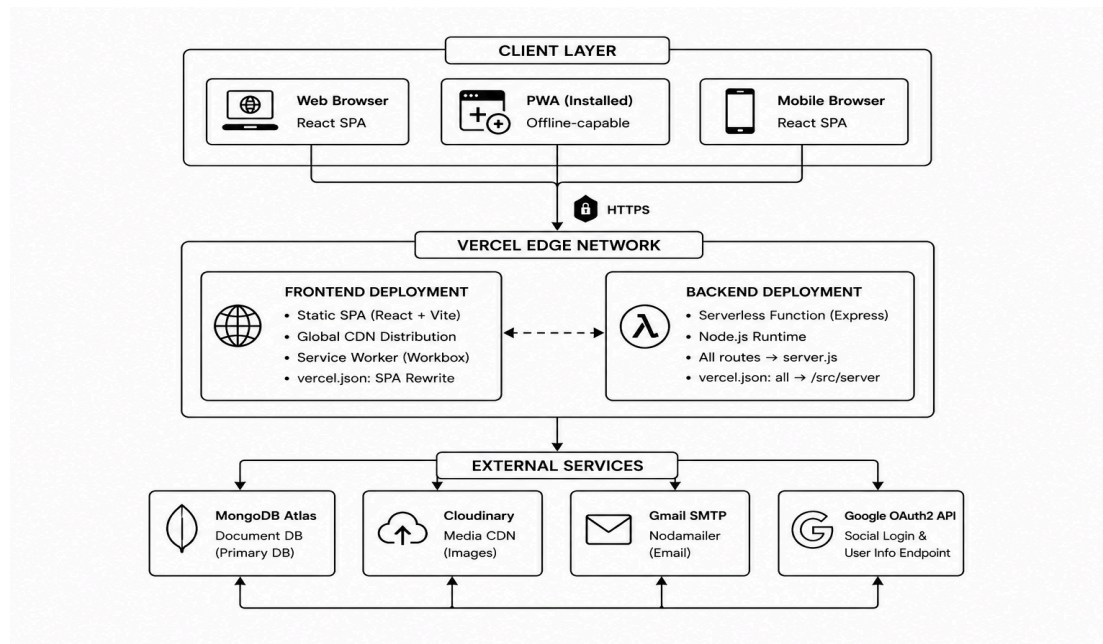
BENEFIT: A user can conceptually hold multiple roles (the data model supports seller + consumer in a single account via the sellerId foreign key on the User document).

TRADEOFF: The User document accumulates fields from all role contexts (cart, wishlist, sellerId, shippingAddress) even when irrelevant to a given role. This is an acceptable denormalization at current data volume.

TRADEOFF: Role-checking logic is duplicated across middleware files rather than centralized in a permission policy object.

SECTION 03: SYSTEM TOPOLOGY & SERVICE MAP

3.1 Deployment Topology



3.2 Data Flow Architecture

There are three distinct data flow patterns in Browse-Mart:

PATTERN A — Synchronous Request/Response (Dominant Pattern)

None

Client —HTTP→ Vercel Function —Mongoose→ MongoDB Atlas

Description:

The client sends an HTTP request to a Vercel serverless function. The function interacts with MongoDB Atlas using Mongoose and returns a JSON response to the client.

PATTERN B — Synchronous with External Side Effect

None

```
Client —HTTP→ Vercel Function —Mongoose→ MongoDB
                                —Nodemailer→ Gmail SMTP
                                —Cloudinary→ CDN
```

Description:

This pattern includes database operations along with external service interactions such as email delivery and image uploads.

Used In:

- User registration (OTP email)
- Order submission (confirmation email)
- Product creation (image upload)
- Password reset workflow

Risk / Limitation:

The side effect (email, image upload) is not transactional with the database write. If the DB write succeeds but email fails, the user receives no OTP. If the image upload fails, the product creation fails mid-way.

Current Handling:

Failures are currently managed with try/catch blocks and error responses. No rollback or distributed transaction mechanism is implemented.

PATTERN C — Static Asset Delivery (Frontend)

None

```
Browser —HTTPS→ Vercel CDN Edge → Cached Static Bundle
```

Description:

Frontend static assets are delivered through the Vercel CDN edge network for low-latency access and caching efficiency. For Progressive Web App (PWA) resources, the Service Worker intercepts requests before they reach the network and serves cached assets directly using the Workbox cache.

3.3 Network Security Boundary

- **PUBLIC ZONE** — Unauthenticated endpoints (register, login, products, categories, Google OAuth initiation)
- **AUTHENTICATED ZONE** — JWT-protected endpoints (user profile, cart, orders, wishlist, checkout)
- **SELLER ZONE** — JWT + userType=seller + sellerId required
- **ADMIN ZONE** — JWT + checkUserStatus + userType=admin required
- **INTERNAL (implicit)** — MongoDB Atlas access restricted to Vercel outbound IPs or Atlas IP allowlist

3.4 Cross-Origin Resource Sharing (CORS) Configuration

CORS is configured on the Express application with credentials support (withCredentials: true on Axios). This is necessary because the frontend and backend are deployed on separate Vercel projects (different origins).

The allowed origin is controlled by the FRONTEND_URL environment variable, ensuring production requests only accept traffic from the known frontend domain. In development, this is set to localhost:5173 (Vite dev server).

Credentials mode must be enabled on both server (cors credentials: true) and client (axios withCredentials: true) for the Authorization header to propagate correctly on cross-origin requests.

SECTION 04: FRONTEND ARCHITECTURE

4.1 Rendering Architecture

Browse-Mart's frontend is a fully client-side rendered (CSR) Single-Page Application. There is no server-side rendering, no static site generation, and no incremental static regeneration at this time.

RENDERING MODEL: CSR (Client-Side Rendering)

All HTML is generated in the user's browser via React's virtual DOM reconciler. The server delivers a minimal index.html shell (the Vite build output) that bootstraps the React application. Routing is handled entirely in the browser by React Router v7.

WHY CSR OVER SSR:

1. Browse-Mart's SEO requirements are secondary to functionality at this stage. Product pages are not indexed by search crawlers in the current deployment configuration — this is acknowledged as a growth limitation.
2. Vercel SSR (Next.js) would require migrating from Vite, which adds build complexity without clear short-term benefit.
3. The React 19 concurrent renderer provides sufficient perceived performance through streaming Suspense boundaries even in CSR mode.

SSR MIGRATION PATH (future):

Migrating to Next.js App Router would enable server components for product pages (zero JS for static content), generateStaticParams for popular product/category pages, and incremental static regeneration for the homepage. This would address both SEO limitations and initial paint time.

4.2 Application Bootstrap Sequence

None

Browser loads index.html



Vite bundle loaded (main.jsx entry point)



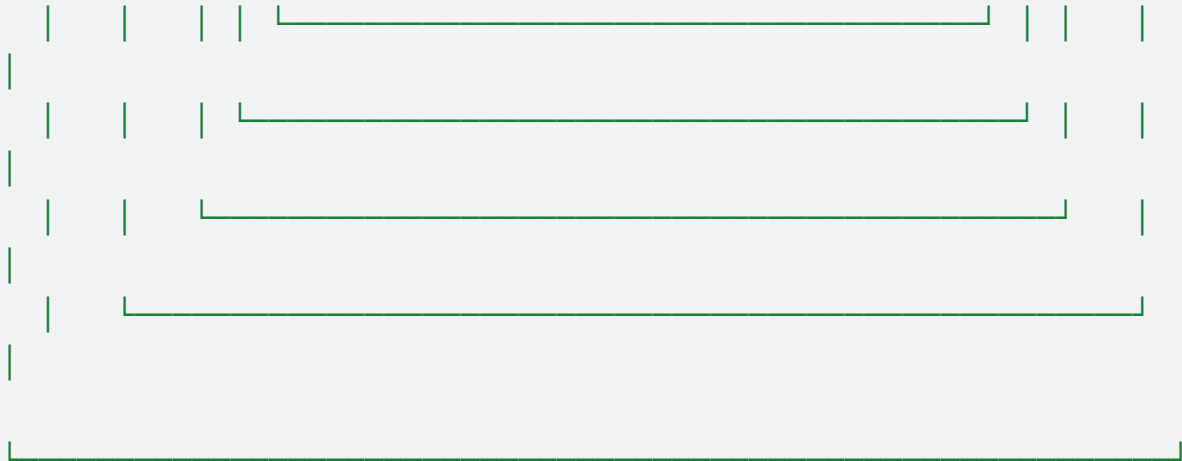
React root created with StrictMode



Provider tree mounted (order matters):

```
| ThemeProvider
```

```
|
```

The `CategoryProvider` is the only provider that performs a network request on mount. This means category data is available globally and **synchronously** (after the single fetch completes) to any component that calls `useCategory()`.

4.3 State Management Architecture

Browse-Mart uses React's built-in **Context API** for all global state. There is no Redux, Zustand, MobX, or Jotai. This is a deliberate simplicity choice documented in DECISION-006.

The five context domains and their responsibilities:

- **authContext.jsx — Identity Plane**
 - **State:** `authToken` (string | null)
 - **Source:** `localStorage["token"]`
 - **Mutation:** `setAuthToken()` — updates state + `localStorage`
 - **Consumers:** Protected routes, all API calls via Axios headers
 - **Pattern:** Auth token is NOT decoded client-side. The frontend treats it as an opaque credential. Role information is fetched separately via the user profile endpoint.
- **userContext.jsx — Profile Plane**
 - **State:** `userDetail` (object | null)
 - **Source:** Fetched from `/api/user/profile` after login
 - **Mutation:** `setUserDetail()` — used by profile and login flows
 - **Consumers:** Profile pages, role-based navigation, address pre-fill.
- **cartContext.jsx — Commerce Plane**
 - **State:** `cartCount` (integer)
 - **Source:** Returned by cart mutation endpoints
 - **Mutation:** `setCartCount()` — updated on add/remove/update cart operations

- **Pattern:** The count is a scalar optimistic indicator. The actual cart items are NOT stored globally—they are fetched per page visit. This avoids stale cart state across tab sessions.
- **categoryContext.jsx — Catalog Plane**
 - **State:** categories (array), loading (boolean)
 - **Source:** GET /api/category on provider mount
 - **Mutation:** fetchCategories() — manual refresh trigger
 - **Pattern:** Category data is read-heavy and changes rarely. Fetching once on app bootstrap and serving from context for all navigation is correct. No TTL invalidation exists—a hard page refresh or explicit fetchCategories() call is required.
- **themeContext.jsx — Presentation Plane**
 - **State:** theme ("dark" | "light")
 - **Source:** localStorage["theme"]
 - **Mutation:** toggleTheme() — flips and persists
 - **Consumers:** CSS class application, SweetAlert2 theming, component styles.

MISSING STATE DOMAINS (Gaps):

There is no global notification state, no order state, and no seller context. These data types are fetched per-page, which is correct for correctness (always fresh) but creates redundant fetches when navigating between pages in the same session. A shared SWR or React Query layer would optimize this without the complexity of manual cache management.

4.4 Routing Architecture

React Router v7 handles all client-side navigation. Routes are organized into three role-based route groups:

- **CONSUMER ROUTES** (ProtectedRoute wrapper — requires authToken)
 - / — Homepage (product discovery)
 - /product/:id — Product detail page
 - /cart — Cart management
 - /checkout — Checkout flow
 - /orders — Order history
 - /order/:id — Order detail
 - /profile — User profile
 - /wishlist — Wishlist management
- **SELLER ROUTES** (SellerRoute wrapper — requires authToken + seller userType)
 - /seller/dashboard — Analytics overview
 - /seller/products — Product inventory management

- /seller/orders — Order fulfillment queue
- /seller/customers — Customer insights
- **ADMIN ROUTES** (AdminRoute wrapper — requires authToken + admin userType)
 - /admin/dashboard — Platform statistics
 - /admin/users — User governance
 - /admin/products — Product moderation
 - /admin/categories — Category CRUD
- **PUBLIC ROUTES** (no auth required)
 - /login — Email/password + Google login
 - /register — Consumer registration
 - /forgot-password — Password reset flow
 - /privacy-policy — Legal page
 - /terms-of-service — Legal page

The Vercel SPA rewrite (source: "/(.*)" → destination: "/index.html") ensures deep-link navigation works correctly on page refresh. Without this, a direct browser request to /product/123 would return a 404 from Vercel's static server because no physical file exists at that path.

4.5 Component Architecture Philosophy

Browse-Mart's component structure follows a flat composition model rather than Atomic Design or strict Container/Presentational separation. Components are organized by page/feature rather than by abstraction level.

The practical implication is that some components contain both business logic (API calls, state transformations) and presentation markup. This is pragmatic for a small team but introduces testability challenges because business logic is coupled to the rendering context.

4.6 PWA Architecture

Browse-Mart is a Progressive Web App, meaning it can be installed on devices and operates with offline capability for previously visited resources.

SERVICE WORKER CACHING STRATEGY (via Workbox):

- **API Responses (/api/*):**
 - *Strategy*: StaleWhileRevalidate
 - *Behavior*: Serve from cache immediately (fast response), revalidate in background, update cache with fresh response.
 - *Max Age*: 1 hour (80 entries)
 - *Rationale*: Product listings and categories change infrequently. Serving stale data for seconds is acceptable for perceived performance.
- **Images (Cloudinary URLs):**

- *Strategy*: CacheFirst
- *Behavior*: Serve from cache if available; only fetch from network on cache miss.
- *Max Age*: 30 days (120 entries)
- *Rationale*: Product images are immutable by URL. This aggressive strategy is safe because image URLs change when images are replaced.
- **Static Assets (JS/CSS/Fonts)**:
 - *Strategy*: CacheFirst (30-day expiry)
 - *Rationale*: Vite produces content-hashed filenames on each build, meaning there is no risk of serving stale code.
- **Navigation (HTML)**:
 - *Strategy*: navigateFallback: '/index.html'
 - *Behavior*: Any navigation request that cannot be fulfilled by the cache or network falls back to the index shell, enabling offline SPA operation.

PWA INSTALLATION:

The web manifest defines the app's standalone display mode, theme colors, and icon set. Mobile browsers will trigger the "Add to Home Screen" prompt when the manifest is detected.

SERVICE WORKER AUTO-UPDATE:

The service worker is configured to self-update without prompting the user, ensuring new deployments propagate automatically when users revisit the site.

4.7 Frontend Performance Architecture

Vite's production build optimizes the delivery pipeline via:

- Code-split chunks (vendor, app, and per-route lazy-loaded chunks).
- Content-hashed filenames for long-term browser caching.
- Minified and tree-shaken JavaScript for dead code elimination.
- CSS purging via Tailwind's JIT mode to include only used utility classes.

LAZY LOADING:

React Router's `lazy()` API enables route-based code splitting. Major routes like product pages and admin panels are loaded only when navigated to.

IMAGE OPTIMIZATION:

Product images are served via Cloudinary's CDN. Implementing responsive image transformations via the URL API would significantly reduce mobile bandwidth consumption.

4.8 Frontend Observability

Currently, the frontend has no error tracking integration. Errors are filtered in `main.jsx` via the following listener:

JavaScript

```
window.addEventListener('error', (e) => {  
  if (e.message.includes('Extension context')) return;  
  // default behavior  
});
```

The absence of reporting infrastructure means client-side JavaScript errors in production remain invisible to the engineering team.

SECTION 05: BACKEND ARCHITECTURE

5.1 Server Initialization

The backend entry point (`server.js`) performs the following bootstrap sequence:

1. Express application instantiation.
2. Global middleware registration (CORS, JSON parser, URL encoder).
3. Route handler registration where all domain routers are mounted.
4. MongoDB connection establishment via `MONGO_URI`.
5. Conditional TCP listener setup for local development.

The Vercel serverless adapter imports and re-exports the Express app as the default export. Each incoming request invokes the app as a standard HTTP handler.

COLD START IMPLICATION: Idle functions trigger a cold start on the first request, re-running module code and re-establishing DB connections. While connection pooling mitigates subsequent request latency, Atlas connection limits remain a scaling concern under high load.

5.2 Modularization Philosophy

Each functional concern lives in its own file within a clearly named directory structure:

None

`src/`


```
|— config/           - Environment configuration (Cloudinary
setup)
|— controllers/      - Request handlers (one file per
operation)
|   |— authController/
|   |— userController/
|   |— productController/
|   |— orderController/
|   |— sellerController/
|   |— adminController/
|   |— categoryController/
|   |— reviewController/
|— middleware/       - Cross-cutting concerns (auth, status,
OTP, upload)
|— models/           - Mongoose schema definitions
|— routes/           - Express Router definitions
|— services/         - External service integrations (email,
OTP, Cloudinary)
|— utils/            - Pure utility functions (ID generation,
helpers)
|— server.js         - Application entry point
```

This structure enforces the Single Responsibility Principle at the file level. Each controller handles exactly one HTTP operation. This makes the codebase navigable: any engineer can find the code for any operation in $O(1)$ time.

5.3 Controller Design Pattern

Controllers in Browse-Mart follow a consistent pattern:

JavaScript

```
async function operationName(req, res) {
  // 1. Extract inputs from req (body, params, query, user
  from middleware)
```

```
// 2. Validate inputs (required fields, type checks,
business rules)
// 3. Perform business logic (DB queries, transformations)
// 4. Execute side effects (email, Cloudinary, cart updates)
// 5. Return response (JSON with success flag, data,
message)
}
```

Response Format

- Success: { success: true, data: {...}, message: "..."} }
- Failure: { success: false, message: "..."} }

HTTP Status Codes

- **200** — Success
- **400** — Validation failure / bad request
- **401** — Authentication required
- **403** — Authorization denied (wrong role, blocked status, wrong token type)
- **404** — Resource not found
- **409** — Conflict (duplicate email, existing resource)
- **500** — Unhandled server error

5.4 Service Layer Architecture

Services are thin wrappers around external dependencies with a clean interface. This decouples controllers from implementation details of external APIs.

- `emailService.js`: Encapsulates Nodemailer transporter creation and `sendMail` invocation. Returns boolean (true = sent, false = failed). Controllers check return value but do not retry on failure. **RISK**: A single failure does not queue for retry. OTP emails that fail leave users unable to verify, with no automated recovery.
- `otpService.js`: Generates cryptographically-random 6-digit OTP and `bcrypt`-hashes it before returning for storage. Sends the plaintext OTP via `emailService`. This separation is clean — the controller stores the hash, the service sends the plaintext.
- `cloudinary.js`: Contains config and upload utilities. `uploadImageToCloudinary` streams a Buffer through Cloudinary's API (critical for serverless). `deleteImageFromCloudinary` extracts the `public_id` from the URL. **RISK**: This extraction is fragile — if Cloudinary's URL format changes, it breaks silently.

- `sendOrderConfirmationEmail.js`: Composes HTML email body from order data and delegates to `emailService` for delivery. Called synchronously during `submitOrder`; if it throws, the order is not affected but the user gets no email.

5.5 Multer File Upload Integration

File uploads use Multer with **MemoryStorage** configuration. This means:

- File buffers are held in process memory (not written to disk)
- Files are accessible as `req.files` (array) or `req.file` (single)
- Buffers are passed directly to Cloudinary's streaming uploader
- No temp file cleanup required

MEMORY PRESSURE: Large file uploads (high-resolution product images) consume process memory during upload. In a serverless environment with 1024MB RAM limits, a single request uploading 10x 5MB images consumes 50MB during processing. This is manageable but warrants a file size limit on Multer configuration. Current code should enforce limits (e.g., `limits: { fileSize: 5 * 1024 * 1024 }`) to prevent memory exhaustion attacks.

5.6 Database Interaction Patterns

All database interaction goes through Mongoose. The following patterns are used consistently:

- **FIND BY ID WITH VALIDATION:**

JavaScript

```
const product = await Product.findById(productId);
```

UPSERT PATTERN (not explicitly used — update is preferred):

JavaScript

```
User.findOneAndUpdate({email}, {$set: data}, {upsert: true, new: true})
```

ARRAY MUTATION (embedded documents):

JavaScript

```
user.cart.push({productId, quantity: 1})
```

- adds to array

JavaScript

```
user.cart = user.cart.filter(...)
```

- removes from array

JavaScript

```
user.save()
```

- persists entire document

PARALLEL QUERIES:

JavaScript

```
const [users, sellers, products] = await Promise.all([
  User.countDocuments({userType: "consumer"}),
  SellerUser.countDocuments(),
  Product.countDocuments()
])
```

AGGREGATION PIPELINE (seller dashboard):

JavaScript

```
Order.aggregate([
  { $match: { sellerId } },
  { $group: { _id: null, totalRevenue: { $sum: "$grandTotal" } } }
])
```

LEAN QUERIES (performance):

JavaScript

```
Product.find({sellerId}).lean()
```

Returns plain JavaScript objects instead of Mongoose documents, skipping instance method hydration. Used in seller product listings where Mongoose getters/setters are not needed.

5.7 Transaction Handling

Browse-Mart does **NOT** use MongoDB multi-document transactions. All operations are single-document atomic (leveraging MongoDB's document-level atomicity) or sequential multi-document operations without rollback.

This is the most significant **reliability gap** in the backend architecture.

CRITICAL SCENARIO — Order Submission:

1. Decrease stock for each product (multiple `Product.save()` calls)
2. Create Order document per seller
3. Add `order._id` to `User.order` array and save User
4. Send confirmation email
5. Clear cart (`user.cart = []`)

If Step 2 fails after Step 1 completes, stock has been decremented but no order exists. The inventory is incorrect and the user has no order. If Step 3 fails after Step 2 completes, orders exist in the DB but the user has no reference to them.

MITIGATION (current): None. The code uses try/catch with error logging, but there is no compensation logic.

CORRECT APPROACH: MongoDB 4.x+ multi-document transactions should be used for the order submission flow:

JavaScript

```
const session = await mongoose.startSession();
session.startTransaction();
try {
  // All writes inside session
  await session.commitTransaction();
} catch (e) {
  await session.abortTransaction();
  throw e;
} finally {
  session.endSession();
}
```

Note: MongoDB transactions require a replica set (not standalone). MongoDB Atlas clusters are replica sets by default, so this is available without infrastructure changes.

5.8 Concurrency Model

Node.js is single-threaded. Concurrent requests are handled via the event loop — I/O operations (DB queries, HTTP calls) yield the thread while awaiting responses.

Browse-Mart's backend uses `async/await` throughout, correctly yielding on all I/O.

There are no blocking synchronous operations in the hot path.

CONCURRENCY RISK — Inventory:

Two concurrent checkout requests for the same product can both read stock: 1, both pass the stock check, both decrement to 0, and the second decrement results in stock: -1. This is a classic lost-update problem.

CURRENT BEHAVIOR: `decreaseProductQuantity` uses:

JavaScript

```
product.stock = product.stock - quantity
product.save()
```

This is **NOT** atomic. The decrement should use MongoDB's atomic increment operator.

CORRECT BEHAVIOR: The atomic increment operator is:

JavaScript

```
Product.updateOne(
  { _id: productId, stock: { $gte: quantity } },
  { $inc: { stock: -quantity } }
)
```

This operation is atomic at the document level and will naturally prevent negative stock without application-level locking.

SECTION 06: AUTHENTICATION & AUTHORIZATION ARCHITECTURE

6.1 Authentication Architecture Overview

Browse-Mart implements a multi-modal authentication system supporting three primary flows: Email + Password (bcrypt-hashed and OTP-verified), Google OAuth2 (social login with JWT issued on callback), and OTP-gated operations for password resets and email verification. The system is stateless with no server-side session store; identity is asserted on every request via a JWT Bearer token.

6.2 JWT Token Architecture

Token Structure and Lifecycle

JSON

```
Header : { "alg": "HS256", "typ": "JWT" }
Payload : { "_id": userId, "email": userEmail, "iat": issued,
"exp": expiry }
Signature: HMAC-SHA256(base64(header) + "." + base64(payload),
JWT_SECRET)
```

Tokens are issued at login, Google OAuth callback, or email verification and have an expiry of 7 days (604,800 seconds). They are stored in the frontend via `localStorage`—an acknowledged security tradeoff—and transmitted via the `Authorization: Bearer {token}` header on all API requests. Note that token revocation is currently not supported as there is no token blacklist.

Token Verification (userAuthentication middleware)

The middleware uses `jwt.verify(token, JWT_SECRET)` to decode the payload and fetch the full User document from the database, which is then attached to `req.user` for downstream controllers. This ensures that `req.user` always reflects the current database state rather than stale JWT claims. While this introduces the cost of one additional database query per authenticated request, it allows the `checkUserStatus` middleware to immediately catch blocked accounts without needing complex token invalidation infrastructure.

6.3 Password Reset Flow

SEQUENCE:

None

```
Client: POST /api/auth/forget-password {email}
  └─ Server: Generate OTP → bcrypt hash → store in User.otp
           Set User.otpExpireAt = now + 10min
           Send OTP email via Nodemailer
  └─ Client receives: {success: true, message}

Client: POST /api/auth/verify-otp {email, otp}
  └─ Middleware (otpVerification):
       Fetch User by email
```

```
    bcrypt.compare(providedOtp, user.otp)
    Check: now < user.otpExpireAt
    If valid: clear OTP fields, attach user to req
└─ Controller (verifyOtp):
    Issue RESET TOKEN (JWT, 10-minute expiry, email claim)
    Return resetToken to client
```

Client: POST /api/auth/set-password {resetToken, newPassword, confirmPassword}

```
└─ Server: jwt.verify(resetToken, JWT_SECRET)
    Extract email from decoded token
    Fetch User by email
    Validate passwords match, length >= 6
    Update user.password (pre-save hook hashes)
    Clear user.otp, user.otpExpireAt
    Send confirmation email
```

SECURITY PROPERTIES:

- OTP is never stored in plaintext (bcrypt hash at rest).
- Reset token is scoped to password-reset only (10-minute window).
- OTP cleared after verification (single-use).
- Confirmation email on success (alerts user of change).

ATTACK SURFACE:

- No rate limiting on /forget-password — OTP bombing is possible.
- bcrypt compare on every OTP attempt is expensive (deliberate, slows brute force) but could be used as a CPU DoS vector.
- 6-digit OTP has 1,000,000 possibilities; 10-minute window with no attempt limiting means ~10,000 guesses possible. Rate limiting is essential for production hardening.

6.4 Google OAuth2 Flow

SEQUENCE:

None

Client: GET /api/auth/google

└─ Server: Create fresh oauth2Client (per-request, Vercel-safe)

Generate authorization URL (offline access, profile/email scopes)

└─ Client: Redirect browser to Google authorization URL

User: Authenticates with Google, approves scopes

Google: Redirects to callback URL with authorization code

Client: POST /api/auth/google/callback {code}

└─ Server: oauth2Client.getToken(code) → tokens

Set credentials on oauth2Client

Fetch user info from Google userinfo endpoint

Upsert User: find by googleId or email

If new user: create with hasPassword: false

If existing email user: link googleId, skip password override

Issue JWT → return to client

DESIGN DECISION — Per-Request OAuth Client:

A new `google.auth.OAuth2` instance is created on every request to `/api/auth/google`. This is intentional for Vercel serverless environments. A module-level singleton `oauth2Client` would have its credentials mutated across concurrent serverless invocations, causing token cross-contamination between users. Per-request instances are stateless and safe.

6.5 OTP Architecture

OTP GENERATION:

None

`Math.floor(100000 + Math.random() * 900000)`

→ 6-digit integer in range [100000, 999999]

→ Converted to string

→ `bcrypt.hash(otp, 10)` → stored hash

DESIGN CONCERN: `Math.random()` is not cryptographically secure (it uses a pseudo-random number generator). For security-critical OTPs, the standard is `crypto.randomInt(100000, 999999)` from Node.js's built-in `crypto` module, which uses the OS-level CSPRNG. This is a security recommendation that should be remediated.

OTP VALIDATION (`otpVerification` middleware):

1. Extract email + otp from request body.
2. Fetch User document by email.
3. Check `user.otp` exists (not cleared).
4. `bcrypt.compare(providedOtp, user.otp)`.
5. Check `new Date() <= user.otpExpireAt`.
6. If valid: clear otp + `otpExpireAt` fields, set `req.user`.
7. Call `next()` → controller runs.

MIDDLEWARE COMPOSITION:

None

The `otpVerification` middleware is composed into specific routes:

```
POST /api/auth/email-verification → [otpVerification,
verifyOTPForRegistration]
POST /api/auth/verify-otp → [otpVerification, verifyOtp]
```

This middleware composition pattern means OTP-gating any endpoint is a single middleware addition rather than duplicated logic.

6.6 Authorization Architecture

Authorization is enforced through a **layered middleware stack**:

- **LAYER 1 — Authentication Check** (`userAuthentication`):
 - Validates JWT exists and is not expired.
 - Attaches `req.user` (full User document).
 - All authenticated endpoints start here.
- **LAYER 2 — Status Check** (`checkUserStatus`):
 - Performs a **FRESH DB lookup** of user status.

- Blocks blocked users (`status === "blocked"`).
- **Critical:** Prevents the 7-day JWT window from allowing blocked users to continue operating. Without this layer, blocking a user is only effective when their JWT expires (up to 7 days later).
- **LAYER 3A — Seller Authorization** (`sellerAuthentication`):
 - Checks `req.user.userType === "seller"`.
 - Checks `req.user.sellerId` is populated.
 - A user who registered as a consumer and changed their `userType` via direct DB manipulation would still be blocked if `sellerId` is absent.
- **LAYER 3B — Admin Authorization** (`adminAuthentication`):
 - Checks `req.user.userType === "admin"`.
 - Applied after `checkUserStatus` (admin routes are 3-layer protected).

RBAC MODEL:

Browse-Mart implements a simple Role-Based Access Control (RBAC) model with three roles:

- **consumer:** Default role, can purchase products.
- **seller:** Can list/manage products, view own orders/customers.
- **admin:** Full platform access, no user-defined scope restrictions.

There is no ABAC (Attribute-Based Access Control)—a seller cannot be granted granular permissions beyond the seller role boundary. This is appropriate for the current feature set.

6.7 Session vs. Token Security Tradeoffs

Storing **JWT** in **localStorage** (as Browse-Mart does) is a known **XSS risk**. If an attacker injects malicious **JavaScript** (via a stored **XSS vulnerability**), they can read `localStorage["token"]` and exfiltrate the token.

The alternative — **HttpOnly cookie** storage — prevents JavaScript access but requires **CSRF protection** and complicates the **CORS configuration**.

CURRENT POSTURE: **localStorage** storage with the assumption that **CSP headers** and input sanitization prevent **XSS injection**. No **CSP headers** are currently configured — this is the primary security gap to address.

RECOMMENDED HARDENING:

1. Set **HttpOnly**, **Secure**, **SameSite=Strict** cookies for JWT storage
2. Implement **CSRF token** for state-mutating requests
3. Configure **Content-Security-Policy** headers
4. Implement token refresh mechanism (short-lived access token + long-lived refresh token in **HttpOnly cookie**)

SECTION 07: DATABASE DESIGN & SCHEMA ENGINEERING

7.1 Data Modeling Philosophy

Browse-Mart's MongoDB schema design follows the "embed for locality, reference for growth" principle.

EMBED when:

- Data is always accessed with the parent (e.g., cart with user).
- Data cardinality is bounded and small (e.g., reviews per product).
- Data is exclusive to one parent (e.g., shipping address on user).

REFERENCE when:

- Data has independent identity and lifecycle (e.g., Product, Order).
- Data is shared across multiple parents (e.g., Category referenced by Product).
- Data grows unbounded over time (e.g., orders referenced by User._id array).

7.2 User Schema Deep Dive

User Document Structure:

```
JSON
{
  _id: ObjectId,
  name: String,
  email: String (unique, lowercase, trimmed),
  password: String (bcrypt hash, nullable),
  googleId: String (nullable),
  hasPassword: Boolean,
  userType: "consumer" | "seller" | "admin",
  sellerId: ObjectId (ref: SellerUser, nullable),
  phoneNumber: String,
  address: String,
  profilePic: String (URL),
  cart: [{ productId: ObjectId, quantity: Number }],
  wishlist: [{ productId: ObjectId }],
```

```
order: [ObjectId] (ref: Order),
shippingAddress: {
  addressLine1: String, addressLine2: String, state: String,
  city: String, pinCode: String, country: String
},
otp: String (bcrypt hash, nullable),
otpExpireAt: Date (nullable),
isVerified: Boolean, status: "active" | "blocked",
createdAt: Date, updatedAt: Date
}
```

Embedded Arrays — Design Analysis

Cart Array:

- **Read Pattern:** Fetched on every cart page visit (populated query).
- **Write Pattern:** Push/filter on every add/remove operation.
- **Bounded:** Consumer carts are limited by practice (~50 items max).
- **Concern:** Embedded carts mean every operation reads/writes the entire User document. Large order histories can cause bloat.
- **Recommendation:** Extract to a separate collection at scale to decouple from user document size.

Wishlist Array:

- Similar concerns to cart regarding document bloat.
- **Pagination:** Currently done in-memory after fetching the full User document. Users with 1000+ items will cause performance degradation.

Order Array:

- Stores only ObjectId references, which is correct for independent lifecycles.
- **Concern:** Grows unboundedly. Queries should use `Order.find({customerId})` instead of population to avoid loading the full array.

7.3 Product Schema Deep Dive

Product Document Structure:

JSON

```
{
  _id: ObjectId,
  name: String, price: Number,
  mrpPrice: Number, sellingPrice: Number,
  description: String, image: [String] (Cloudinary URLs),
  category: ObjectId (ref: Category, required),
  subCategory: ObjectId (ref: Category, nullable),
  brand: String, stock: Number, inStock: Boolean,
  isHide: Boolean (soft-delete flag),
  review: [{
    rating: Number, title: String, message: String,
    userId: ObjectId, userName: String, createdAt: Date
  }],
  userID: ObjectId (ref: User), sellerId: ObjectId (ref:
User),
  visibilityState: Boolean, createdAt: Date, updatedAt: Date
}
```

Pricing Model Analysis

Three price fields exist: price, mrpPrice, and sellingPrice.

- **mrpPrice:** Manufacturer's Recommended Price (displayed as original price, struck through).
- **sellingPrice:** Actual checkout price.
- **price:** Likely an alias; usage is inconsistent technical debt.
- **Risk:** Multiple price fields create potential display vs. checkout discrepancies.

Embedded Reviews

Reviews are currently embedded in the Product document, which presents several scaling issues:

- No duplicate prevention (users can review multiple times).
- Unbounded array growth with product popularity.
- Sorting/filtering requires loading all reviews into memory.
- **Recommendation:** Extract to a separate collection at scale.

Soft Delete and Visibility

- **isHide:** When true, hides product from public listings but preserves it for order historical integrity.

- **visibilityState:** A secondary flag with unclear architectural distinction (technical debt).

7.4 Order Schema Deep Dive

Order Document Structure:

JSON

```
{
  "_id": "ObjectId",
  "orderId": "String (unique)",
  "customerId": "ObjectId (ref: User)",
  "sellerId": "ObjectId (ref: User)",
  "orderItems": [{
    "productId": "ObjectId",
    "productName": "String (snapshot)",
    "price": "Number (snapshot)",
    "mrpPrice": "Number (snapshot)",
    "sellingPrice": "Number (snapshot)",
    "quantity": "Number"
  }],
  "orderStatus": "String",
  "grandTotal": "Number",
  "shippingAddress": { "...": "snapshot" },
  "paymentStatus": "String"
}
```

Snapshot Design Pattern

- Order items copy details at order time to ensure historical accuracy if prices change or products are deleted.
- Shipping addresses are snapshotted so past orders reflect the destination at the time of purchase.

Order ID Generation

- Current format: "ORD-YYMMDD-HHMMSS-XXXXXXXX".
- **Risk:** Potential collisions under high concurrency.
- **Recommendation:** Use MongoDB ObjectIds or UUID v4 for global uniqueness.

7.5 Category Schema

JSON

```
{
  "_id": "ObjectId",
  "name": "String (unique)",
  "description": "String",
  "parentCategory": "ObjectId | null (ref: Category)"
}
```

Hierarchy Analysis

- **Self-Referential Hierarchy:** Supports a single-level tree (main category to sub-category).
- **Constraint Enforcement:** Current logic prevents category reorganization after creation to maintain tree consistency.

7.6 Indexing Strategy

Current Unique Indexes:

- User: email
- Order: orderId
- Category: name

Missing Performance Indexes (High Priority):

- **Product:** sellerId, category, isHide, and text indexes for search.
- **Order:** customerId, sellerId, orderStatus, and createdAt (for sorting).

7.7 Migration Strategy

- **Current State:** No formal framework; relies on MongoDB's schemaless nature.
- **Recommendation:** Adopt 'migrate-mongo' to handle renames, type changes, and large-scale reindexing safely.

SECTION 08: DOMAIN MODELING & BOUNDED CONTEXTS

8.1 Domain Map

Browse-Mart's business domains and their data ownership:

None

IDENTITY DOMAIN

Owns: User, SellerUser

Core concepts: registration, authentication, role assignment, seller onboarding, account status

CATALOG DOMAIN

Owns: Product, Category

Core concepts: product creation, pricing, categorization, visibility management, image assets, reviews

COMMERCE DOMAIN

Owns: Order, Cart (embedded in User)

Core concepts: cart management, checkout, order lifecycle,
payment recording, shipping, pricing calculation

NOTIFICATION DOMAIN

Owns: Email templates (no persistent model)

Core concepts: transactional email delivery, OTP
communication, |
order confirmation, password reset confirmation

ANALYTICS DOMAIN (incipient)

Owns: Aggregation over Order, Product, User

Core concepts: seller revenue, customer count, product
performance, |

```
| admin platform statistics
```

8.2 Domain Event Flows (Conceptual)

While Browse-Mart does not implement an explicit event bus, the following domain events are implicit in the system and would form the event backbone if the system were to adopt event-driven architecture:

- . **UserRegistered**
 - . Triggered by: POST /api/auth/register success
 - . Side effects: Send OTP email
- . **UserVerified**
 - . Triggered by: POST /api/auth/email-verification success
 - . Side effects: Issue JWT
- . **OrderPlaced**
 - . Triggered by: POST /api/order/submit-order success
 - . Side effects: Decrement stock, send order confirmation email, add to user.order array
- . **OrderStatusChanged**
 - . Triggered by: PATCH /api/seller/orders/:id/status
 - . Side effects: (currently none — could notify customer via email)
- . **ProductCreated / ProductUpdated / ProductDeleted**
 - . Triggered by: Seller product management operations
 - . Side effects: Cloudinary image operations
- . **UserBlocked**
 - . Triggered by: PATCH /api/admin/users/:id/status {status: "blocked"}
 - . Side effects: (currently none — active JWTs remain valid for up to 7 days)

8.3 Multi-Seller Order Architecture

A consumer's cart may contain products from multiple sellers. When an order is submitted, the system groups cart items by sellerId and creates one Order document per seller. This is the correct architecture for a marketplace model:

None

Consumer Cart:

```
[productA (seller1), productB (seller1), productC (seller2)]  
|  
└─ Submit Order  
    |  
    └─ Order 1 (seller1): [productA, productB]  
    └─ Order 2 (seller2): [productC]
```

SELLER ISOLATION REASONING:

- Sellers need their own order view — they should only see orders containing their products.
- Shipping, fulfillment, and status updates are per-seller.
- Revenue reporting is per-seller (each seller sees their own revenue).
- If seller1 ships early and seller2 ships late, order statuses diverge. Combined orders would force a complex status aggregation.

GRAND TOTAL CONCERN:

When multiple orders are created from a single cart, each order has its own grandTotal. The consumer's checkout total is the sum of all created orders' grandTotals. Currently, the consumer UI must sum these to display a total. This should be surfaced via a cart summary calculation before order submission.

PART II — CORE COMMERCE ENGINE

SECTION 09: ORDER MANAGEMENT SYSTEM

9.1 Order Lifecycle

An order in Browse-Mart traverses a 7-state linear state machine:

None

order placed -> order confirmed -> processing -> shipped -> out for delivery -> delivered / cancelled

State Transition Ownership:

1. Sellers control all state transitions via PATCH /api/seller/orders/:id/status.
2. Consumers have no ability to advance order status (they can only view it).
3. Admins have no order status mutation endpoint.

Terminal State Enforcement:

The updateOrderStatus controller checks:

JavaScript

```
const lockedStatuses = ["delivered", "cancelled"];  
if (lockedStatuses.includes(order.orderStatus)) return 400;
```

9.2 Order Submission Flow (Critical Path)

The submitOrder.js execution path follows these steps:

1. Validate cart is non-empty.
2. Decrease product quantities in parallel via Promise.all().
3. Group cart items by sellerId.
4. Calculate totals, apply shipping, and create Order documents.
5. Save default shipping address if applicable.
6. Clear user cart and persist the user document.

9.3 Order Data Integrity Gaps

GAP 1: Atomic Stock Deduction

To prevent lost updates, use MongoDB atomic findOneAndUpdate with a precondition:

JavaScript

```
Product.findOneAndUpdate(  
  { _id, stock: { $gte: quantity } },  
  { $inc: { stock: -quantity } },  
  { new: true }  
)
```

GAP 2: Multi-Document Transaction

The entire flow should be wrapped in a MongoDB session transaction to ensure all-or-nothing atomicity.

GAP 3: Cart Recovery

Order confirmation emails that fail silently are not queued for retry.
An email delivery retry queue would address this at low scale.

9.4 Order Query Architecture

Consumer View:

Queries `Order.find({ _id: { $in: user.order } })`. Scaling requires direct query by `customerId`.

Seller View:

Queries `Order.find({ sellerId })`. Needs server-side pagination for high-volume sellers.

9.5 Order Status Notification Gap

Production systems should notify consumers of status changes via email, push, or in-app badges.

=====

SECTION 10: CART & CHECKOUT ARCHITECTURE

=====

10.1 Cart Data Model

The cart is stored as an embedded array in the User document:

JSON

```
user.cart = [  
  { productId: ObjectId, quantity: Number },
```

```
{ productId: ObjectId, quantity: Number }  
]
```

This model has distinct tradeoffs:

ADVANTAGES:

- Single DB query to load user + cart (no join)
- Cart is always colocated with user identity
- Cart naturally scoped to one user
- Cart persists across sessions (not ephemeral like localStorage-only carts)

DISADVANTAGES:

- All cart operations require loading the full User document
- Concurrent cart updates from multiple sessions/tabs could cause write conflicts on the User document
- Cart analytics (abandoned carts) require scanning all User documents
- Cannot easily share or transfer carts between users

10.2 Cart Operations

ADD TO CART (addToCart.js)

Logic:

JavaScript

```
const existingItem = user.cart.find(item =>  
  item.productId.toString() === productId);  
if (existingItem) {  
  existingItem.quantity += 1;  
} else {  
  user.cart.push({ productId, quantity: 1 });  
}  
await user.save();  
return user.cart.length;
```

Edge Cases:

- No maximum quantity per item constraint exists.
- Stock levels are not validated at add-to-cart time (only during checkout).
- Product existence/visibility is not verified (hidden products can be added via direct ID).

UPDATE CART (updateCart.js)

UPDATE CART (updateCart.js)

Logic:

JavaScript

```
if (quantity === 0) {
  user.cart = user.cart.filter(item =>
    item.productId.toString() !== productId);
} else if (quantity > 0) {
  const item = user.cart.find(item =>
    item.productId.toString() === productId);
  if (item) item.quantity = quantity;
} else {
  return res.status(400).json({ message: "Invalid quantity"
});
}
await user.save();
```

DESIGN NOTE: Quantity 0 as the remove signal is a reasonable API design.
This avoids a separate DELETE endpoint.

GET CART ITEMS (getCartItems.js):

Logic:

For each item in user.cart:

Fetch product data via Product.findById(item.productId)

Merge item quantity with product fields

Return enriched cart array

PERFORMANCE CONCERN: This issues N separate Product.findById() queries
for N cart items (N+1 query problem). Fix:

```
const productIds = user.cart.map(i => i.productId);
```

```
const products = await Product.find({ _id: { $in: productIds } });
```

```
// merge in application layer
```

10.3 Checkout Flow

CHECKOUT SEQUENCE:

1. Consumer reviews cart (GET cart items)
2. Consumer selects/confirms shipping address
3. Consumer selects payment method
4. Consumer submits order (POST /api/order/submit-order)
5. Server validates, decrements stock, creates orders, clears cart

6. Consumer receives order confirmation (email + UI response)

PRICING CALCULATION AT CHECKOUT:

For each seller group: $\text{totalMrpPrice} = \sum(\text{item.mrpPrice} * \text{item.quantity})$,
 $\text{totalSellingPrice} = \sum(\text{item.sellingPrice} * \text{item.quantity})$, $\text{totalDiscount} = \text{totalMrpPrice} - \text{totalSellingPrice}$, $\text{shippingCharge} = \text{totalSellingPrice} < 1000 ? \text{totalSellingPrice} * 0.1 : 0$,
 $\text{grandTotal} = \text{totalSellingPrice} + \text{shippingCharge}$.

SHIPPING LOGIC: 10% of order value for orders under Rs.1000, free above. This is a hardcoded business rule. Changing thresholds requires code deployment -- it should be configurable via admin settings.

10.4 Checkout Race Conditions

SCENARIO: Flash Sale Inventory Race

Two users add the last unit of Product X to their carts. Both submit orders simultaneously.

CURRENT BEHAVIOR:

Request A reads: $\text{product.stock} = 1$, Request B reads: $\text{product.stock} = 1$, Request A writes: $\text{product.stock} = 0$, Request B writes: $\text{product.stock} = -1$. Both orders are created. Product has negative stock.

CORRECT BEHAVIOR:

Use MongoDB atomic update with precondition: `db.products.updateOne({ _id: productId, stock: { $gte: 1 } }, { $inc: { stock: -1 } })`. If 0 documents updated: fail with "Product out of stock" error.

SCENARIO: Price Change After Cart Addition

A seller updates a product price after a consumer has added it to cart. The cart stores only productId + quantity (not price snapshot). At `getCartItems` time, it fetches current price. This is correct behavior but no cart price-change notification exists.

=====

=====

SECTION 11: INVENTORY CONSISTENCY DESIGN

12.1 Current Inventory Model

Inventory state is stored on the Product document using two fields: `stock` (an absolute unit count) and `inStock` (a derived boolean flag that is true if $\text{stock} > 0$).

The `inStock` boolean is redundant with `stock`. Maintaining two fields creates consistency risk—they can diverge if one is updated without the other. The correct approach is to use a Mongoose virtual:

```
productSchema.virtual('inStock').get(function() { return this.stock > 0; });
```

12.2 Stock Decrement Flow (Current)

The current decreaseProductQuantity logic decrements `product.stock` and manually updates the `inStock` flag before calling `await product.save()`.

PROBLEMS:

- Non-atomic read-modify-write (concurrent race condition).
- No check that `stock >= quantity` before decrement (negative stock possible).
- No reservation mechanism (allows oversell).

12.3 Inventory Reservation Pattern (Recommended)

A robust inventory system uses two-phase reservation to prevent oversell while allowing the shopping experience to complete before payment:

- **PHASE 1 — Reserve (at checkout initiation):** Increment `reservedStock` and calculate `availableStock (stock - reservedStock)`.
- **PHASE 2 — Commit (at successful payment):** Decrease `stock` and `reservedStock` by the purchased quantity.
- **PHASE 3 — Release (on cart expiry or order cancellation):** Decrease `reservedStock` to return units to availability.

12.4 Cancellation Stock Restoration

When an order is cancelled, stock is NOT currently restored. A fix is required in the `updateOrderStatus` controller to iterate through `order.orderItems` and use `$inc` to restore stock levels in the Product collection.

12.5 Stock Reporting

The seller product listing supports stock status filtering based on units: "in stock" (`> 20`), "low stock" (`0 < stock <= 20`), and "out of stock" (`0`). The "low stock" threshold of 20 units is currently hardcoded and should be made configurable per seller preference.

SECTION 12: CACHING STRATEGY

15.1 Current Cache Architecture

Browse-Mart currently has NO server-side caching layer; every request queries MongoDB directly. The only caching present is the PWA Service Worker cache (client-side, Workbox).

15.2 Server-Side Caching Requirements

- **CATEGORY LIST:** 1-hour TTL, invalidated on admin operations.

- **PRODUCT LISTINGS:** 5-15 minute TTL, invalidated on seller updates.
- **PRODUCT DETAIL:** 30-minute TTL, invalidated on specific product updates.
- **SELLER DASHBOARD STATS:** 5-minute TTL for acceptable analytics staleness.

15.3 Cache Implementation Pattern

The recommended stack is Upstash Redis (serverless-compatible). Implementation should follow the Cache-Aside pattern where the application checks Redis before querying MongoDB and sets the cache on misses. Invalidation must be explicit on data updates.

15.4 HTTP Cache Headers

No Cache-Control headers are currently set. Recommended headers include: **public, max-age=300** for listings, **public, max-age=1800** for details, and **private, no-store** for authenticated user data.

SECTION 13: SEARCH ARCHITECTURE

16.1 Current Search Implementation

Product search is implemented as a MongoDB regex query:

JavaScript

```
Product.find({
  $or: [
    { name: { $regex: searchQuery, $options: 'i' } },
    { description: { $regex: searchQuery, $options: 'i' } }
  ]
})
```

Performance Characteristics:

- Regex queries without ^ anchor CANNOT use B-tree indexes.
- They perform a full collection scan.
- For 10,000 products: scans all 10,000 documents per search request.
- Performance degrades linearly with collection size (O(n)).

16.2 Category Search

When a category filter is applied as a string (not ObjectId), the system performs a two-step approach: first, it queries the Category collection for name regex matches to extract matching `_ids`; second, it filters products by `category.$in [matched ids]`. This issues two database queries and is currently not cached.

16.3 Search Architecture Upgrade Path

For production search quality, MongoDB Atlas Search (Lucene-powered) is the natural upgrade given existing Atlas infrastructure. An example Atlas Search Index and Aggregate are shown below:

JSON

```
{
  "mappings": {
    "fields": {
      "name": { "type": "string", "analyzer":
"lucene.standard" },
      "description": { "type": "string", "analyzer":
"lucene.standard" },
      "brand": { "type": "string" }
    }
  }
}
```

JavaScript

```
Product.aggregate([
  {
    $search: {
      index: "product-search",
      text: {
        query: searchQuery,
        path: ["name", "description"],
        fuzzy: { maxEdits: 1 }
      }
    }
  },
  { $limit: 24 }
])
```

Benefits over Regex:

- Inverted index (O(1) lookup vs O(n) scan).
- Relevance scoring (most relevant results first).
- Typo tolerance (fuzzy matching).

- Autocomplete support.
- Faceted filtering (by price range, rating, brand).

16.4 Autocomplete Architecture (Not Implemented)

Currently, no autocomplete or typeahead functionality exists. Implementing this feature would require the following components:

- An Atlas Search autocomplete index on product.name.
- A GET /api/product/suggest?q={prefix} endpoint.
- A client-side debounced input handler with a 300ms delay.
- A Redis cache for popular prefixes with a 10-minute TTL.

SECTION 14: REALTIME SYSTEMS

17.1 Current Realtime Architecture

Browse-Mart has NO realtime communication infrastructure. There are no WebSocket connections, Server-Sent Events (SSE), long-polling mechanisms, Firebase Realtime Database integration, or Pusher/Abyl integration. All data freshness is achieved through polling (client-initiated periodic requests) or explicit page navigation.

- ORDER STATUS UPDATES: Consumers must navigate to the order detail page and manually refresh to see status changes.
- SELLER DASHBOARD: Counts for total orders and revenue are stale from page load time.
- ADMIN DASHBOARD: Platform statistics are static from page load time.

17.2 Realtime Requirements for Production

CONSUMER-FACING REALTIME:

- Order status push: live updates for consumers.
- Cart sync across tabs.
- Live stock alerts during sales.

SELLER-FACING REALTIME:

- In-app notifications for new orders.
- Live dashboard counters for revenue and counts.

ADMIN-FACING REALTIME:

- Real-time platform health monitoring (throughput, errors, activity).

17.3 Realtime Architecture Options

Option A: Server-Sent Events (SSE)

SSE is a unidirectional push channel (server to client) over HTTP. It requires only a single persistent GET connection per client. However, Vercel serverless functions cannot hold persistent connections, requiring a dedicated always-on process or third-party service.

Option B: WebSockets (Socket.io)

This provides bidirectional communication required for features like typing indicators and live chat. It faces the same serverless constraint as SSE, requiring a persistent backend process.

Option C: Third-Party Realtime Service (RECOMMENDED for Vercel)

Using services like Pusher Channels or Ably is the recommended path. The server publishes events to a channel via the SDK, and the client subscribes. This removes the need for a persistent backend process, making it serverless-compatible.

JavaScript

```
pusherServer.trigger('orders', 'status-changed', { orderId, status })
```

SECTION 15: NOTIFICATION SYSTEMS

18.1 Current Notification Channels

Browse-Mart supports two notification delivery channels:

- **EMAIL (Nodemailer + Gmail SMTP):** Used for OTP for registration and password reset, order confirmation on purchase, and password reset confirmation. The current implementation has significant gaps: there is no rate limiting on sends, no delivery tracking or bounce handling, and no retry mechanism on failure.
- **BROWSER PUSH (Web Notifications API):** The infrastructure exists in `notifications.js`, including the permission request flow and a test function (`showExampleNotification()`). However, this is NOT integrated into any business flow and is never triggered automatically.

18.2 Email Architecture Analysis

SMTP DELIVERY VIA GMAIL: Gmail SMTP has sending limits (Free Gmail: 500 emails/day; Google Workspace: 2000 emails/day). For higher volumes, migration to SendGrid, Postmark, or AWS SES is necessary to gain higher limits, delivery analytics, bounce handling, and proper SPF/DKIM configuration. The current risk is that Gmail may flag application emails as spam if sending volume spikes suddenly or if SPF/DKIM records are not configured for the sending domain.

EMAIL TEMPLATE ARCHITECTURE: HTML templates are currently constructed as inline strings within service files. This is not maintainable at scale, as markup is embedded in JavaScript. The templates are not testable in isolation and are not

localizable. The recommended fix is to extract templates to .html or .mjml files, use a template engine (Handlebars, Nunjucks) for variable interpolation, and test rendering independently of email delivery.

18.3 Push Notification Architecture

The browser push infrastructure (`notifications.js`) is present but unused. To integrate push notifications into order status changes, the following steps are required:

1. Request permission during onboarding or order placement confirmation.
2. Subscribe to push service (`PushManager.subscribe()`).
3. Store subscription endpoint in the User document.
4. On order status change: the server sends a push payload to the subscription.
5. The Service Worker push event handler displays the notification.

VAPID KEYS REQUIRED: Browser push requires VAPID key authentication. Keys must be generated and stored as environment variables.

18.4 In-App Notification System (Not Implemented)

No in-app notification center currently exists. Implementing a comprehensive notification system would require:

- Notification document model (`userId`, `type`, `message`, `read`, `createdAt`).
- `POST /api/notifications` endpoint (internal, triggered by events).
- `GET /api/notifications` endpoint (consumer-facing, paginated).
- `PATCH /api/notifications/:id/read` endpoint.
- An unread count badge in the navigation (requiring global state).
- Real-time delivery implemented via SSE/WebSocket/Pusher.

SECTION 16: ADMIN PLATFORM DESIGN

19.1 Admin Capabilities

The admin panel provides four functional areas:

- **DASHBOARD STATISTICS:** Displays total consumers (`User.countDocuments({userType:"consumer"})`), total sellers (`SellerUser.countDocuments()`), total products (`Product.countDocuments()`), and total orders (`Order.countDocuments()`). These statistics are computed via parallel DB count queries on every page load and are not cached.
- **USER MANAGEMENT:** Allows viewing all users with pagination and role/search filters, excluding sensitive fields (`password`, `otp`, `otpExpireAt`). It supports blocking/unblocking users via a status toggle, with protection ensuring admin accounts cannot be blocked.

- **PRODUCT MANAGEMENT:** Allows viewing all products with pagination and filters. It lacks product moderation capability (approve/reject) and lacks the ability to remove malicious listings without seller cooperation, as `deleteProduct` requires seller ownership verification.
- **CATEGORY MANAGEMENT:** Provides full CRUD (Create, Read, Update, Delete) on categories, hierarchical support (main + sub categories), and referential integrity protection (cannot delete if products are attached).

19.2 Admin Authorization Architecture

Admin routes use a three-layer middleware chain: `userAuthentication` -> `checkUserStatus` -> `adminAuthentication`. This architecture ensures:

6. The token is valid.
7. The account is not blocked (preventing a blocked admin from using admin features).
8. The user has the admin role.

19.3 Admin Capability Gaps

The platform is missing several critical admin capabilities:

- **MISSING: Order Intervention:** Admins cannot modify order status. An admin order override capability is required for dispute resolution, especially if a seller abandons orders without updating the status.
- **MISSING: Seller Verification Workflow:** Although `SellerUser` collects KYC data (PAN, GST, trade license, bank details), there is no admin review/approval workflow. All seller registrations are immediately active.
- **MISSING: Content Moderation:** Products can be listed without admin approval. No flagging system, review queue, or automated content policy enforcement.
- **MISSING: Revenue Analytics:** There is no platform-level revenue view. Total GMV, seller performance ranking, category-level sales, and time-series revenue charts are absent.
- **MISSING: Audit Log:** There is no record of admin actions (e.g., who blocked which user, and when). Audit logging is a compliance requirement for most production platforms.

19.4 Admin Seeding

A dedicated `seedAdmin.js` script creates the platform's initial admin account with hardcoded credentials: email: `admin@gmail.com` and password: `adminuser`. This default credential is a critical security issue for production. The seed script must use a strong random password, communicated through a secure channel, and changed immediately after the first login. The hardcoded seed credentials must never reach a production database.

SECTION 17: SECURITY ARCHITECTURE

20.1 Security Posture Overview

Browse-Mart implements several security controls appropriate for its scale. This section analyzes each control's strength and gaps.

20.2 Authentication Security

BCRYPT PASSWORD HASHING

Algorithm: bcrypt with salt rounds = 10. Cost factor: $2^{10} = 1024$ iterations per hash.

Time per hash: ~65ms on modern hardware. Strength: Highly resistant to offline brute-force. Rainbow tables are infeasible due to per-password salting.

RECOMMENDATION: Consider cost factor 12 (250ms) for increased resistance as hardware performance improves.

JWT TOKEN SECURITY

Algorithm: HS256 (HMAC-SHA256 with shared secret). Expiry: 7 days. Secret:

JWT_SECRET environment variable. CONCERN: HS256 uses a single shared secret.

If compromised, all tokens are forgeable. RS256 (asymmetric) would be better for multi-service architectures. For the current monolith, HS256 is acceptable with a strong secret (≥ 256 bits entropy). CONCERN: 7-day lifetime is long. A compromised token is valid for up to 7 days with no revocation. Recommended: Short-lived access token (15 minutes), Long-lived refresh token (7 days) in HttpOnly cookie, and a Refresh endpoint that issues new access tokens.

TOKEN STORAGE (localStorage)

Tokens in localStorage are accessible to JavaScript on the same origin. XSS attacks can exfiltrate them. Mitigation: HttpOnly cookies. Current risk: Medium -- depends on XSS surface area.

20.3 Input Validation & Injection Prevention

MONGOOSE SCHEMA VALIDATION

Mongoose enforces type coercion and required fields. MongoDB does not interpret strings as operators -- injection via string fields is structurally prevented by Mongoose's typed document model. EXCEPTION: Regex patterns from user input can trigger ReDoS (Regular Expression Denial of Service). Search regex should sanitize input: escape special regex characters before use.

NO RATE LIMITING

No request throttling exists. Critical missing protections: /api/auth/login (Brute-force password attacks), /api/auth/forget-password (OTP spam / email bombing), /api/auth/verify-otp (OTP brute-force), and /api/product/get-all-products (Scraping / data harvesting). RECOMMENDED IMPLEMENTATION: Use express-rate-limit middleware to set a window of 15 minutes with a maximum of 10 attempts for authentication endpoints.

20.4 CORS Security

CORS is configured with origin set to FRONTEND_URL (environment variable) and credentials enabled. This is correctly configured to prevent cross-origin credential sharing from arbitrary domains. POTENTIAL MISCONFIGURATION: If FRONTEND_URL is not set in a deployment environment, CORS behavior is undefined; explicit startup validation is recommended.

20.5 Sensitive Data Handling

PAN NUMBER / GST NUMBER

Seller KYC data (panNumber, gstNumber, bankAccountNumber, ifscCode) is stored as plaintext in MongoDB. RECOMMENDATION: Encrypt at application layer using AES-256-GCM before storage, decrypt only when rendering in admin/seller interfaces, and log access to sensitive fields for an audit trail.

EMAIL/PHONE IN LOGS

No explicit log redaction exists. If application errors log req.body, passwords and OTPs in transit could appear in logs. Ensure loggers never log request bodies on authentication endpoints.

20.6 Security Headers

No security HTTP headers are currently set. Required additions via Helmet.js include: Content-Security-Policy (to restrict sources for scripts, styles, and images), X-Content-Type-Options (nosniff), X-Frame-Options (DENY), Strict-Transport-Security, and Referrer-Policy.

20.7 File Upload Security

File uploads go through Multer (memory storage) and then Cloudinary. MISSING controls include file type validation, file size limits, and virus scanning.

RECOMMENDED: Configure Multer with memoryStorage, a 5MB fileSize limit, and a fileFilter to allow only jpeg, png, and webp types.

20.8 Google OAuth Security

The OAuth2 flow uses per-request client instances and server-side code exchange.

CONCERN: The state parameter (CSRF protection) is not validated. RECOMMENDED: Generate a CSRF state token on initiation, store it in a signed cookie, and verify it on callback.

20.9 OTP Security

OTP generation currently uses Math.random(), which is not cryptographically secure.

For security-critical OTPs, the standard is to use crypto.randomInt() from Node.js, which utilizes the OS-level CSPRNG.

SECTION 18: SCALABILITY STRATEGY

23.1 Current Scalability Characteristics

Browse-Mart scales horizontally at the Vercel layer automatically. Vercel spins up additional function instances under load. The application is stateless (no in-memory state shared between instances), making horizontal scaling viable.

SCALING LIMITS:

- **MONGODB ATLAS:** Connection limits per cluster tier. Each warm Vercel instance holds a Mongoose connection pool. Under high load, connection exhaustion is the primary scaling limit. MITIGATION: Configure pool size: `mongoose.connect(URI, {maxPoolSize: 5})`. Use connection pooler for higher concurrency.
- **VERCEL FUNCTION LIMITS:** Hobby plan: 10-second execution limit; Pro plan: 60-second execution limit; File size: 50MB compressed bundle limit.
- **GMAIL SMTP:** Hard send limit (500-2000 emails/day). Under load, email delivery becomes a bottleneck before technical infrastructure does.

23.2 Scaling Roadmap

- **PHASE 1 -- Current State (< 1000 concurrent users):** Architecture: Monolith + Vercel + MongoDB Atlas. Changes needed: Add indexes, fix N+1 queries, add rate limiting.
- **PHASE 2 -- Growth Scale (1000-10,000 concurrent users):**
 - Add Redis cache (Upstash) for product listings and categories
 - Add email queue (BullMQ or Upstash Queue)
 - Migrate email to SendGrid/SES
 - Add server-side pagination to all list endpoints
 - Add Atlas Search for product search
 - Add health checks and uptime monitoring
- **PHASE 3 -- High Scale (10,000+ concurrent users):**
 - Extract cart to separate collection (reduce User document bloat)
 - Extract reviews to separate collection
 - Implement connection pooler for MongoDB
 - Add read replicas for analytics queries
 - Consider CQRS: separate read/write models for seller dashboard
 - Extract notification service to dedicated process
 - Add CDN-level caching for product catalog API
- **PHASE 4 -- Hyperscale (100,000+ concurrent users):**
 - Service decomposition: catalog, order, identity services
 - Event streaming backbone (Kafka or AWS Kinesis)
 - Database sharding strategy (by sellerId for order/product data)
 - Global CDN with regional edge caching
 - Microservices with inter-service mesh

23.3 Database Scaling Strategy

- **READ SCALING:** MongoDB Atlas supports read replicas. Route read-heavy operations to secondary replica:

```
const products = await Product.find({...}).read('secondary');
```

- **WRITE SCALING:** MongoDB write throughput scales with cluster tier. For the current operation profile, Atlas M10 tier is sufficient up to tens of thousands of daily orders.
- **SHARDING READINESS:** Current schema is not optimized for sharding but can be sharded by sellerId:

`db.adminCommand({ shardCollection: "browsemart.products",`

- Consumer-facing queries (by customerId) would become cross-shard after sharding, requiring careful coordination.

SECTION 19: CI/CD & DEPLOYMENT

24.1 Current Deployment Architecture

Browse-Mart currently has no formal CI/CD pipeline documented in the codebase. There are no GitHub Actions workflows, CircleCI configs, or Jenkinsfiles present. Based on the Vercel deployment configuration (`vercel.json` in both frontend and backend directories), the current workflow is inferred as follows:

- Developer pushes code to the GitHub repository.
- Vercel GitHub integration detects the push.
- Vercel automatically builds and deploys: the frontend is built via `npm run build` and served via CDN, while the backend is deployed as a serverless function.
- Vercel Preview URLs are generated for non-main branches.
- Production is deployed upon merging to the main branch.

24.2 Environment Configuration

Backend Environment Variables:

- **MONGO_URI:** MongoDB Atlas connection string.
- **JWT_SECRET:** High-entropy JWT signing secret.
- **CLOUDINARY_API_KEY/SECRET:** Media storage credentials.
- **EMAIL_USER/PASS:** Gmail SMTP credentials.
- **GOOGLE_CLIENT_ID/SECRET:** OAuth2 credentials.

Frontend Environment Variables:

- **VITE_BACKEND_URL:** API base URL.
- **VITE_CLOUD_NAME:** Public Cloudinary identifier.

Security Note: Variables prefixed with `VITE_` are embedded in the client bundle and publicly visible. Secrets must never use this prefix.

24.3 Recommended CI/CD Pipeline

- **Stage 1: Code Quality** — Includes ESLint, Prettier formatting checks, and dependency vulnerability scans (npm audit).
- **Stage 2: Testing** — Unit tests for logic, integration tests for API endpoints, and E2E tests for critical user paths.
- **Stage 3: Build Verification** — Production builds and bundle size threshold checks.
- **Stage 4: Deployment** — Automated database migrations, Vercel deployment, and health check polling.
- **Stage 5: Post-Deploy Verification** — Synthetic monitoring and error rate spike detection.

24.4 Deployment Rollback Strategy

Vercel supports instant rollbacks through its deployment history. Database migration rollbacks require manual effort until a framework like `migrate-mongo` is adopted to provide reversible schema changes.

24.5 Preview Environments

Vercel creates unique preview deployments for every branch. However, these currently share the production database cluster. It is recommended to configure a separate Atlas cluster for staging to prevent production data contamination.

RECOMMENDATION: Configure a separate Atlas cluster for development/staging, controlled by `MONGO_URI` in Vercel's preview environment settings.

SECTION 20: INFRASTRUCTURE & CLOUD DESIGN

25.1 Infrastructure Topology

VERCEL (Primary hosting): Frontend: Static SPA served from Vercel's global CDN. Edge nodes: 35+ regions globally. Assets: Immutable content-hashed files (long TTL cache). Routing: SPA rewrite (all paths -> index.html).

Backend: Serverless functions in Vercel's Node.js runtime. Runtime: Node.js 18+ (Vercel managed). Memory: 1024MB default. Timeout: 10s (Hobby) / 60s (Pro).

Regions: Single region default.

MONGODB ATLAS (Database): Assumed M0 (free tier) or M10+ (production tier). M0 limitations: 512MB storage, 100 connections, no backup. M10+: Dedicated cluster, PITR backup, connection pooling. Atlas IP allowlist should include Vercel outbound IPs.

CLOUDINARY (Media CDN): Free tier: 25GB storage, 25GB bandwidth/month. Images served from `res.cloudinary.com` global CDN. Transformation API available at URL level.

GMAIL SMTP (Email): Google's SMTP relay (`smtp.gmail.com` port 587 TLS).

Rate limited per account tier.

25.2 Environment Isolation

RECOMMENDED ENVIRONMENT STRATEGY: development -- Local MongoDB (Docker), local environment variables; staging -- Separate Atlas cluster, Vercel preview deployment; production -- Production Atlas cluster, Vercel production deployment.

CURRENT STATE: No explicit environment isolation documented. Development likely connects directly to Atlas (same cluster as production) -- data integrity risk.

25.3 Secrets Management

CURRENT: Environment variables stored in Vercel dashboard (encrypted at rest, injected into function environment at runtime).

SECURITY PROPERTIES: Secrets are encrypted in Vercel's backend; Never exposed in git repository (if .env is gitignored); Accessible only to team members with Vercel project access.

GAPS: No secret rotation policy or automation; JWT_SECRET never rotates (if compromised, manual rotation requires re-deploy); No audit trail of secret access or modification.

RECOMMENDED ENHANCEMENTS: Use Vercel's team-level environment variables with appropriate scopes; Implement secret rotation for API keys on 90-day schedule; Store critical secrets in a secrets manager (HashiCorp Vault, AWS Secrets Manager) with automatic rotation.

25.4 Infrastructure as Code

No IaC tooling (Terraform, Pulumi, CDK) exists. Infrastructure is manually configured via Vercel and Atlas dashboards.

For production maturity, recommended IaC scope: Vercel project configuration (via Vercel Terraform provider); Atlas cluster specification (MongoDB Atlas Terraform provider); Cloudinary upload presets; DNS configuration.

Benefits: reproducible environments, PR-reviewed infrastructure changes, disaster recovery via infrastructure recreation.

SECTION 21: OBSERVABILITY & MONITORING

=====

=====

26.1 Current Observability State

Browse-Mart has MINIMAL observability. LOGGING: Uses console.error() in backend catch blocks, lacks structured logging and log aggregation, and Vercel runtime logs

have short retention (1-hour Hobby, 3 days Pro). METRICS, TRACING, and ALERTING are all absent (no APM, no custom metrics, no infra metrics, no distributed tracing, no request correlation IDs, no error rate, latency, or business metric alerts). FRONTEND: No error tracking (no Sentry/Bugsnag/Rollbar), no user behavior analytics, and no performance monitoring (no Web Vitals tracking).

26.2 Observability Gap Impact

INCIDENT DETECTION: The engineering team learns about outages from user complaints, not automated alerts. Mean Time to Detect (MTTD) is measured in hours, not minutes. **ROOT CAUSE ANALYSIS:** No structured logs means debugging production issues requires adding log statements and re-deploying. **PERFORMANCE REGRESSION:** No baseline metrics means slow query regressions are invisible until users report "the site feels slow." **BUSINESS METRICS:** No visibility into conversion rates, cart abandonment, checkout failure rates, or order success rates.

26.3 Recommended Observability Stack

STRUCTURED LOGGING (immediate priority): Replace `console.error` with Pino or Winston:

JavaScript

```
import pino from 'pino';
const logger = pino({ level: 'info' });

logger.info({ orderId, userId, total }, 'order.created');
logger.error({ err, userId, action: 'checkout' },
  'order.failed');
```

Configure Vercel Log Drains to send logs to:

- Datadog (full observability platform)
- Logtail (Vercel-native, affordable)
- Axiom (modern log management, generous free tier)

ERROR TRACKING (high priority): Add Sentry to both frontend and backend:

- Backend: `@sentry/node` (automatic Express integration)
- Frontend: `@sentry/react` (error boundaries, performance monitoring)

Key metrics captured:

- Unhandled exceptions with full stack trace
- Affected user count per error
- Release-tagged errors (know which deploy introduced a bug)
- Performance transaction traces (p50/p95/p99 latency per route)

APM / METRICS (medium priority):

- Vercel Analytics (built-in): Web Vitals (LCP, FID, CLS) per page, real user performance data, and geographic distribution.
- Custom Business Metrics: Track key events to a time-series store:
 - orders_placed_per_minute
 - checkout_success_rate
 - cart_abandonment_rate
 - api_error_rate_per_endpoint

26.4 Request Correlation

No request correlation IDs exist. Adding correlation IDs makes distributed debugging possible:

Middleware:

```
JavaScript
import { v4 as uuidv4 } from 'uuid';
app.use((req, res, next) => {
  req.requestId = req.headers['x-request-id'] || uuidv4();
  res.setHeader('x-request-id', req.requestId);
  next();
});
```

Include requestId in every log entry for trace-level debugging.

26.5 SLO Recommendations

Target SLOs for production readiness:

- API Availability: 99.5% uptime (< 3.65 hours/month downtime)
- API Latency (p95): < 500ms for product queries, < 1000ms for order submission
- Error Rate: < 0.5% of all requests
- Checkout Success Rate: > 95% of initiated checkouts complete
- Email Delivery: > 99% delivery success rate

```
=====
=====
END OF PART III
=====
=====
PART IV — OPERATIONAL EXCELLENCE
=====
=====
```


SECTION 22: ERROR HANDLING STRATEGY

27.1 Error Classification Model

Browse-Mart currently uses a flat error model (success: true/false + message string).

Production systems require classified errors:

- **CLASS 1 — Validation Errors (4xx, client fault):** Missing required fields, type mismatch, business rule violations, or resources not found. Action: Return immediately with a specific message; no retry.
- **CLASS 2 — Authorization Errors (401/403):** Invalid/expired JWT, wrong roles, or blocked accounts. Action: Return immediately; client must re-authenticate.
- **CLASS 3 — Conflict Errors (409):** Duplicate email registrations or concurrent modification conflicts. Action: Return with a specific conflict reason.
- **CLASS 4 — Transient Errors (503, retry-eligible):** Timeouts for MongoDB connections, Cloudinary uploads, or Gmail SMTP. Action: Retry with exponential backoff; return 503 if all retries fail.
- **CLASS 5 — Internal Errors (500, engineering attention required):** Unhandled exceptions, unexpected data shapes, or code bugs. Action: Log full stack trace, return generic message to hide internals, and alert engineering via error tracking.

27.2 Current Error Handling Gaps

- **NO ERROR CODES:** Errors return only message strings. Client code cannot programmatically distinguish specific issues like "wrong password" from "account blocked" without string matching. Fix: Add structured error code fields to responses.
- **NO GLOBAL ERROR HANDLER:** Each controller wraps its own try/catch. Unhandled rejections in middleware may not be caught. Fix: Implement Express global error middleware to log and standardize internal error responses.
- **NO ASYNC ERROR PROPAGATION:** Express 4.x does not automatically catch async errors. Controllers currently require manual try/catch or an asyncHandler utility to propagate errors to the next middleware.

27.3 Error Response Standardization

A standard error response should include a success flag, machine-readable code, human-readable message, requestId for correlation, and a timestamp. This allows for precise client-side handling and server-side log correlation.

27.4 Client-Side Error Handling

Frontend handling currently uses Axios interceptors and SweetAlert2 toasts but lacks a systematic boundary strategy. Recommended patterns include React Error Boundaries

for route sections, automated 401 redirection via interceptors, and Sentry integration for capturing unexpected exceptions.

SECTION 23: PRODUCTION READINESS ASSESSMENT

31.1 Production Readiness Scorecard

DIMENSION	CURRENT STATE	PRODUCTION READY?
Authentication	JWT + bcrypt	Mostly yes (needs token refresh)
Authorization (RBAC)	3 role middleware	Yes
Input Validation	Basic	Needs regex escaping, rate limiting
Database Indexes	Minimal	No -- add immediately
Transaction Integrity	None	No -- critical gap
Payment Processing	Mock only	No -- critical gap
Email Reliability	No retry/queue	No -- single SPOF
Error Handling	Basic try/catch	Needs structured logs
Observability / Logging	None structured	No
Monitoring / Alerting	None	No
Health Checks	None	No
Rate Limiting	None	No
Security Headers	None	No
CSRF Protection	None	No (OAuth gap)

DIMENSION	CURRENT STATE	PRODUCTION READY?
Test Coverage	Zero	No
CI/CD Pipeline	Inferred Vercel	Minimal
Backup / Recovery	Atlas dependent	Needs verification
Secrets Management	Vercel env vars	Acceptable baseline
Caching	PWA client-side	No server cache
Cancellation Stock Restore	Not implemented	No
Inventory Atomicity	Not atomic	No
Search Performance	Full scan regex	No

OVERALL PRODUCTION READINESS: Suitable for low-traffic MVP/beta with careful monitoring. Requires significant hardening for high-traffic production deployment.

31.2 Minimum Viable Production Checklist

The following changes are required before high-traffic production launch:

CRITICAL (blocking):

- ☐ Add MongoDB indexes (User.email, Product.sellerId/category, Order.customerId/sellerId/createdAt)
- ☐ Implement atomic stock decrement (\$inc with precondition)
- ☐ Implement MongoDB session transactions for order submission
- ☐ Add rate limiting to auth endpoints (express-rate-limit)
- ☐ Add Helmet.js security headers
- ☐ Validate FRONTEND_URL at startup
- ☐ Implement stock restoration on order cancellation
- ☐ Change default admin seed credentials

HIGH PRIORITY (within 2 weeks of launch):

- ☐ Structured logging (Pino + log aggregation)
- ☐ Error tracking (Sentry frontend + backend)
- ☐ Health check endpoint
- ☐ Email delivery queue (retry on failure)
- ☐ File upload size/type limits in Multer
- ☐ Fix N+1 queries in cart and order controllers

MEDIUM PRIORITY (within 1 month):

- ☐ Redis caching for category list and product listings
- ☐ Uptime monitoring alerts (UptimeRobot or Pingdom)

- ☐ CSP headers (Content-Security-Policy)
- ☐ Payment gateway integration
- ☐ Separate staging environment from production MongoDB
- ☐ OTP: replace `Math.random()` with `crypto.randomInt()`

SECTION 24: FUTURE SCALING CHALLENGES

34.1 Flash Sale Inventory Management

The current non-atomic stock decrement is the most immediate scaling challenge. During a flash sale with 1000 concurrent users competing for 100 units of a product, the system will create many more than 100 successful orders, result in deeply negative stock values, and require manual inventory reconciliation. This is not a "future" challenge -- it exists today. It must be fixed before any promotional campaign that drives concurrent demand.

34.2 MongoDB Connection Exhaustion

At high load (many simultaneous Vercel function warm instances), the number of MongoDB connections can exceed Atlas tier limits. An M10 Atlas cluster has a default connection limit of 500. With 100 warm Vercel instances each holding a pool of 5 connections, that's 500 connections -- exactly at limit.

When the limit is exceeded, new requests receive "connection refused" errors from MongoDB. This manifests as 500 errors to users.

Mitigation: Reduce `maxPoolSize` per function instance, use MongoDB Atlas connection pooler (Serverless driver), or consider switching to MongoDB Atlas Data API (HTTP-based, no persistent connections, scales to unlimited concurrency).

34.3 Search at Scale

The current regex-based search degrades linearly with product count. At 10,000 products, performance is ~50ms per search; at 100,000 products, ~500ms; and at 1,000,000 products, it reaches ~5000ms (timeout).

Atlas Search must be adopted before the product catalog reaches ~50,000 items, or the search experience will become unacceptably slow.

34.4 User Document Bloat

The `user.cart`, `user.wishlist`, and `user.order` arrays grow over time. A user with 5 years of purchase history may have hundreds of order references. Every authenticated request fetches the full User document.

At 1000 order references per user, the User document is still small (ObjectId array). But the cart and wishlist arrays can grow unexpectedly if not bounded. Adding a maximum size constraint (e.g., wishlist: 100 items) or migrating to separate collections is needed for long-term health.

34.5 Email Delivery at Scale

Gmail SMTP cannot support high order volumes. At 1000 orders/day, the free Gmail limit (500/day) is exceeded. Even with Workspace (2000/day), growth beyond that requires a professional transactional email provider.

Migration to SendGrid/SES/Postmark should happen BEFORE the limit is hit, not after (emergency migrations under load are error-prone).

34.6 Seller Dashboard Analytics

The seller dashboard aggregation pipeline runs against all Orders for a seller on every dashboard page load. For high-volume sellers (10,000+ orders), this aggregation becomes slow.

Solutions: Cache dashboard results with 5-minute TTL, maintain materialized views (pre-aggregated revenue/order counts) updated incrementally on each new order, or use MongoDB's \$facet for efficient multi-metric aggregation.

34.7 Order History Pagination

The current implementation of getAllOrderByUserId loads all order ObjectIds from the User.order array and then queries a paginated subset. For users with thousands of orders, the array itself is loaded on every request.

The correct approach is a direct query: Order.find({customerId}).skip().limit(). This bypasses the User document entirely and scales to any order volume.

SECTION 25: ENGINEERING BEST PRACTICES

35.1 Code Organization Philosophy

The code organization philosophy prioritizes clarity and audibility:

- **ONE FILE, ONE RESPONSIBILITY:** Each controller handles exactly one operation, making navigation trivially easy (find "add to cart" → addToCart.js) and keeping files small enough to understand in one reading.
- **EXPLICIT OVER IMPLICIT:** Authorization checks are explicit middleware, not implicit in controllers. Validation is explicit at the start of each controller, not

inferred. This makes code auditable — you can read a route definition and immediately know what security checks apply.

- **CONSISTENT NAMING:** Controller files use camelCase verbs (addToCart), Model files use camelCase with Schema suffix (userSchema), and Routes are kebab-case (/add-to-cart). This convention is consistent throughout the codebase.

35.2 Dependency Management

PACKAGE PHILOSOPHY: Browse-Mart uses well-established, maintained packages: Express, Mongoose, jsonwebtoken, bcrypt, multer, cloudinary, and nodemailer. This is a conservative dependency selection, favoring specialist libraries with narrow scope over monolithic frameworks.

SECURITY AUDITING: Automated vulnerability scanning is currently absent. The recommendation is to run `npm audit --audit-level=high` and enforce it in the CI pipeline.

35.3 API Design Conventions

CURRENT CONVENTIONS (observed): All endpoints return JSON with a success boolean, error responses include a human-readable message, and authentication is handled via the Authorization header. POST is used for most operations, including reads.

NOTE ON POST FOR READS: Several read operations use POST instead of GET (/api/user/get-cart-items, /api/order/get-all-order, /api/user/get-wishlist). This deviates from REST conventions (GET for reads). The likely reason is that a request body is easier to pass in POST than GET query parameters. However, GET is the REST-correct approach and enables HTTP caching semantics.

35.4 Database Best Practices (In Practice)

THINGS DONE CORRECTLY:

- Unique indexes defined inline in schemas (email, orderId, category name).
- Pre-save hooks for password hashing (DRY, cannot bypass).
- Instance methods on schemas (generateToken, passwordCompare).
- Soft delete with isHide flag (data preservation).
- Snapshot denormalization in orders (price, name copied at purchase time).
- Parallel promise execution for independent DB operations (Promise.all).

THINGS NEEDING IMPROVEMENT:

- Missing performance indexes (described in Section 07).
- Missing transaction semantics (described in Section 05).
- Non-atomic inventory operations (described in Section 12).
- No formal migration strategy.

35.5 Frontend Best Practices (In Practice)

THINGS DONE CORRECTLY:

- Provider hierarchy clearly separated by concern.

- PWA caching strategies differentiated by content type.
- Consistent Axios usage with centralized base URL.
- SweetAlert2 for user feedback (accessible, styled consistently).
- Cloudinary for image delivery (not loading images from backend).
- Tailwind CSS for consistent design (no custom CSS sprawl).

THINGS NEEDING IMPROVEMENT:

- No React error boundaries.
- No frontend error tracking (Sentry).
- No accessibility audit (WCAG compliance unknown).
- POST used for read operations (semantic mismatch).
- No end-to-end type safety (TypeScript not used).

35.6 Security Best Practices (In Practice)

THINGS DONE CORRECTLY:

- bcrypt with appropriate cost factor (10 rounds).
- Fresh DB lookup for user status check (prevents zombie tokens).
- OTP time-expiry enforcement.
- OTP hashed at rest.
- CORS restricted to known origin.
- Per-request OAuth2 client (serverless safety).
- Cloudinary server-side upload (client never holds API credentials).

THINGS NEEDING IMPROVEMENT:

- Rate limiting absent on auth endpoints.
- Security headers absent.
- CSRF protection absent on OAuth.
- Insecure OTP generation (`Math.random`).
- Sensitive data stored plaintext (PAN, GST, bank details).
- JWT stored in `localStorage` (XSS risk).

SECTION 35.6 Security Best Practices (In Practice) - THINGS NEEDING IMPROVEMENT (Cont.):

- Rate limiting absent on auth endpoints
- Security headers absent
- CSRF protection absent on OAuth
- Insecure OTP generation (`Math.random`)
- Sensitive data stored plaintext (PAN, GST, bank details)
- JWT stored in `localStorage` (XSS risk)

SECTION 26: UI DESIGN

26.1 Design Philosophy and Stack

Browse-Mart's UI design philosophy is rooted in pragmatism, speed, and mobile compatibility, driven by the following technical choices:

- **Technology Stack:** Built with React 19 (CSR) and styled using Tailwind CSS, which provides a utility-first approach to ensure visual consistency and rapid development.
- **Mobile-First Approach:** The UI is inherently responsive, utilizing Tailwind's breakpoint system (Flexbox and CSS Grid) to ensure full compatibility across mobile, tablet, and desktop devices.
- **Aesthetics & Affordance:** The design focuses on high usability and clear affordances. Key features like shopping cart buttons and product cards are designed to be obvious and unsurprising, prioritizing function over pure aesthetics.

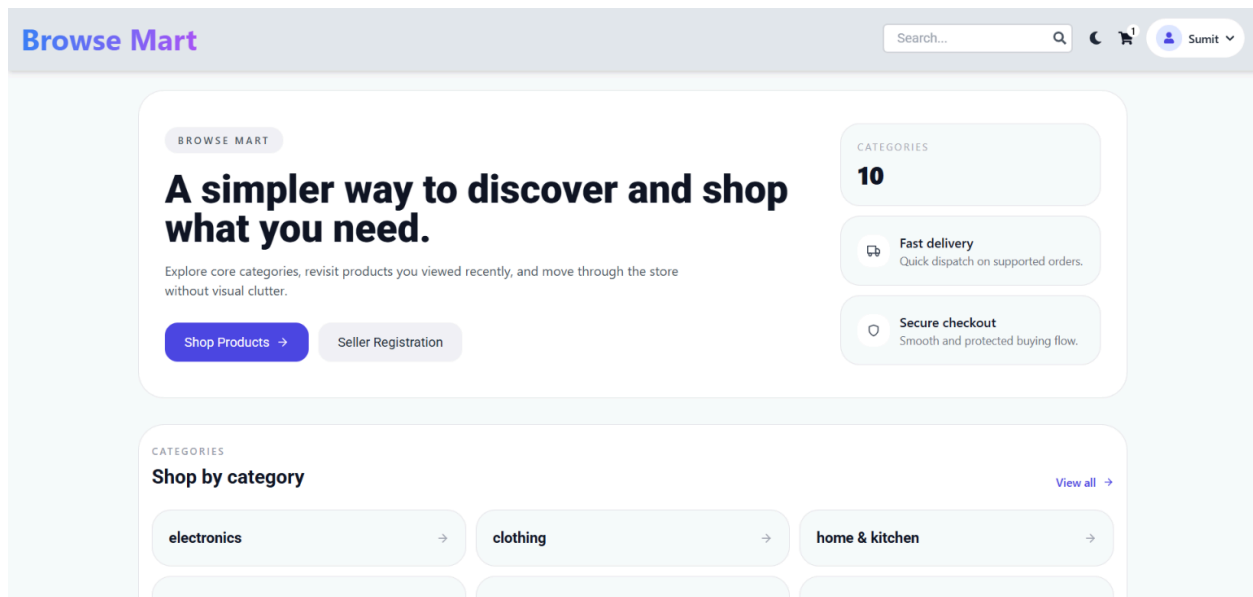
26.2 Core UI/UX Features

The application meets several core user interface requirements to enhance the shopping experience:

- **Responsive Design:** Optimized layouts are available for all supported device types.
- **Theme Support:** Supports both Dark and Light themes, managed via the `themeContext.jsx` and persisted in `localStorage`.
- **Product Presentation:** Products are displayed in a clean, responsive grid view with detailed image galleries and clear pricing information.
- **Fast Navigation:** Uses React Router v7 and client-side rendering (CSR) for quick, dynamic page updates without full page reloads, complemented by PWA caching for fast asset delivery.

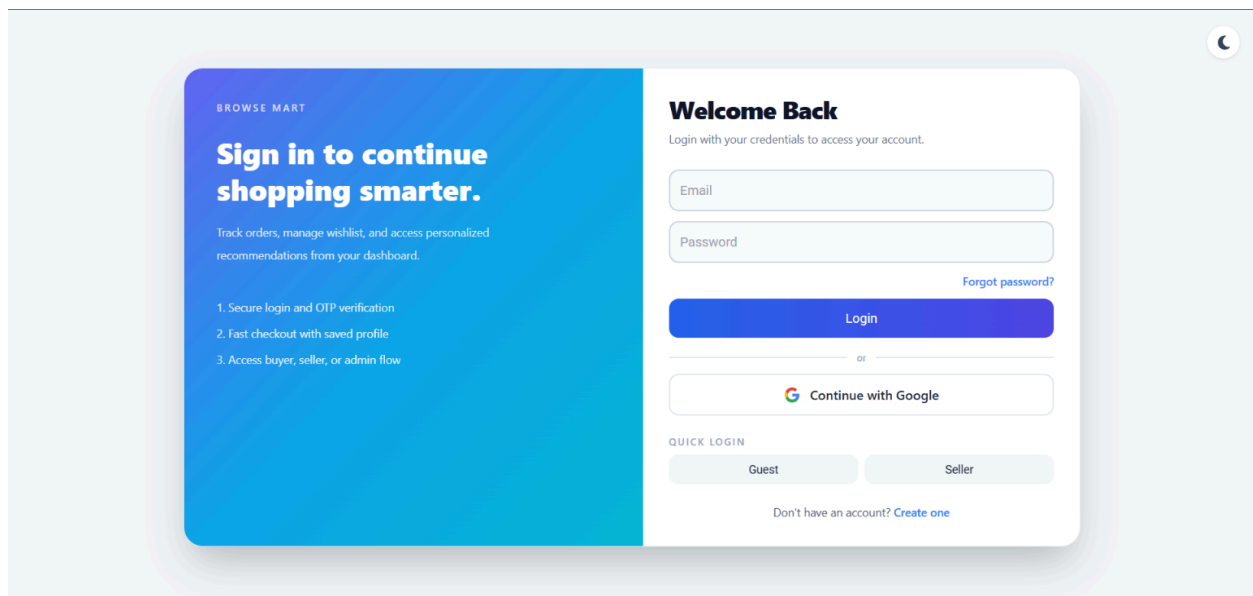
26.3 Home Page

An illustration of the mobile-first homepage, featuring the responsive product grid and dual-theme design (light/dark elements) described in the design philosophy.



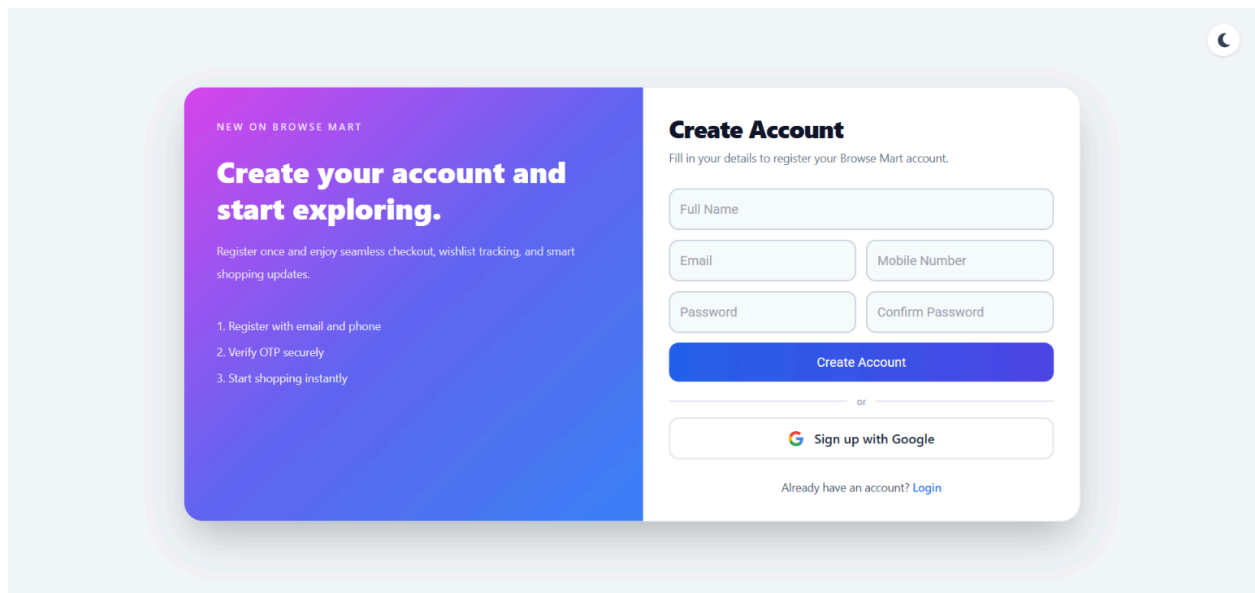
26.4 Login Page

The Login page provides the primary access point for all users (consumer, seller, admin). It supports email/password authentication and a one-click sign-in option via Google OAuth. The design is minimal, focusing on high-contrast input fields.



26.5 Sign Up Page

The consumer registration form collects Name, Email, and Password, with a field for Confirm Password. It utilizes client-side validation and is the initial point for the OTP email verification process upon submission. It includes a clear link for users who wish to join the platform as vendors.



NEW ON BROWSE MART

Create your account and start exploring.

Register once and enjoy seamless checkout, wishlist tracking, and smart shopping updates.

1. Register with email and phone
2. Verify OTP securely
3. Start shopping instantly

Create Account

Fill in your details to register your Browse Mart account.

Full Name

Email Mobile Number

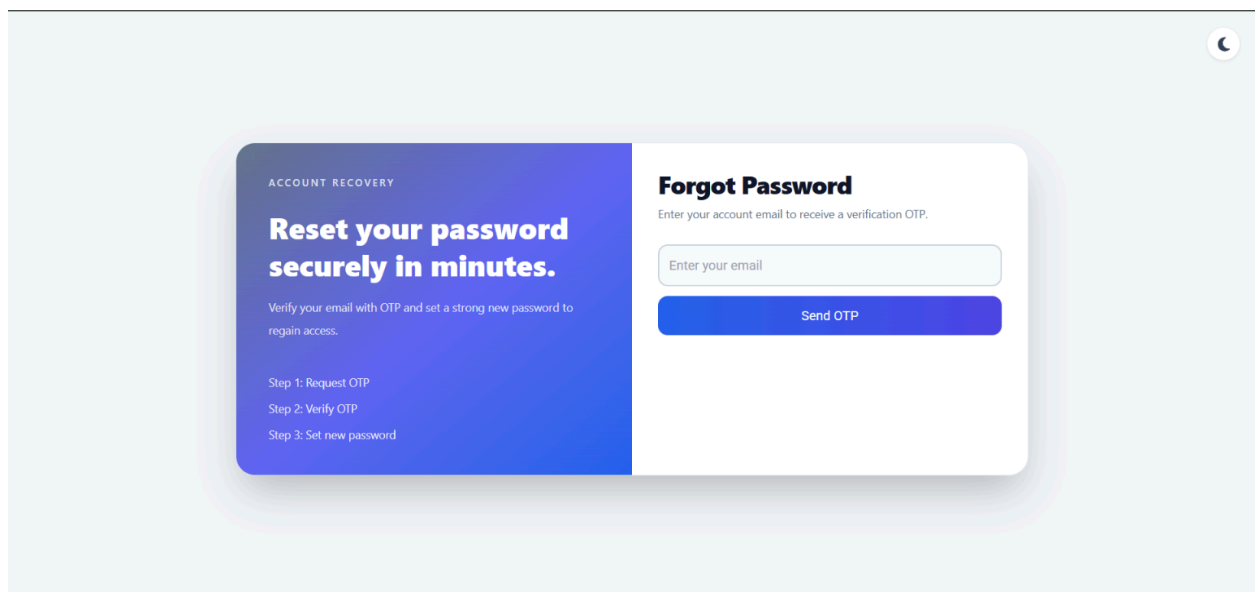
Password Confirm Password

Create Account

or

 Sign up with Google

Already have an account? [Login](#)



ACCOUNT RECOVERY

Reset your password securely in minutes.

Verify your email with OTP and set a strong new password to regain access.

- Step 1: Request OTP
- Step 2: Verify OTP
- Step 3: Set new password

Forgot Password

Enter your account email to receive a verification OTP.

Enter your email

Send OTP

26.6 Seller Registration Page

This specialized form is used by existing consumers to upgrade their role to a seller. It collects mandatory business and KYC details, such as PAN, GST, and trade license information, which are required for legal compliance but are currently stored as plaintext (acknowledged security debt in Section 17.5).

SELLER ONBOARDING

Launch your seller profile with a cleaner, guided flow.

Complete your business, legal, and payout information in a single polished application designed to feel consistent with the rest of the storefront.

PROGRESS

0/3

Sections ready for review



Application Steps

Move section by section and submit once everything looks right.



Business Information

Store identity, contact, and address details



Legal Information

Business compliance and registration records



Financial Information

Settlement account and payout information

STEP 1

Business Information

Store identity, contact, and address details

Business Name *

Enter business name

Phone Number *

Enter phone number

Email Address *

Enter email address

Address Line 1 *

Enter business address

City *

Enter City name

State *

Enter State Name

Pin Code *

Enter Pin Code

Country *

Enter Country Name

Business Type *

Enter business type

Website URL

Enter website URL

< Previous

Next Section >

Browse Mart

Your one-stop shop for all things amazing.



Quick Links

Seller Registration
About Us
Contact
FAQs
Shipping

Customer Service

My Account
View Orders
Track Order
Returns
Wishlist

Newsletter

Subscribe to get special offers and updates.

Your email



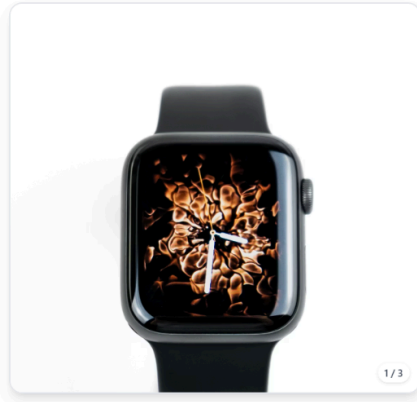
26.7 Product Detail Page

Browse Mart

Search...

 Test





ELECTRONICS

NoiseFit Active Smartwatch Silver

₹4,299

₹5,999

28% OFF

Inclusive of all taxes

Free Delivery

On orders above ₹499

Easy Returns

7-day return policy

Secure Payment

100% safe checkout

Add To Cart

Buy Now

Estimated delivery: 3-5 business days

Share this product

Product Description

A modern smartwatch with fitness tracking, Bluetooth calling support, and a sharp display that works well for day-to-day use.

NoiseFit Active Smartwatch Silver

₹4,299

Add To Cart

Buy Now

Customer Reviews

Write a Review

No reviews yet

Be the first to share your experience!

You May Also Like

Asus TUF F15 Laptop

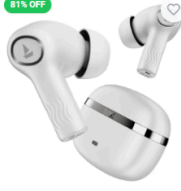


JBL Tune 510BT, On Ear Wireless



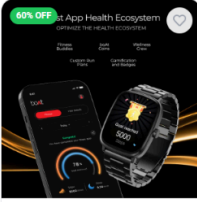
boAt Nirvana Ion

81% OFF



boAt Ultima Real

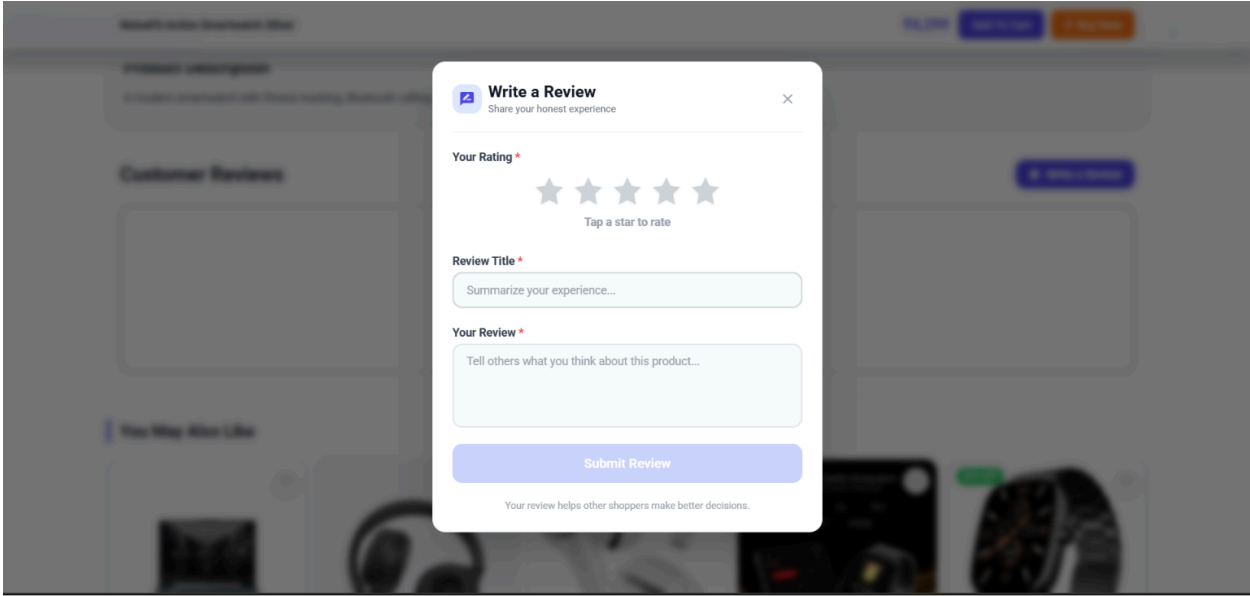
60% OFF



Bolt Watch

50% OFF





26.9 Checkout Page

Secure Checkout

SSL Secured

Customer Details

Full Name *

Test User

Phone Number *

7073384969

Shipping Address

Use Default Address

Address Line 1 *

Address Line 1

Address Line 2

Address Line 2

Country *

Country

State *

State

City *

City

PinCode *

PinCode

☐ Save as default shipping address

Payment Method

Debit / Credit Card

UPI UPI

Cash On Delivery

Your Order (6 items)

Asus TUF F15 Laptop

Qty: 9 × ₹60,000

₹540,000

Dell Inspiron 15 Laptop 16GB 512GB SSD

Qty: 3 × ₹52,990

₹158,970

Sony Alpha ZV-E10 Mirrorless Camera

Qty: 1 × ₹64,990

₹64,990

Price Details

MRP Total

₹788,813

Discount

- ₹23,500

Shipping

FREE

Grand Total

₹765,313

✓ You save ₹23,500 on this order!

Place Order · ₹765,313

Secure · Fast Delivery · Easy Returns

Browse Mart

Your one-stop shop for all things amazing.

f

t

@

Quick Links

Seller Registration

About Us

Contact

FAQs

Shipping

Customer Service

My Account

View Orders

Track Order

Returns

Wishlist

Newsletter

Subscribe to get special offers and updates.

Your email

Privacy Policy · Terms of Service · Contact Us

© 2026 BrowseMart. All rights reserved.

26.10 Profile Page

Browse Mart

Search...

⁶

 Test ▾



Test User
guest@gmail.com

Profile Overview

Password & Security

Orders

Wishlist

Profile Overview

Personal Information

Saved Addresses

Shipping Address

First Name

test

Last Name

user

Email

guest@gmail.com

Phone

7073384969

Home

Bikaner, Rajasthan

Primary Address

Bikaner, Rajasthan

Saved shipping address for checkout.

Edit

Current saved shipping address

laxminath ji mandir ke pass
bikaner rajasthan, rajasthan
india - 334001

Address Line 1

laxminath ji mandir ke pass

Address Line 2

City

bikaner rajasthan

State

rajasthan

Pin Code

334001

Country

india

Browse Mart

Your one-stop shop for all things amazing.



Quick Links

[Seller Registration](#)
[About Us](#)
[Contact](#)
[FAQs](#)
[Shipping](#)

Customer Service

[My Account](#)
[View Orders](#)
[Track Order](#)
[Returns](#)
[Wishlist](#)

Newsletter

Subscribe to get special offers and updates.

Your email



[Privacy Policy](#) · [Terms of Service](#) · [Contact Us](#)

© 2026 BrowseMart. All rights reserved.



Test User

guest@gmail.com



Profile Overview



Password & Security



Orders



Wishlist

Password & Security

Change Password

Update your current password. You will need your existing password to confirm this change.

Current Password

Enter current password

New Password

Min. 6 characters

Confirm New Password

Repeat new password

Change Password



Test User

guest@gmail.com



Profile Overview



Password & Security



Orders



Wishlist

Order History

ORDER NUMBER

ORD-20260505-531504-64329505

ORDER DATE

May 5, 2026

TOTAL AMOUNT

₹1,985

Order Placed

View Order

View Invoice



BENYAR Stylish Mens Watch

Elevate your style with the BENYAR Stylish Mens Watch, a true masterpiece of modern design and robust functionality. Its striking all-black metallic finish, featuring a sophisticated chronograph dial with luminous...

Order Status: Order Placed

PRICE

QTY

AMOUNT

₹1,985

1

₹1,985

View Product

Buy Again

ORDER NUMBER

ORD-20261604-062011-31721432

ORDER DATE

Apr 16, 2026

TOTAL AMOUNT

₹170.5

Order Placed

View Order

View Invoice



Gandhi 777 Vaseline

Discover Gandhi 777 Vaseline, an A Grade Petroleum Jelly encased in its iconic, robust jar with a vibrant orange cap. This superior formulation delivers exceptional purity and lasting hydration, providing reliable skin protection...

PRICE

QTY

AMOUNT

₹155

1

₹155

Showing 1 to 5 of 80 results

Previous

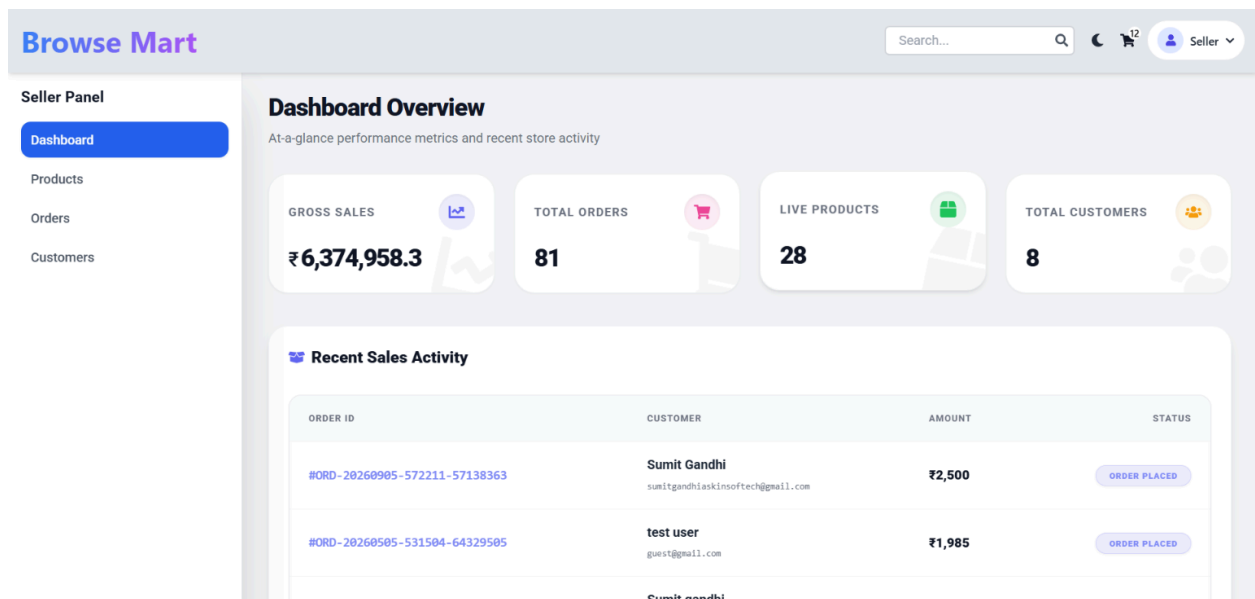
1

2

...

16

Next



Seller Panel

Dashboard

Products

Orders

Customers

Inventory

Manage your product catalog

+ Add New Product

Search Products by name or ID...

All Categories ▾

All Status ▾

PRODUCT	CATEGORY	PRICE	STOCK	VISIBILITY	ACTIONS
 Dell Inspiron 15 Laptop 16GB... Elevate your workflow with the Dell Inspiron 15, a...	ELECTRONICS	₹58,990	5 LOW STOCK	☒	 
 Lego Classic Creative Brick Box Dive into boundless creativity with the Lego Class...	TOYS	₹2,499	17 LOW STOCK	☒	 
 Yonex Practice Tennis Ball Pack ... Durable tennis balls with consistent bounce...	SPORTS	₹499	110 IN STOCK	☒	 
 Milton Thermosteel Water Bottle... Experience premium hydration with the Milton...	HOME & KITCHEN	₹1,199	74 IN STOCK	☒	 

Seller Panel

Dashboard

Products

Orders

Customers

Order Management

Review orders placed for your products and update shipping statuses

#ORD-20260905-572211-57138363

ORDER PLACED

UPDATE ORDER STATUS

May 10, 2026, 04:52 AM · Pending (Cod)

Order Placed ▾



CUSTOMER SUMMARY

Sumit Gandhi
sumitgandhiaskinsofttech@gmail.com



SHIPPING DESTINATION

laxminath ji mandir ke pass, deepnath ji mandir gali, bada bazar
bikaner, rajasthan, 334001
india

ORDERED PRODUCTS



Bolt Watch
Bolt Watch has a its feature

₹2,500 x 1

₹2,500

Browse Mart

Search...

12

Seller

Seller Panel

Dashboard

Products

Orders

Customers

Customers

Review customer demographics and order history

CUSTOMER	CONTACT INFO	TOTAL ORDERS	LAST ORDER ID	LATEST ORDER DATE
S Sumit Gandhi	sumitgandhiaskinsofttech@gmail.com No phone	1	#ORD-20260905-572211-57138363	May 10, 2026
T test user	guest@gmail.com 7073384969	48	#ORD-20260505-531504-64329505	May 5, 2026
S Sumit gandhi	sumitgandhi932@gmail.com 7073384969	9	#ORD-20261604-082511-41711878	Apr 16, 2026
S Sumit Gandhi	jarvisasus+lg@gmail.com 7073384969	6	#ORD-20260404-185102-53681373	Apr 4, 2026
S Seller	seller@gmail.com 123456789	14	#ORD-20253010-232310-49254450	Oct 30, 2025
G Gunjan Vishwakarma	gunjan320vi@gmail.com 256023151	1	#ORD-20251404-585212-90276465	Apr 14, 2025

26.12 Admin Dashboard Page

Browse Mart

Search...

1

BrowseMart

Admin Panel

Dashboard

Users Directory

Global Inventory

Categories

Platform Overview

Real-time statistics covering all marketplace entities.

ACTIVE BUYERS

34

VERIFIED SELLERS

4

LIVE CATALOG

31

TOTAL ORDERS

139

Admin Panel

Dashboard

Users Directory

Global Inventory

Categories

User Management

Control access for all buyers and sellers.

Q Search by name or email...

USER DETAILS	ROLE	JOINED	STATUS	ACTION
Mi Tv mltv707338@gmail.com	CONSUMER	Invalid Date	● ACTIVE	🚫 Block
Sumit Gandhi sumitgandhiaskinssofttech@gmail.com	CONSUMER	Invalid Date	● SUSPENDED	🔄 Restore
Sumit Gandhi itssumitgandhi@gmail.com	CONSUMER	Invalid Date	● ACTIVE	🚫 Block
Sumit Gandhi sumitgandhi932+300@gmail.com	CONSUMER	Invalid Date	● ACTIVE	🚫 Block
BrowseMart Command admin@gmail.com	ADMIN	Invalid Date	● ACTIVE	🔄 Block

Admin Panel

Dashboard

Users Directory

Global Inventory

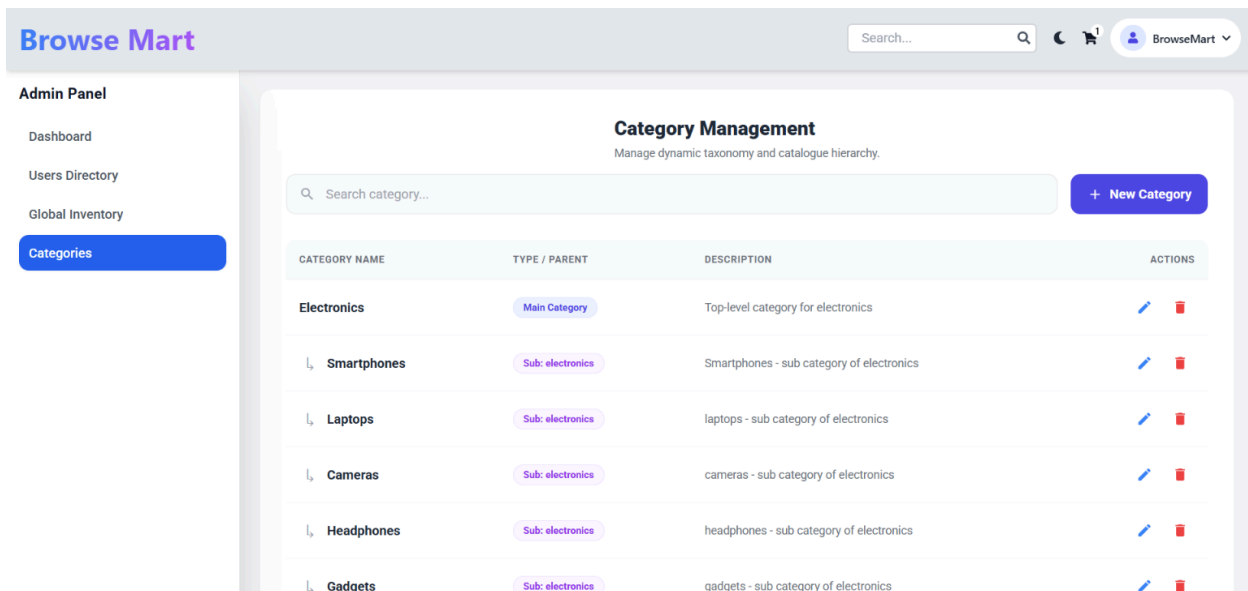
Categories

Global Inventory

Platform-wide overview of all products and sellers.

Q Search products by title...

PREVIEW	PRODUCT SPECS	BUSINESS / SELLER	PRICING	INVENTORY
	Dell Inspiron 15 Laptop 16GB 512GB SSD ELECTRONICS	Seller seller@gmail.com	₹58990 Sale: ₹52990	5 in stock
	Lego Classic Creative Brick Box TOYS	Seller seller@gmail.com	₹2499 Sale: ₹1999	17 in stock
	Yonex Practice Tennis Ball Pack of 3 SPORTS	Seller seller@gmail.com	₹499 Sale: ₹349	110 in stock
	Milton Thermosteel Water Bottle 1L HOME & KITCHEN	Seller seller@gmail.com	₹1199 Sale: ₹849	74 in stock
	Bose SoundLink Mini II Bluetooth Speaker ELECTRONICS	Seller seller@gmail.com	₹6600 Sale: ₹5990	10 in stock



- **Consumer Interface:**
 - **Homepage:** Features dynamic banners and product carousels for discovery.
 - **Shopping Cart:** A dedicated management page for calculating totals, updating quantities, and viewing breakdowns.
 - **Order History:** A user-specific dashboard to track past and active orders.
- **Seller Interface:**
 - **Seller Dashboard:** Features visual analytics (Charts) of sales performance and revenue (though performance bottlenecks exist for high-volume sellers).
 - **Product Upload:** A complex form with multi-image integration via Cloudinary.
 - **Order Management:** An interface for viewing customer orders and transitioning order statuses.
- **Admin Interface:**
 - **Admin Dashboard:** Overview of total users, active sellers, and global sales metrics.
 - **User Management:** Centralized table for monitoring user roles and status (Block/Unblock).
 - **Category Management:** Full CRUD interface for managing the site's category hierarchy.

26.13 Component and State Architecture

The component architecture emphasizes maintainability through:

- **Component Structure:** Components follow a flat composition model, organized by feature/page rather than strict abstract levels (like Atomic Design).

- **Global State:** Global UI concerns (theme, cart count) are managed using the React Context API, which simplifies state management compared to Redux for the current complexity.
- **CSS Implementation:** Tailwind CSS is used to enforce a consistent design rhythm, eliminating the need for custom CSS sprawl and providing utility classes for responsive grids and layouts.

26.14 Future UI/UX Gaps

While usable, the UI/UX layer has acknowledged limitations:

- **No Dedicated UX State:** No global notification state exists, requiring redundant fetches for data freshness.
- **Accessibility:** No formal accessibility audit (WCAG compliance) has been conducted.
- **Frontend Observability:** Absence of frontend error tracking (e.g., Sentry) means client-side JavaScript errors are invisible to the engineering team.

SECTION 27: PROJECT LINKS

27.1 Live Deployment URL

The Browse-Mart application is currently live and deployed to Vercel's global edge network.

<https://browsemart.vercel.app/>

27.2 GitHub Repository

The full source code, including the backend API (Node.js/Express) and the frontend SPA (React/Vite), is hosted on GitHub.

<https://github.com/sumit-gandhi-sg/Browse-Mart>

APPENDIX A: GLOSSARY OF TERMS

TERM	DEFINITION
Atlas	MongoDB Atlas — the managed MongoDB cloud database service used as Browse-Mart's primary data store.

TERM	DEFINITION
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator — a random number generator suitable for security-critical operations. Node.js provides this via the crypto module.
CSR	Client-Side Rendering — HTML is generated in the browser by JavaScript. The server sends a minimal HTML shell.
CSRF	Cross-Site Request Forgery — an attack that tricks a user's browser into making unwanted requests to an authenticated site.
DLQ	Dead Letter Queue — a message queue where failed messages are sent after exhausting retry attempts.
GMV	Gross Merchandise Value — the total value of goods sold through a marketplace.
HttpOnly	A cookie attribute that prevents JavaScript access to the cookie, mitigating token theft via XSS.
idempotency	A property of an operation where performing it multiple times has the same effect as performing it once.
isHide	Browse-Mart's soft-delete flag on Product documents. When true, the product is hidden from public listings but preserved in the database.
JWT	JSON Web Token — a compact, URL-safe token format for representing claims between parties.
N+1 Query	A database anti-pattern where N+1 queries are executed to fetch N related records (one query for the list, one per item).
PITR	Point-In-Time Recovery — the ability to restore a database to the exact state it was in at any specific moment in the past.
PWA	Progressive Web App — a web application that can be installed on devices and operates with offline capability.
RBAC	Role-Based Access Control — a security model where permissions are assigned to roles (consumer, seller, admin) rather than individual users.

TERM	DEFINITION
ReDoS	Regular Expression Denial of Service — an attack that exploits catastrophically backtracking regex patterns to consume CPU and deny service.
RPO	Recovery Point Objective — the maximum acceptable amount of data loss measured in time.
RTO	Recovery Time Objective — the maximum acceptable time to restore service after a failure.
SLA	Service Level Agreement — a formal commitment to minimum service availability or performance.
SLO	Service Level Objective — an internal target for service availability or performance.
SPA	Single Page Application — a web application that loads a single HTML page and dynamically updates content without full page reloads.
SSR	Server-Side Rendering — HTML is generated on the server before being sent to the browser.
VAPID	Voluntary Application Server Identification — a mechanism for Web Push authentication that uses public/private key pairs.
Workbox	Google's library for service worker-based caching strategies, used by vite-plugin-pwa in Browse-Mart.
XSS	Cross-Site Scripting — an attack where malicious scripts are injected into a trusted web application.

=====

=====

APPENDIX B: ENGINEERING DECISION RECORDS (EDRs)

=====

=====

EDR-001: Serverless Deployment on Vercel
Date: Project inception

Status: Active

Context: Browse-Mart needed a deployment platform that minimizes infrastructure operations overhead for a small team.

Decision: Deploy both frontend (static SPA) and backend (Express serverless function) on Vercel.

Rationale: Zero-ops deployment, automatic HTTPS, global CDN for frontend assets, automatic horizontal scaling, pay-per-invocation billing at low traffic, and Vercel preview deployments for PRs.

Tradeoffs: Serverless cold starts add latency to first request after idle. Execution time limits (10s/60s) constrain long-running operations. Additionally, there is no persistent process, meaning no WebSockets or background workers.

Consequences: Long-running jobs must be handled via external queues, and WebSocket/SSE requires a third-party service like Pusher or Ably.

EDR-002: Per-Request Google OAuth2 Client

Date: During Google OAuth implementation

Status: Active

Context: Vercel serverless functions can have multiple concurrent warm instances. A module-level singleton oauth2Client would have its credentials mutated across concurrent invocations.

Decision: Create a new google.auth.OAuth2 instance on each request to /api/auth/google and /api/auth/google/callback.

Rationale: Prevents token cross-contamination between concurrent users in a serverless environment.

Tradeoffs: There is slightly higher memory allocation per request and constructor overhead (~1ms) on every OAuth request.

Consequences: The system remains stateless and safe for serverless deployment with no shared mutable state.

EDR-003: Embedded Cart in User Document

Date: Database design phase

Status: Active, with known limitations

Context: Cart management requires frequent reads and writes tied to a specific user's session.

Decision: Embed cart as array within User document rather than a separate Cart collection.

Rationale: Single-query access to user + cart. Simple ownership model. Appropriate for expected cart sizes (< 50 items).

Tradeoffs: User document bloat at scale. Write conflicts in multi-tab scenarios. Cannot support guest carts.

Future: Extract to Cart collection if guest checkout or cart analytics are required, or if cart sizes grow significantly.

EDR-004: Order Snapshot Denormalization

Date: Order schema design

Status: Active

Context: Orders must accurately reflect prices and product details at time of purchase, regardless of future product changes.

Decision: Copy productName, price, mrpPrice, sellingPrice, and shippingAddress into Order document at checkout time.

Rationale: Historical accuracy. If a seller changes a product price or deletes a product, past orders retain correct information. Standard e-commerce pattern at Shopify, Amazon, etc.

Tradeoffs: Data duplication between Product and Order collections. Product information in orders can become stale relative to current product (but this is intentional, not a bug).

Consequences: Order display should use order snapshot data, not re-fetch from Product collection (except for images/description which are currently re-fetched -- see Section 09.4).

EDR-005: Context API over Redux

Date: Frontend architecture design

Status: Active

Context: Global state in Browse-Mart is limited to five concerns: auth token, cart count, user detail, categories, theme.

Decision: Use React Context API with five focused providers.

Rationale: Redux adds boilerplate (actions, reducers, middleware, store configuration, devtools) that is not justified by the state complexity. Five contexts are simpler to understand, maintain, and onboard new developers to.

Tradeoffs: Context value changes trigger re-renders of all consumers (no selector optimization like useSelector in Redux). For large consumer trees, this can cause performance issues.

Future: If performance profiling shows excessive re-renders from context changes, migrate high-frequency state (cart count) to Zustand (lightweight store with selector support).

EDR-006: JWT localStorage Storage

Date: Authentication design

Status: Active (acknowledged security debt)

Context: JWT tokens must persist across browser refreshes.

Decision: Store JWT in localStorage.

Rationale: Simple implementation. Works without HttpOnly cookie configuration and CSRF protection overhead.

Tradeoffs: XSS attack can read localStorage and steal the JWT.

Mitigation: CSP headers (not yet implemented).

Future: Migrate to HttpOnly cookie storage with CSRF token protection as part of the security hardening sprint.

EDR-007: OTP-Based Email Verification

Date: Authentication design

Status: Active

Context: Email verification is required before account activation.

Decision: 6-digit numeric OTP with 10-minute expiry, bcrypt-hashed at rest.

Rationale: Works in email clients that strip URLs (magic links require clickable links). OTP is user-memorizable. bcrypt ensures that even if the DB is compromised, OTPs are not readable.

Tradeoffs: bcrypt comparison is ~65ms (intentionally slow). Under OTP brute-force attacks, this creates CPU pressure. Math.random() is not cryptographically secure (known gap).

Future: Migrate to crypto.randomInt(). Add rate limiting on OTP verification endpoint.

=====

APPENDIX C: DEPENDENCY MANIFEST & RATIONALE

Dependency	Purpose	Rationale
BACKEND DEPENDENCIES		
express@4.x	HTTP server framework, routing, middleware pipeline	De-facto standard for Node.js APIs. Minimal, unopinionated. Extensive middleware ecosystem. Vercel-compatible.
mongoose@8.x	MongoDB ODM (Object Document Mapper)	Schema validation, pre-save hooks, instance methods, query building. Abstracts MongoDB driver complexity.
jsonwebtoken	JWT signing and verification	Industry standard JWT implementation for Node.js.
bcrypt	Password hashing	Purpose-built slow hash for passwords. Automatic salt

Dependency	Purpose	Rationale
		generation. Industry standard for password storage.
clouddinary (v2 SDK)	Image upload, storage, and CDN delivery	Official SDK. Supports streaming upload from Buffer (required for serverless -- no disk access). Provides transformation API.
nodemailer	Transactional email via SMTP	Mature, widely-used email library. Flexible transport configuration (supports Gmail, SendGrid, AWS SES, etc.).
multer	File upload middleware for Express	Standard Express file upload solution. MemoryStorage mode is serverless-compatible.
cors	Cross-Origin Resource Sharing middleware	Required because frontend and backend are on different origins.
googleapis	Google OAuth2 authentication	Official Google API client library. Provides OAuth2 client with token exchange and credential management.
dotenv	Environment variable loading for local development	Standard Node.js env var management. Not needed in production (Vercel injects env vars directly).
FRONTEND DEPENDENCIES		
react@19.1	UI rendering library	Industry standard. Concurrent renderer for smooth UX.
react-router-dom@7	Client-side routing	Standard React routing library. v7 supports data loading APIs.
axios	HTTP client	Better API than fetch (interceptors, automatic JSON parsing, cancellation, timeout). Consistent cross-browser behavior.

Dependency	Purpose	Rationale
tailwindcss@3.4	Utility-first CSS framework	Eliminates custom CSS. JIT compiler removes unused classes. Consistent design system via configuration.
vite@6	Build tool and dev server	10x faster than webpack for HMR. Native ES modules. Excellent plugin ecosystem.
vite-plugin-pwa	PWA (Service Worker + manifest) generation	Integrates Workbox with Vite build pipeline. Zero-config caching strategy configuration.
sweetalert2	User notification dialogs and toasts	Accessible, customizable, themeable. Better than native alert().
react-dropzone	Drag-and-drop file upload	Accessible file input with drag-and-drop. Handles multiple files, file type filtering, size validation.
lucide-react	Icon library	Tree-shakeable (only imported icons are bundled). Consistent design language. React-native SVG components.

=====

=====

APPENDIX D: CONFIGURATION REFERENCE

D.1 Backend Environment Variables

Variable	Required	Description
MONGO_URI	YES	Full MongoDB Atlas connection string including credentials, cluster host, and database name. Format: mongodb+srv://user:pass@cluster/dbname
JWT_SECRET	YES	HMAC-SHA256 signing secret for JWT tokens. Must be >= 32 characters, high entropy. Rotating this invalidates all active tokens.

Variable	Required	Description
CLOUDINARY_API_KEY	YES	Cloudinary API key
CLOUDINARY_API_SECRET	YES	Cloudinary API secret (treat as sensitive)
EMAIL_USER	YES	Gmail address used for SMTP sending
EMAIL_PASS	YES	Gmail App Password (not account password). Generate at myaccount.google.com/apppasswords .
GOOGLE_CLIENT_ID	YES	OAuth2 client ID from Google Cloud Console
GOOGLE_CLIENT_SECRET	YES	OAuth2 client secret (sensitive)
GOOGLE_REDIRECT_URI	YES	Callback URL (must match registered URIs in Google Cloud Console)
FRONTEND_URL	YES	Full URL of frontend deployment (e.g., https://browse-mart.vercel.app). Used for CORS allowed origin.
NODE_ENV	NO	"production" or "development". Controls error verbosity and debug behavior.

D.2 Frontend Environment Variables

Variable	Required	Description
VITE_BACKEND_URL	YES	Full URL of backend API deployment (e.g., https://browse-mart-api.vercel.app)
VITE_CLOUD_NAME	YES	Cloudinary cloud name for client-side URL construction (non-sensitive)

D.3 Vercel Configuration Files

Backend (vercel.json):

```
JSON
{
  "version": 2,
```

```
"builds": [{ "src": "/src/server.js", "use": "@vercel/node"
}],
"routes": [{ "src": "/(.*)", "dest": "/src/server.js" }]
}
```

Explanation: All HTTP routes are forwarded to the Express app in `server.js`.

Frontend (vercel.json):

```
JSON
{
  "rewrites": [{ "source": "/(.*)", "destination":
"/index.html" }]
}
```


PROJECT SOURCE CODE

File Name: frontend\src\Component\Admin\ProductManagement\AdminProductList.jsx

```
1. import React, { useState, useEffect } from "react";
2. import axios from "axios";
3. import { useAuth } from "../../../Context/authContext";
4. import { useTheme } from "../../../Context/themeContext";
5. import { MdSearch } from "react-icons/md";
6.
7. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
8.
9. const AdminProductList = () => {
10.   const { authToken } = useAuth();
11.   const { theme } = useTheme();
12.   const [products, setProducts] = useState([]);
13.   const [loading, setLoading] = useState(true);
14.   const [page, setPage] = useState(1);
15.   const [totalPages, setTotalPages] = useState(1);
16.   const [searchTerm, setSearchTerm] = useState("");
17.
18.   const fetchProducts = async () => {
19.     try {
20.       setLoading(true);
21.       const { data } = await axios.get(
22.         `${SERVER_URL}/api/admin/products?page=${page}&limit=10&search=${searchTerm}
23.       `,
24.       { headers: { Authorization: `Bearer ${authToken}` } }
25.     );
26.     if (data.success) {
27.       setProducts(data.products);
28.       setTotalPages(data.totalPages);
29.     }
30.   } catch (err) {
31.     console.error("Failed to fetch products", err);
32.   } finally {
33.     setLoading(false);
34.   }
35.
36.   useEffect(() => {
37.     if (authToken) {
38.       const delayDebounceFn = setTimeout(() => {
39.         fetchProducts();
40.       }, 500);
41.       return () => clearTimeout(delayDebounceFn);
42.     }
43.   }, [authToken, page, searchTerm]);
44.
45.   return (
46.     <div className={` ${theme === 'dark' ? 'bg-gray-800 border-gray-700'
47. : 'bg-white border-gray-100'} rounded-2xl shadow-sm border p-8 animate-
48. fade-in-up transition-colors`} >
49.       <div className="flex flex-col md:flex-row justify-between items-
50. center mb-8 gap-4">
51.         <div>
52.           <h2 className={`text-2xl font-black ${theme === 'dark' ?
53. 'text-white' : 'text-gray-800'}`}>Global Inventory</h2>
```

```

50.         <p className={`text-sm mt-1 ${theme === 'dark' ? 'text-gray-
400' : 'text-gray-500'}}`>Platform-wide overview of all products and
sellers.</p>
51.     </div>
52.
53.     <div className="relative w-full md:w-96">
54.         <MdSearch className="absolute left-4 top-1/2 -translate-y-1/2
text-gray-400 text-xl" />
55.         <input
56.             type="text"
57.             placeholder="Search products by title..."
58.             value={searchTerm}
59.             onChange={(e) => { setSearchTerm(e.target.value);
setPage(1); }}
60.             className={`w-full pl-12 pr-4 py-3 border rounded-xl
focus:outline-none focus:ring-2 focus:ring-blue-500/20 focus:border-blue-
500 transition-all ${
61.                 theme === 'dark' ? 'bg-gray-900 border-gray-700 text-
white placeholder-gray-500' : 'bg-gray-50 border-gray-200 text-gray-900'
62.             }`>
63.         </div>
64.     </div>
65.
66.
67.     <div className="overflow-x-auto">
68.         <table className="w-full text-left border-collapse">
69.             <thead>
70.                 <tr className={` ${theme === 'dark' ? 'bg-gray-900 border-
gray-700' : 'bg-gray-50 border-gray-100'} border-b text-xs uppercase
tracking-wider text-gray-500 font-bold`}>
71.                     <th className="p-4 rounded-tl-xl w-16">Preview</th>
72.                     <th className="p-4">Product Specs</th>
73.                     <th className="p-4">Business / Seller</th>
74.                     <th className="p-4">Pricing</th>
75.                     <th className="p-4 text-right rounded-tr-
xl">Inventory</th>
76.                 </tr>
77.             </thead>
78.             <tbody className={`divide-y ${theme === 'dark' ? 'divide-
gray-700/60' : 'divide-gray-100/60'} ${loading ? 'opacity-50 pointer-
events-none' : ''} transition-opacity duration-200`}>
79.                 {loading && products.length === 0 ? (
80.                     <tr><td colspan="5" className="p-8 text-center text-gray-
400">Loading catalog...</td></tr>
81.                 ) : !loading && products.length === 0 ? (
82.                     <tr><td colspan="5" className="p-8 text-center text-gray-
400">No products match your search.</td></tr>
83.                 ) : (
84.                     products.map((t) => (
85.                         <tr key={t._id} className={` ${theme === 'dark' ?
'hover:bg-gray-700/50' : 'hover:bg-gray-50/50'} transition-colors`}>
86.                             <td className="p-4">
87.                                 <img src={t.image?.[0] ||
'https://via.placeholder.com/64'} alt={t.name} className={`w-14 h-14
object-cover rounded-lg border ${theme === 'dark' ? 'border-gray-700 bg-
gray-900' : 'border-gray-200 bg-white'}}` />
88.                             </td>
89.                             <td className="p-4">
90.                                 <div className={`font-bold max-w-xs truncate
${theme === 'dark' ? 'text-white' : 'text-gray-800'}}`
title={t.name}>>{t.name}</div>

```

```

91.                <div className="text-xs text-gray-400 mt-1
uppercase tracking-widest">{t.category?.name}</div>
92.                </td>
93.                <td className="p-4">
94.                    <div className={`font-semibold ${theme === 'dark' ?
'text-gray-300' : 'text-gray-700'}`}>{t.sellerId?.businessName || "Unknown
Business"}</div>
95.                    <div className="text-xs text-blue-
500">{t.sellerId?.emailAddress || "N/A"}</div>
96.                </td>
97.                <td className="p-4">
98.                    <div className={`font-black ${theme === 'dark' ?
'text-white' : 'text-gray-800'}`}>₹{t.mrpPrice || t.price || 0}</div>
99.                    {t.sellingPrice && (t.sellingPrice < (t.mrpPrice ||
t.price)) && (
100.                        <div className="text-xs text-emerald-500 font-
bold mt-1">Sale: ₹{t.sellingPrice}</div>
101.                    )}
102.                </td>
103.                <td className="p-4 text-right">
104.                    <span className={`px-3 py-1 rounded-full text-xs
font-bold inline-block border ${
105.                        t.stock > 10 ? 'bg-emerald-50 text-emerald-600
border-emerald-100' : 'bg-orange-50 text-orange-600 border-orange-100'
106.                    }`}>
107.                        {t.stock} in stock
108.                    </span>
109.                </td>
110.            </tr>
111.        ))
112.    )}
113.    </tbody>
114.    </table>
115.    </div>
116.
117.    {/* Pagination Controls */}
118.    {totalPages > 1 && (
119.        <div className={`mt-8 flex justify-between items-center pt-6
border-t ${theme === 'dark' ? 'border-gray-700' : 'border-gray-100'}`}>
120.            <p className="text-sm text-gray-500 font-medium">Page {page}
of {totalPages}</p>
121.            <div className="flex gap-2">
122.                <button
123.                    disabled={page === 1}
124.                    onClick={() => setPage(page - 1)}
125.                    className={`px-4 py-2 border rounded-lg text-sm font-
medium disabled:opacity-50 transition-colors ${
126.                        theme === 'dark' ? 'border-gray-600 text-gray-300
hover:bg-gray-700' : 'border-gray-200 text-gray-700 hover:bg-gray-50'
127.                    }`}
128.                >
129.                    Previous
130.                </button>
131.                <button
132.                    disabled={page === totalPages}
133.                    onClick={() => setPage(page + 1)}
134.                    className={`px-4 py-2 border rounded-lg text-sm font-
medium disabled:opacity-50 transition-colors ${
135.                        theme === 'dark' ? 'border-gray-600 text-gray-300
hover:bg-gray-700' : 'border-gray-200 text-gray-700 hover:bg-gray-50'
136.                    }`}

```

```

137.         >
138.         Next
139.     </button>
140. </div>
141. </div>
142.     })
143. </div>
144. );
145. };
146.
147. export default AdminProductList;
148.

```

File Name: frontend\src\Component\App\App.jsx

```

1. import './App.css';
2. import { lazy, Suspense, useEffect } from "react";
3. import { createBrowserRouter, RouterProvider } from "react-router-dom";
4. import axios from "axios";
5. import { Navigate } from "react-router-dom";
6. import { useCart } from "../../Context/cartContext";
7. import { useAuth } from "../../Context/authContext";
8. import { useUser } from "../../Context/userContext";
9. import { useTheme } from "../../Context/themeContext";
10. import { ConsumerRoutes, SellerRoutes } from "../../routes";
11. import AdminRoutes from "../../routes/AdminRoutes";
12.
13. const AppLayout = lazy(() => import("../AppLayout/AppLayout"));
14. const Login = lazy(() => import("../Pages/Login"));
15. const RegisterPage = lazy(() => import("../Pages/RegisterPage"));
16. const ForgetPasswordPage = lazy(() =>
import("../Pages/ForgetPasswordPage"));
17. const GoogleCallback = lazy(() => import("../Pages/GoogleCallback"));
18. const Profile1 = lazy(() => import("../Profile/profile1"));
19. const ProfileIndexRedirect = lazy(() =>
20.   import("../Profile/profile1").then((module) => ({
21.     default: module.ProfileIndexRedirect,
22.   })),
23. );
24. const ProfileOrdersPage = lazy(() =>
25.   import("../Profile/profile1").then((module) => ({
26.     default: module.ProfileOrdersPage,
27.   })),
28. );
29. const ProfileOverviewPage = lazy(() =>
30.   import("../Profile/profile1").then((module) => ({
31.     default: module.ProfileOverviewPage,
32.   })),
33. );
34. const ProfileSecurityPage = lazy(() =>
35.   import("../Profile/profile1").then((module) => ({
36.     default: module.ProfileSecurityPage,
37.   })),
38. );
39. const ProfileWishlistPage = lazy(() =>
40.   import("../Profile/profile1").then((module) => ({
41.     default: module.ProfileWishlistPage,
42.   })),
43. );

```

```

44.
45. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
46.
47. // Global Security Interceptor
48. axios.interceptors.response.use(
49.   (response) => response,
50.   (error) => {
51.     if (error.response && error.response.status === 403) {
52.       const message = error.response.data?.message || "";
53.       if (message.includes("suspended")) {
54.         // Enforce block mechanism globally
55.         localStorage.removeItem("AuthToken");
56.         window.location.href =
57.           `/login?error=${encodeURIComponent(message)}`;
58.       }
59.       return Promise.reject(error);
60.     },
61.   );
62.
63. const router = createBrowserRouter([
64.   {
65.     path: "/login",
66.     element: <Login />,
67.   },
68.   {
69.     path: "/register",
70.     element: <RegisterPage />,
71.   },
72.   {
73.     path: "/forget-password",
74.     element: <ForgetPasswordPage />,
75.   },
76.   {
77.     path: "/oauth/callback",
78.     element: <GoogleCallback />,
79.   },
80.   {
81.     path: "/",
82.     element: <AppLayout />,
83.     children: [
84.       ...ConsumerRoutes,
85.       SellerRoutes,
86.       AdminRoutes,
87.       {
88.         path: "/profile",
89.         element: <Profile1 />,
90.         children: [
91.           {
92.             index: true,
93.             element: <ProfileIndexRedirect />,
94.           },
95.           {
96.             path: "overview",
97.             element: <ProfileOverviewPage />,
98.           },
99.           {
100.            path: "security",
101.            element: <ProfileSecurityPage />,
102.          },
103.        ]

```

```

104.         path: "orders",
105.         element: <ProfileOrdersPage />,
106.     },
107.     {
108.         path: "wishlist",
109.         element: <ProfileWishlistPage />,
110.     },
111.     {
112.         path: "*",
113.         element: <Navigate to="/profile/overview" replace />,
114.     },
115. ],
116. },
117. {
118.     path: "*",
119.     element: <Navigate to="/" />,
120. },
121. ],
122. },
123. ]);
124.
125. const App = () => {
126.     const { setUserDetail } = useUser();
127.     const { setCartCount } = useCart();
128.     const { authToken, setAuthToken } = useAuth();
129.     const { theme } = useTheme();
130.     const getUserDetail = async () => {
131.         try {
132.             const response = await axios({
133.                 method: "POST",
134.                 url: `${SERVER_URL}/api/user/profile`,
135.                 headers: {
136.                     "Content-type": "application/json; charset=UTF-8",
137.                     Authorization: `Bearer ${authToken}`,
138.                 },
139.             });
140.
141.             const { data } = response;
142.             setUserDetail(data?.userDetail);
143.             if (data?.userDetail?.cartCount) {
144.                 setCartCount(data?.userDetail?.cartCount);
145.             }
146.         } catch (error) {
147.             console.log(error?.response?.data?.message);
148.             if (error?.response?.data?.message === "Token expired") {
149.                 localStorage.removeItem("AuthToken");
150.                 setAuthToken(null);
151.             }
152.         }
153.     };
154.
155.     String.prototype.toCapitalise = function () {
156.         if (this.length === 0) return "";
157.         const words = this.split(" ");
158.         for (let i = 0; i < words.length; i++) {
159.             words[i] = words[i].charAt(0).toUpperCase() + words[i].slice(1);
160.         }
161.         return words.join(" ");
162.     };
163.     useEffect(() => {
164.         if (authToken) {

```

```

165.     getUserDetail();
166.   } else {
167.     setUserDetail(null);
168.   }
169.   // eslint-disable-next-line react-hooks/exhaustive-deps
170. }, [authToken]);
171.
172. return (
173.   <Suspense
174.     fallback={
175.       <div
176.         className={`app-route-loader ${
177.           theme === "dark" ? "bg-gray-900" : "bg-white"
178.         } flex items-center justify-center`}
179.       >
180.         <div
181.           className={`animate-spin rounded-full h-12 w-12 border-4
182.             ${
183.               theme === "dark"
184.                 ? "border-gray-700 border-t-white"
185.                 : "border-gray-300 border-t-gray-900"
186.             }}
187.         ></div>
188.       </div>
189.     }
190.   >
191.     <RouterProvider router={router} />
192.   </Suspense>
193. );
194. };
195. export default App;
196.

```

File Name: frontend\src\Component\Cart\Cart.jsx

```

1. import axios from "axios";
2. import React, { useEffect, useState } from "react";
3. import { FiArrowRight, FiCreditCard, FiShield, FiShoppingBag } from
"react-icons/fi";
4. import { Link, useNavigate } from "react-router-dom";
5. import CartCard from "../CartCard";
6. import EmptyCart from "../EmptyCart";
7. import { Loader, ServerError, Button } from "../../LIBS";
8. import { formatNumber } from "../../utility/constant";
9. import { useTheme } from "../../Context/themeContext";
10. import { useAuth } from "../../Context/authContext";
11. import { useUser } from "../../Context/userContext";
12.
13. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
14.
15. const Cart = () => {
16.   const { theme } = useTheme();
17.   const { authToken } = useAuth();
18.   const { userDetails } = useUser();
19.   const navigate = useNavigate();
20.   const [cartItem, setCartItem] = useState([]);
21.   const [isDataFetching, setIsDataFetching] = useState(false);
22.   const [error, setError] = useState(null);

```

```

23.   const isDark = theme === "dark";
24.
25.   const getCartItem = async () => {
26.     try {
27.       setIsDataFetching(true);
28.       const response = await axios.post(
29.         `${SERVER_URL}/api/user/get-cart-items`,
30.         { userId: userDetails?.id },
31.         { headers: { Authorization: `Bearer ${authToken}` } }
32.       );
33.       setCartItem(response?.data?.cartProduct || []);
34.     } catch (requestError) {
35.       const { data, status } = requestError?.response || {};
36.       if (status === 404) {
37.         setError({ message: data?.message, status });
38.       } else {
39.         setError({ error: "Internal Server Error", status: 500 });
40.       }
41.     } finally {
42.       setIsDataFetching(false);
43.     }
44.   };
45.
46.   useEffect(() => {
47.     if (!authToken) {
48.       navigate("/login");
49.     } else {
50.       getCartItem();
51.     }
52.   }, [authToken, navigate]);
53.
54.   useEffect(() => {
55.     window.scrollTo(0, 0);
56.   }, []);
57.
58.   const subtotal = cartItem?.reduce((total, product) => {
59.     return total + (product?.mrpPrice || product?.price || 0) *
60.     (product?.quantity || 0);
61.   }, 0);
62.
63.   const totalDiscount = cartItem?.reduce((total, product) => {
64.     return (
65.       total +
66.       ((product?.mrpPrice || product?.price || 0) -
67.       (product?.sellingPrice || product?.price || 0)) *
68.       (product?.quantity || 0)
69.     );
70.   }, 0);
71.
72.   const finalTotal = cartItem?.reduce((total, product) => {
73.     return total + (product?.sellingPrice || product?.price || 0) *
74.     (product?.quantity || 0);
75.   }, 0);
76.
77.   if (isDataFetching) return <Loader />;
78.   if (error?.status === 500) return <ServerError />;
79.   if (!isDataFetching && cartItem?.length === 0) return <EmptyCart />;
80.
81.   return (
82.     <div
83.       className={`min-h-screen transition-all duration-300 ${

```



```

81.         isDark ? "bg-gray-900 text-white" : "bg-slate-50 text-gray-900"
82.     }` }
83.     >
84.     <div className="mx-auto flex max-w-7xl flex-col gap-6 px-4 py-6
mobile:px-4 tablet:px-6 laptop:px-8">
85.         <section
86.             className={`overflow-hidden rounded-[28px] border px-6 py-8
shadow-sm transition-all duration-300 mobile:px-4 ${
87.                 isDark
88.                 ? "border-gray-800 bg-gradient-to-br from-gray-900 via-
gray-800 to-indigo-950"
89.                 : "border-indigo-100 bg-gradient-to-br from-white via-
indigo-50 to-sky-50"
90.             }` }
91.         >
92.             <div className="flex flex-col gap-8 laptop:flex-row
laptop:items-end laptop:justify-between">
93.                 <div className="max-w-2xl">
94.                     <span
95.                         className={`inline-flex items-center gap-2 rounded-full
px-4 py-2 text-xs font-semibold uppercase tracking-[0.2em] ${
96.                             isDark
97.                             ? "bg-white/10 text-indigo-200"
98.                             : "bg-white text-indigo-600 shadow-sm"
99.                         }` }
100.                     >
101.                         <FiShoppingBag />
102.                         Your cart
103.                     </span>
104.                     <h1 className="mt-4 font-roboto text-3xl font-bold
mobile:text-2xl">
105.                         Review everything before checkout.
106.                     </h1>
107.                     <p
108.                         className={`mt-3 max-w-xl font-roboto text-sm leading-
6 ${
109.                             isDark ? "text-gray-300" : "text-gray-600"
110.                         }` }
111.                     >
112.                         A cleaner summary, quick quantity controls, and a
checkout panel that
113.                         matches the rest of Browse Mart.
114.                     </p>
115.                 </div>
116.
117.                 <div className="grid gap-3 small-device:grid-cols-3">
118.                     <div
119.                         className={`rounded-2xl border px-4 py-4 backdrop-blur
${
120.                             isDark
121.                             ? "border-white/10 bg-white/5"
122.                             : "border-white bg-white/80"
123.                         }` }
124.                     >
125.                         <p className={`text-xs uppercase tracking-[0.18em]
${isDark ? "text-gray-400" : "text-gray-500"}`}>
126.                             Items
127.                         </p>
128.                         <p className="mt-2 text-2xl font-
bold">{cartItem?.length}</p>
129.                     </div>

```

```

130.         <div
131.             className={`rounded-2xl border px-4 py-4 backdrop-blur
132.             ${
133.                 isDark
134.                 ? "border-white/10 bg-white/5"
135.                 : "border-white bg-white/80"
136.             }`}
137.             >
138.                 <p className={`text-xs uppercase tracking-[0.18em]
139.                 ${isDark ? "text-gray-400" : "text-gray-500"}}`}>
140.                     Savings
141.                 </p>
142.                 <p className="mt-2 text-2xl font-bold text-emerald-
143.                 500">
144.                     {formatNumber(totalDiscount || 0)}
145.                 </p>
146.             </div>
147.             <div
148.                 className={`rounded-2xl border px-4 py-4 backdrop-blur
149.                 ${
150.                     isDark
151.                     ? "border-white/10 bg-white/5"
152.                     : "border-white bg-white/80"
153.                 }`}
154.                 >
155.                     <p className={`text-xs uppercase tracking-[0.18em]
156.                     ${isDark ? "text-gray-400" : "text-gray-500"}}`}>
157.                         Payable
158.                     </p>
159.                     <p className="mt-2 text-2xl font-
160.                     bold">{formatNumber(finalTotal || 0)}</p>
161.                 </div>
162.             </div>
163.         </div>
164.     </section>
165.
166.     <div className="grid gap-6 laptop:grid-cols-
167.     [minmax(0,1.6fr)_380px]">
168.         <section
169.             className={`rounded-[28px] border p-4 shadow-sm
170.             transition-all duration-300 mobile:p-3 ${
171.                 isDark ? "border-gray-800 bg-gray-950/70" : "border-
172.                 gray-200 bg-white"
173.             }`}
174.             >
175.             <div
176.                 className={`mb-4 flex items-center justify-between
177.                 rounded-2xl border px-4 py-4 ${
178.                     isDark ? "border-gray-800 bg-gray-900" : "border-gray-
179.                     200 bg-gray-50"
180.                 }`}
181.                 >
182.                 <div>
183.                     <h2 className="font-roboto text-2xl font-bold
184.                     mobile:text-xl">Shopping Cart</h2>
185.                     <p className={`mt-1 text-sm ${isDark ? "text-gray-400"
186.                     : "text-gray-500"}}`}>
187.                         Adjust quantities, remove products, or continue
188.                         shopping.
189.                     </p>
190.                 </div>

```

```

177.         <Link
178.             to="/products"
179.             className={`hidden rounded-full px-4 py-2 text-sm
font-semibold transition-all duration-300 small-device:inline-flex ${
180.                 isDark
181.                 ? "bg-gray-800 text-white hover:bg-gray-700"
182.                 : "bg-indigo-50 text-indigo-600 hover:bg-indigo-
100"
183.             }`}
184.         >
185.             Continue Shopping
186.         </Link>
187.     </div>
188.
189.     <div className="flex flex-col gap-4">
190.         {cartItem?.map((product) => (
191.             <CartCard
192.                 product={product}
193.                 key={product?._id || product?.id}
194.                 authToken={authToken}
195.                 setCartItem={setCartItem}
196.             />
197.         ))}
198.     </div>
199. </section>
200.
201.     <aside className="flex flex-col gap-4">
202.         <div
203.             className={`rounded-[28px] border p-6 shadow-sm
transition-all duration-300 ${
204.                 isDark ? "border-gray-800 bg-gray-950/70" : "border-
gray-200 bg-white"
205.             }`}
206.         >
207.             <div className="flex items-center justify-between">
208.                 <h2 className="font-roboto text-2xl font-bold">Order
Summary</h2>
209.                 <FiCreditCard className={isDark ? "text-indigo-300" :
"text-indigo-500"} />
210.             </div>
211.
212.             <div className="mt-6 space-y-4">
213.                 <div className="flex items-center justify-between
text-sm">
214.                     <span className={isDark ? "text-gray-400" : "text-
gray-500"}>Subtotal</span>
215.                     <span className="font-
semibold">{formatNumber(subtotal || 0)}</span>
216.                 </div>
217.                 <div className="flex items-center justify-between
text-sm">
218.                     <span className={isDark ? "text-gray-400" : "text-
gray-500"}>Discount</span>
219.                     <span className="font-semibold text-emerald-500">-
{formatNumber(totalDiscount || 0)}</span>
220.                 </div>
221.                 <div className="flex items-center justify-between
text-sm">
222.                     <span className={isDark ? "text-gray-400" : "text-
gray-500"}>Delivery</span>

```

```

223.             <span className="font-semibold text-emerald-
500">Free</span>
224.             </div>
225.             <div className={`border-t pt-4 ${isDark ? "border-
gray-800" : "border-gray-200"}}`>
226.                 <div className="flex items-center justify-between">
227.                     <span className="text-base font-
semibold">Total</span>
228.                     <span className="text-2xl font-
bold">{formatNumber(finalTotal || 0)}</span>
229.                 </div>
230.                 <p className={`mt-2 text-xs ${isDark ? "text-gray-
400" : "text-gray-500"}}`>
231.                     Taxes included. No extra shipping charges on this
order.
232.                 </p>
233.             </div>
234.         </div>
235.
236.         <Link to="/checkout" className="mt-6 block">
237.             <Button
238.                 btnText="Proceed to Checkout"
239.                 icon={<FiArrowRight />}
240.                 iconPosition="right"
241.                 className="w-full rounded-2xl bg-indigo-600 px-5 py-
3 text-white hover:bg-indigo-700"
242.             />
243.         </Link>
244.
245.         <Link
246.             to="/products"
247.             className={`mt-3 inline-flex w-full items-center
justify-center rounded-2xl px-5 py-3 text-sm font-semibold transition-all
duration-300 small-device:hidden ${
248.                 isDark
249.                 ? "bg-gray-800 text-white hover:bg-gray-700"
250.                 : "bg-gray-100 text-gray-700 hover:bg-gray-200"
251.             }`>
252.             >
253.                 Continue Shopping
254.             </Link>
255.         </div>
256.
257.         <div
258.             className={`rounded-[28px] border p-5 transition-all
duration-300 ${
259.                 isDark ? "border-gray-800 bg-gray-950/70" : "border-
gray-200 bg-white"
260.             }`>
261.             >
262.                 <div className="flex items-start gap-3">
263.                     <div
264.                         className={`rounded-2xl p-3 ${isDark ? "bg-indigo-
900/40 text-indigo-300" : "bg-indigo-50 text-indigo-600"}}`>
265.                     >
266.                         <FiShield />
267.                     </div>
268.                 </div>
269.                 <h3 className="font-roboto text-base font-
semibold">Secure checkout</h3>

```

```

270.                <p className={`mt-1 text-sm leading-6 ${isDark ?
"text-gray-400" : "text-gray-500"}`}>
271.                    Payments stay protected and your items remain easy
to review before
272.                        you place the order.
273.                </p>
274.            </div>
275.        </div>
276.    </div>
277. </aside>
278. </div>
279. </div>
280. </div>
281. );
282. };
283.
284. export default Cart;
285.

```

File Name: frontend\src\Component\Cart\CartCard.jsx

```

1. import React, { useState } from "react";
2. import { FaMinus, FaPlus } from "react-icons/fa6";
3. import { FiTrash2 } from "react-icons/fi";
4. import axios from "axios";
5. import Button from "../../LIBS/Button";
6. import {
7.     formatNumber,
8.     swalWithCustomConfiguration,
9. } from "../../utility/constant";
10. import { useCart } from "../../Context/cartContext";
11. import { useTheme } from "../../Context/themeContext";
12.
13. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
14.
15. const CartCard = ({ product, authToken, setCartItem }) => {
16.     const { setCartCount } = useCart();
17.     const { theme } = useTheme();
18.     const [isUpdating, setIsUpdating] = useState(false);
19.     const isDark = theme === "dark";
20.     const sellingPrice =
21.         product?.sellingPrice || product?.price || product?.mrpPrice || 0;
22.
23.     const updateCart = async (newQuantity = 0) => {
24.         try {
25.             setIsUpdating(true);
26.             const res = await axios({
27.                 method: "POST",
28.                 url: `${SERVER_URL}/api/user/update-cart`,
29.                 data: { productId: product?.id, quantity: newQuantity },
30.                 headers: { Authorization: `Bearer ${authToken}` },
31.             });
32.
33.             setCartCount(res?.data?.cartCount);
34.
35.             if (newQuantity <= 0) {
36.                 setCartItem((prev) =>
37.                     prev.filter(
38.                         (item) =>

```

```

39.          (item?.id || item?._id) !==
40.          (res?.data?.removedProductId || product?.id ||
product?._id)
41.        )
42.      );
43.      return;
44.    }
45.
46.    if (res?.data?.updatedCartItem) {
47.      setCartItem(prev) =>
48.        prev.map((item) =>
49.          (item?.id || item?._id) ===
50.          (res?.data?.updatedCartItem?.id ||
res?.data?.updatedCartItem?._id)
51.            ? { ...item, ...res?.data?.updatedCartItem }
52.            : item
53.        )
54.      );
55.    }
56.  } catch (error) {
57.    console.log(error);
58.    swalWithCustomConfiguration?.fire(
59.      "Something went wrong!",
60.      "Cant update cart",
61.      "error"
62.    );
63.  } finally {
64.    setIsUpdating(false);
65.  }
66. };
67.
68. const handleIncrement = () => {
69.   if (isUpdating) return;
70.   updateCart((product?.quantity || 1) + 1);
71. };
72.
73. const handleDecrement = () => {
74.   if (isUpdating) return;
75.   updateCart((product?.quantity || 1) - 1);
76. };
77.
78. const handleRemoveClicked = () => {
79.   if (isUpdating) return;
80.   updateCart();
81. };
82.
83. return (
84.   <article
85.     className={`rounded-[24px] border p-4 transition-all duration-300
86.       ${
87.         isDark ? "border-gray-800 bg-gray-900" : "border-gray-200 bg-
gray-50/80"
88.       }`>
89.     <div className="flex gap-4 mobile:flex-col small-device:flex-
row">
90.       <div className="flex w-full gap-4">
91.         <div
92.           className={`h-28 w-28 flex-shrink-0 overflow-hidden
rounded-2xl border ${

```

```

93.             isDark ? "border-gray-700 bg-gray-800" : "border-gray-200
bg-white"
94.         }` }
95.     >
96.         <img
97.             src={product?.image || ""}
98.             alt={product?.name || ""}
99.             className="h-full w-full object-cover object-top"
100.         />
101.     </div>
102.
103.     <div className="flex min-w-0 flex-1 flex-col justify-between
gap-3">
104.         <div>
105.             <div className="flex items-start justify-between gap-3">
106.                 <div>
107.                     <h3 className="line-clamp-2 font-roboto text-lg
font-semibold">
108.                         {product?.name}
109.                     </h3>
110.                     <p
111.                         className={`mt-2 line-clamp-2 text-sm leading-6 ${
112.                             isDark ? "text-gray-400" : "text-gray-500"
113.                         }`>
114.                         >
115.                             {product?.description}
116.                         </p>
117.                     </div>
118.                     <Button
119.                         btnText="Remove"
120.                         icon={<FiTrash2 />}
121.                         className={`hidden rounded-full px-3 py-2 text-sm
small-device:inline-flex ${
122.                             isDark
123.                             ? "bg-gray-800 text-gray-200 hover:bg-gray-700"
124.                             : "bg-white text-gray-700 hover:bg-gray-100"
125.                         }`>
126.                             onClick={handleRemoveClicked}
127.                             disabled={isUpdating}
128.                         />
129.                     </div>
130.                 </div>
131.
132.                 <div className="flex flex-wrap items-end justify-between
gap-3">
133.                     <div className="flex flex-col">
134.                         <span className="font-roboto text-xl font-bold">
135.                             {formatNumber(sellingPrice)}
136.                         </span>
137.                         {product?.mrpPrice && product?.mrpPrice > sellingPrice
? (
138.                             <span
139.                                 className={`text-sm line-through ${
140.                                     isDark ? "text-gray-500" : "text-gray-400"
141.                                 }`}
142.                             >
143.                                 {formatNumber(product?.mrpPrice)}
144.                             </span>
145.                         ) : null}
146.                     </div>
147.

```

```

148.         <div className="flex items-center gap-3">
149.             <div
150.                 className={`flex items-center gap-2 rounded-full
border px-2 py-2 ${
151.                     isDark ? "border-gray-700 bg-gray-950" : "border-
gray-200 bg-white"
152.                 }`}
153.             >
154.                 <button
155.                     type="button"
156.                     className={`rounded-full p-2 transition-all
duration-300 ${
157.                         isDark ? "hover:bg-gray-800" : "hover:bg-gray-
100"
158.                     } ${isUpdating ? "pointer-events-none opacity-50"
: ""}`} `}
159.                     onClick={handleDecrement}
160.                 >
161.                     <FaMinus className="text-xs" />
162.                 </button>
163.                 <span
164.                     className={`min-w-10 rounded-full px-3 py-1 text-
center text-sm font-semibold ${
165.                         isDark ? "bg-gray-800 text-white" : "bg-gray-100
text-gray-900"
166.                     }`}
167.                 >
168.                     {product?.quantity}
169.                 </span>
170.                 <button
171.                     type="button"
172.                     className={`rounded-full p-2 transition-all
duration-300 ${
173.                         isDark ? "hover:bg-gray-800" : "hover:bg-gray-
100"
174.                     } ${isUpdating ? "pointer-events-none opacity-50"
: ""}`} `}
175.                     onClick={handleIncrement}
176.                 >
177.                     <FaPlus className="text-xs" />
178.                 </button>
179.             </div>
180.
181.             <div className="text-right">
182.                 <p
183.                     className={`text-xs uppercase tracking-[0.18em] ${
184.                         isDark ? "text-gray-500" : "text-gray-400"
185.                     }`}
186.                 >
187.                     Total
188.                 </p>
189.                 <p className="font-roboto text-lg font-bold">
190.                     {formatNumber(sellingPrice * (product?.quantity ||
0))}
191.                 </p>
192.             </div>
193.         </div>
194.     </div>
195.
196.     <Button
197.         btnText="Remove"

```



```

198.             icon={<FiTrash2 />}
199.             className={`rounded-full px-3 py-2 text-sm small-
device:hidden ${
200.                 isDark
201.                 ? "bg-gray-800 text-gray-200 hover:bg-gray-700"
202.                 : "bg-white text-gray-700 hover:bg-gray-100"
203.             }}
204.             onClick={handleRemoveClicked}
205.             disabled={isUpdating}
206.         />
207.     </div>
208. </div>
209. </div>
210. </article>
211. );
212. };
213.
214. export default CartCard;
215.

```

File Name: frontend\src\Component\Cart\EmptyCart.jsx

```

1. import React from "react";
2. import Button from "../../LIBS/Button";
3. import { Link } from "react-router-dom";
4. import { useTheme } from "../../Context/themeContext";
5. import emptyCartImage from "../../assets/images/emptyCart.png";
6.
7. const EmptyCart = () => {
8.     const { theme } = useTheme();
9.     return (
10.         <div
11.             // className={`flex gap-5 p-3 min-h-screen mobile:h-full
mobile:flex-col tablet:flex-row ${
12.                 // theme === "dark" ? "bg-gray-900 text-white" : "bg-white
text-gray-900"
13.                 // } transition-all duration-300 `}
14.             className={`min-h-screen flex justify-center items-center ${
15.                 theme === "dark" ? "bg-gray-900 text-white" : "bg-white text-
gray-900"
16.             } transition-all duration-300 `}
17.         >
18.             <div className="flex flex-col items-center gap-3">
19.                 <img
20.                     src={emptyCartImage}
21.                     alt="Empty Cart"
22.                     className="max-w-[350px] mobile:w-[200px] laptop:w-[300px]"
23.                 />
24.                 <h1 className="text-center text-2xl mobile:text-base
laptop:text-xl font-roboto">
25.                     Ohh... Your cart is empty!
26.                 </h1>
27.                 <Link to={"/"}>
28.                     <Button
29.                         btnText={"Shop Now"}
30.                         className={"font-roboto bg-indigo-600 text-white p-2
rounded"}
31.                     />
32.                 </Link>

```

```

33.         </div>
34.     </div>
35. );
36. };
37.
38. export default EmptyCart;
39.

```

File Name: frontend/src/Component/Pages/ProductPage.jsx

```

1. import React, { useEffect, useState } from "react";
2. import { useParams, Link, useLocation, useNavigate } from "react-router-dom";
3. import axios from "axios";
4. import { FaShare, FaShieldAlt, FaTruck, FaUndo } from "react-icons/fa";
5. import { MdVerified } from "react-icons/md";
6. import { BsBoxSeam, BsLightningChargeFill } from "react-icons/bs";
7. import { FiShoppingCart } from "react-icons/fi";
8. import ProductCard from "../Product/ProductCard";
9. import { Button, Loader, ServerError } from "../..//LIBS";
10. import { ReviewForm, ReviewCard } from "../Review";
11. import ProductImage from "../Product/ProductImage";
12. import { formatNumber, socialMedia } from "../..//utility/constant";
13. import AddToCartButton from "../..//utility/AddToCartButton";
14. import { useTheme } from "../..//Context/themeContext";
15. import { useAuth } from "../..//Context/authContext";
16. import pageNotFound from "../..//assets/images/pageNotFound.jpg";
17. import { useUser } from "../..//Context/userContext";
18.
19. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
20.
21. // — Trust Badge Component

```

```

22. const TrustBadge = ({ icon, label, sub, isDark }) => (
23.     <div className={`flex items-center gap-2 px-3 py-2 rounded-xl border
flex-1 min-w-[120px]
24.         ${isDark ? "border-gray-700 bg-gray-800/60" : "border-gray-200 bg-
gray-50"}`}>
25.         >
26.         <span className="text-indigo-500 text-xl flex-shrink-
0">{icon}</span>
27.         <div>
28.             <p className={`text-xs font-semibold font-roboto ${isDark ?
"text-white" : "text-gray-800"}`}>{label}</p>
29.             <p className={`text-[10px] font-roboto ${isDark ? "text-gray-400"
: "text-gray-500"}`}>{sub}</p>
30.         </div>
31.     </div>
32. );
33.
34. // — Star Rating Row

```

```

35. const StarRow = ({ rating, count, isDark }) => (
36.     <div className="flex items-center gap-2">
37.         <div className="flex gap-0.5">
38.             {[1, 2, 3, 4, 5].map((i) => (
39.                 <svg
40.                     key={i}

```

```

41.         className={`w-4 h-4 ${i <= Math.round(rating) ? "text-yellow-
400" : isDark ? "text-gray-600" : "text-gray-300"}`}
42.         fill="currentColor"
43.         viewBox="0 0 24 24"
44.     >
45.         <path d="M12 213.09 6.26L22 9.27l-5 4.87L18.18 22 12 18.77
5.82 22 7 14.14l-5-4.87 6.91-1.01z" />
46.     </svg>
47.     )})
48. </div>
49. {count > 0 && (
50.     <span className={`text-sm font-roboto font-semibold ${isDark ?
"text-indigo-400" : "text-indigo-600"}`}>
51.         {rating?.toFixed(1)} <span className={`font-normal ${isDark ?
"text-gray-400" : "text-gray-500"}`}>({count} reviews)</span>
52.     </span>
53. )}
54. </div>
55. );
56.
57. //

```

```

58. const ProductPage = () => {
59.     const currentURL = window.location.href;
60.     const { productId } = useParams();
61.     const [productData, setProductData] = useState({});
62.     const [relatedProduct, setRelatedProduct] = useState([]);
63.     const [isDataFetch, setIsDataFetch] = useState(false);
64.     const [isReviewClicked, setIsReviewClicked] = useState(false);
65.     const [isRefreshClicked, setIsRefreshClicked] = useState(false);
66.     const [isShareShow, setIsShareShow] = useState(false);
67.     const [isError, setIsError] = useState({});
68.     const [isExpended, setIsExpended] = useState(false);
69.     const [showStickyBar, setShowStickyBar] = useState(false);
70.     const location = useLocation();
71.     const message = `📢 Exciting News! 🌟\n\nI just discovered the
**${productData?.name}** and I can't stop raving about it! 🌟\n\n🌟 **Why
You'll Love It**:\n- Top-notch quality that speaks for itself!\n- Perfect
for tech enthusiasts.\n- Limited-time offer: Don't miss out! 🕒\n\n👉
Check it out here: ${currentURL}\n\n💬 Let me know what you think, and tag
your friends who need this in their lives!`;
72.     const navigate = useNavigate();
73.     const { theme } = useTheme();
74.     const { authToken } = useAuth();
75.     const { userDetails } = useUser();
76.     const isDark = theme === "dark";
77.
78.     const handleClick = () => {
79.         setIsReviewClicked((isReview) => !isReview);
80.     };
81.
82.     const getProductDataById = () => {
83.         setIsDataFetch(false);
84.         axios({
85.             method: "get",
86.             url: `${SERVER_URL}/api/product/${productId}`,
87.         })
88.             .then((response) => {
89.                 const { data, status } = response;

```

```

90.         if (status === 200) {
91.             setProductData(data?.product);
92.         }
93.         setIsDataFetch(true);
94.     })
95.     .catch((error) => {
96.         const { data, status } = error?.response || {};
97.         if (status === 404) {
98.             console.log("Product not found", data);
99.             setIsError({ message: data?.message, status: status });
100.            setIsDataFetch(true);
101.        } else {
102.            setIsError({ error: "Internal Server Error", status: 500 });
103.            setIsDataFetch(true);
104.        }
105.    });
106. };
107.
108. const addToRecentlyViewed = (product) => {
109.     let totalStarRating = 0;
110.     product?.review?.map((review) => (totalStarRating +=
review?.rating));
111.     let recentlyViewed =
112.         JSON.parse(localStorage.getItem("recentlyViewed")) || [];
113.     const modifiedProduct = {
114.         id: product?.id || product?._id,
115.         name: product?.name,
116.         price: product?.price,
117.         description: product?.description,
118.         image: product?.image?.[0],
119.         category: product?.category,
120.         stock: product?.stock,
121.         rating: Number(totalStarRating / product?.review?.length),
122.         ratingNumber: product?.review?.length,
123.         mrpPrice: product?.mrpPrice,
124.         sellingPrice: product?.sellingPrice,
125.         isAddedToWislist: false,
126.     };
127.
128.     recentlyViewed = recentlyViewed.filter(
129.         (p) => (p.id || p._id) !== modifiedProduct.id
130.     );
131.     recentlyViewed.unshift(modifiedProduct);
132.     if (recentlyViewed.length > 5) {
133.         recentlyViewed.pop();
134.     }
135.     localStorage.setItem("recentlyViewed",
JSON.stringify(recentlyViewed));
136. };
137.
138. const getRelatedProduct = async () => {
139.     try {
140.         const response = await axios.get(
141.             `${SERVER_URL}/api/product/get-related-product`,
142.             {
143.                 params: {
144.                     category: productData?.category?._id ||
productData?.category,
145.                     productId,
146.                 },
147.             }

```

```

148.         );
149.         setRelatedProduct(response?.data?.product);
150.     } catch (error) {
151.         console.error("Error fetching related products:", error);
152.     }
153. };
154.
155. // eslint-disable-next-line
156. useEffect(() => {
157.     getProductDataById();
158. }, [productId, isRefreshClicked]);
159. // eslint-disable-next-line
160. useEffect(() => {
161.     if (productData?.id || productData?._id) {
162.         addToRecentlyViewed(productData);
163.     }
164.     if (productData?.category) {
165.         getRelatedProduct();
166.     }
167. }, [productData]);
168. useEffect(() => {
169.     window.scrollTo(0, 0);
170. }, []);
171.
172. // Sticky Add-to-Cart bar scroll listener
173. useEffect(() => {
174.     const handleScroll = () => {
175.         setShowStickyBar(window.scrollY > 320);
176.     };
177.     window.addEventListener("scroll", handleScroll, { passive: true });
178.     return () => window.removeEventListener("scroll", handleScroll);
179. }, []);
180.
181. // — Computed values

```

```

182. const reviews = productData?.review || [];
183. const avgRating =
184.     reviews.length > 0
185.     ? reviews.reduce((sum, r) => sum + (r?.rating || 0), 0) /
reviews.length
186.     : 0;
187. const mrp = productData?.mrpPrice;
188. const selling = productData?.sellingPrice || productData?.price;
189. const discountPct =
190.     mrp && selling && mrp > selling
191.     ? Math.round(((mrp - selling) / mrp) * 100)
192.     : null;
193. const isLowStock = productData?.stock > 0 && productData?.stock <
10;
194.
195. // — Error / Loading states

```

```

196. if (!isDataFetch && !isReviewClicked) return <Loader />;
197. if (isError?.status === 404) {
198.     return (
199.         <div className={`w-full h-screen flex justify-center items-
center ${isDark ? "bg-gray-900 text-white" : "bg-gray-100"}`}>
200.             <div className="flex flex-col gap-4 items-center">
201.                 <img src={pageNotFound} className="h-[400px] rounded-3xl"
alt="Page Not Found" />

```

```

202.         <div className="font-roboto text-ellipsis text-lg">
203.             <p>Ohh....</p>
204.             <p className="">{isError?.message}</p>
205.         </div>
206.     </div>
207. </div>
208. );
209. }
210. if (isError?.status === 500) return <ServerError />;
211.
212. // — Main Render

```

```

213. return (
214.     productData && relatedProduct && (
215.         <div className={`product-page-section transition-all duration-
300 ${isDark ? "bg-gray-900 text-white" : "bg-white text-gray-900"}`}>
216.
217.             {/* — Sticky Add-to-Cart Bar

```

```

218.             <div className={`
219.                 fixed top-0 left-0 right-0 z-50
220.                 transition-all duration-300 ease-in-out shadow-lg
221.                 ${showStickyBar ? "translate-y-0 opacity-100" : "-translate-
y-full opacity-0"}
222.                 ${isDark ? "bg-gray-900/95 border-b border-gray-700" : "bg-
white/95 border-b border-gray-200"}
223.                 backdrop-blur-md
224.             `}>
225.                 <div className="max-w-7xl mx-auto px-4 py-3 flex items-
center justify-between gap-4">
226.                     <p className={`font-roboto font-semibold text-sm truncate
flex-1 ${isDark ? "text-white" : "text-gray-900"}`}>
227.                         {productData?.name}
228.                     </p>
229.                     <div className="flex items-center gap-3">
230.                         <span className={`font-bold text-lg font-roboto ${isDark
? "text-indigo-400" : "text-indigo-600"}`}>
231.                             {formatNumber(selling)}
232.                         </span>
233.                         {productData?.stock ? (
234.                             userDetails && authToken ? (
235.                                 <div className="flex gap-2">
236.                                     <AddToCartButton
237.                                         authToken={authToken}
238.                                         userDetails={userDetails}
239.                                         productId={productId}
240.                                         className="bg-indigo-600 hover:bg-indigo-700
text-white text-sm font-roboto px-4 py-2 rounded-lg transition-all
duration-200"
241.                                     />
242.                                     <Link to={"/product/buy/" + productId}>
243.                                         <button className="bg-orange-500 hover:bg-
orange-600 text-white text-sm font-roboto px-4 py-2 rounded-lg transition-
all duration-200 flex items-center gap-1">
244.                                             <BsLightningChargeFill className="text-xs" />
245.                                             Buy Now
246.                                         </button>
247.                                     </Link>
248.                                 </div>
249.                             ) : (

```

```

250.                <Link
to={`/login?redirect=${encodeURIComponent(location?.pathname)}`} >
251.                <button className="bg-indigo-600 hover:bg-indigo-
700 text-white text-sm font-roboto px-4 py-2 rounded-lg transition-all
duration-200">
252.                    Login to Buy
253.                </button>
254.            </Link>
255.        )
256.    ) : (
257.        <span className="text-red-500 text-sm font-semibold
font-roboto">Out of Stock</span>
258.    )}
259.    </div>
260. </div>
261. </div>
262.
263.    {/ * — Hero Section: Image + Info
    */}
264.    <div className="flex p-3 gap-3 mobile:flex-col tablet:flex-row
max-w-7xl mx-auto">
265.
266.        {/ * Product Image Gallery */}
267.        <ProductImage productData={productData} />
268.
269.        {/ * Product Info Panel */}
270.        <div className="product-description w-1/2 mobile:w-full
tablet:w-1/2 flex flex-col gap-4 py-4 pr-2">
271.
272.            {/ * Category Badge */}
273.            {productData?.category?.name && (
274.                <span className={`
275.                    inline-flex w-max items-center gap-1 text-xs font-
semibold px-3 py-1 rounded-full font-roboto
276.                    ${isDark ? "bg-indigo-900/50 text-indigo-300 border
border-indigo-700" : "bg-indigo-50 text-indigo-600 border border-indigo-
200"}
277.                `}>
278.                    <BsBoxSeam className="text-xs" />
279.                    {productData?.category?.name?.toUpperCase()}
280.                </span>
281.            )}
282.
283.            {/ * Product Title */}
284.            <h1 className={`product-name font-roboto font-bold text-
2xl tablet:text-3xl text-left leading-snug ${isDark ? "text-white" : "text-
gray-900"}`} >
285.                {productData?.name}
286.            </h1>
287.
288.            {/ * Ratings Row */}
289.            {reviews.length > 0 && (
290.                <StarRow rating={avgRating} count={reviews.length}
isDark={isDark} />
291.            )}
292.
293.            {/ * Divider */}
294.            <div className={`border-t ${isDark ? "border-gray-700" :
"border-gray-200"}`} />
295.
296.            {/ * Pricing Block */}

```

```

297.         <div className="flex flex-col gap-1">
298.             <div className="flex items-baseline gap-3 flex-wrap">
299.                 <span className={`text-3xl font-bold font-roboto
300. ${isDark ? "text-white" : "text-gray-900"}`}>
301.                     {formatNumber(selling)}
302.                 </span>
303.                 {mrp && mrp > selling && (
304.                     <span className={`text-lg line-through font-roboto
305. ${isDark ? "text-gray-500" : "text-gray-400"}`}>
306.                         {formatNumber(mrp)}
307.                     </span>
308.                 )}
309.                 {discountPct && (
310.                     <span className="text-sm font-bold px-2 py-0.5 bg-
311. green-100 text-green-700 rounded-full font-roboto">
312.                         {discountPct}% OFF
313.                     </span>
314.                 )}
315.             </div>
316.             <p className={`text-xs font-roboto ${isDark ? "text-
317. gray-400" : "text-gray-500"}`}>
318.                 Inclusive of all taxes
319.             </p>
320.         </div>
321.
322.         {/* Low Stock Urgency */}
323.         {isLowStock && (
324.             <div className="flex items-center gap-2 px-3 py-2 bg-
325. red-50 border border-red-200 rounded-xl">
326.                 <span className="text-red-500 text-sm">⚠️</span>
327.                 <p className="text-red-600 font-semibold text-sm font-
328. roboto">
329.                     Only {productData?.stock} left in stock – order
330.                     soon!
331.                 </p>
332.             </div>
333.         )}
334.
335.         {/* Out of Stock */}
336.         {!productData?.stock && (
337.             <div className="flex items-center gap-2 px-3 py-2 bg-
338. red-50 border border-red-200 rounded-xl">
339.                 <span className="text-red-500">✖️</span>
340.                 <p className="text-red-600 font-semibold text-sm font-
341. roboto">Currently Out of Stock</p>
342.             </div>
343.         )}
344.
345.         {/* Trust Badges */}
346.         <div className="flex flex-wrap gap-2">
347.             <TrustBadge icon={<FaTruck />} label="Free Delivery"
348. sub="On orders above ₹499" isDark={isDark} />
349.             <TrustBadge icon={<FaUndo />} label="Easy Returns"
350. sub="7-day return policy" isDark={isDark} />
351.             <TrustBadge icon={<FaShieldAlt />} label="Secure
352. Payment" sub="100% safe checkout" isDark={isDark} />
353.         </div>
354.
355.         {/* Divider */}
356.         <div className={`border-t ${isDark ? "border-gray-700" :
357. "border-gray-200"}`} />

```



```

345.
346.      {/* CTA Buttons */}
347.      <div className="buy-buttons flex gap-3 w-full">
348.          {userDetail && authToken ? (
349.              productData?.stock ? (
350.                  <div className="w-full flex gap-3">
351.                      <AddToCartButton
352.                          authToken={authToken}
353.                          userDetail={userDetail}
354.                          productId={productId}
355.                          className={`
356.                              flex-1 flex items-center justify-center gap-2
357.                              py-3 rounded-xl font-roboto font-semibold
text-base
358.                              border-2 border-indigo-500 text-indigo-600
359.                              hover:bg-indigo-50 active:bg-indigo-100
360.                              transition-all duration-200
361.                              ${isDark ? "text-indigo-400 hover:bg-indigo-
900/30 border-indigo-500" : ""}
362.                              `}
363.                      />
364.                      <Link to={"/product/buy/" + productId}
className="flex-1">
365.                          <button className="
366.                              w-full py-3 rounded-xl font-roboto font-
semibold text-base
367.                              bg-indigo-600 hover:bg-indigo-700 active:bg-
indigo-800
368.                              text-white flex items-center justify-center
gap-2
369.                              shadow-md hover:shadow-indigo-200 transition-
all duration-200
370.                              hover:scale-[1.02] active:scale-100
371.                              ">
372.                              <BsLightningChargeFill />
373.                              Buy Now
374.                              </button>
375.                              </Link>
376.                          </div>
377.                      ) : null
378.                  ) : (
379.                      <Link
to={"/login?redirect=${encodeURIComponent(location?.pathname)}"}
className="w-full">
380.                          <button className="
381.                              w-full py-3 rounded-xl font-roboto font-semibold
text-base
382.                              bg-indigo-600 hover:bg-indigo-700 text-white
383.                              flex items-center justify-center gap-2
384.                              shadow-md transition-all duration-200 hover:scale-
[1.02]
385.                              ">
386.                              <FiShoppingCart />
387.                              Login to Buy
388.                              </button>
389.                              </Link>
390.                          )}
391.                      </div>
392.
393.      {/* Delivery Estimate */}

```

```

394.         <div className={`flex items-center gap-2 text-sm font-
roboto ${isDark ? "text-gray-400" : "text-gray-600"}`} >
395.             <FaTruck className="text-indigo-500" />
396.             <span>Estimated delivery: <strong className={isDark ?
"text-white" : "text-gray-900"}>3-5 business days</strong></span>
397.         </div>
398.
399.         {/* Share Button */}
400.         <div className={`border-t pt-3 ${isDark ? "border-gray-
700" : "border-gray-200"}`} >
401.             <div className="flex flex-col gap-3">
402.                 {/* Share trigger */}
403.                 <button
404.                     onClick={() => setIsShareShow((prev) => !prev)}
405.                     className={`
406.                         flex items-center gap-2 w-max text-sm font-
semibold font-roboto
407.                         px-4 py-2 rounded-xl border transition-all
duration-200
408.                         ${isDark
409.                             ? "border-gray-600 text-gray-300 hover:border-
indigo-500 hover:text-indigo-400 hover:bg-indigo-900/20"
410.                             : "border-gray-200 text-gray-600 hover:border-
indigo-300 hover:text-indigo-600 hover:bg-indigo-50"
411.                         }
412.                     `}
413.                 >
414.                     <FaShare className={`transition-transform duration-
300 ${isShareShow ? "rotate-45" : "rotate-0"}`} />
415.                     {isShareShow ? "Close" : "Share this product"}
416.                 </button>
417.
418.                 {/* Animated share panel */}
419.                 <div className={`overflow-hidden transition-all
duration-300 ease-in-out ${
420.                     isShareShow ? "max-h-24 opacity-100" : "max-h-0
opacity-0"
421.                 }`} >
422.                     <div className="flex gap-2 flex-wrap">
423.                         {[
424.                             {
425.                                 name: "WhatsApp",
426.                                 href:
`https://wa.me/?text=${encodeURIComponent(message)}`,
427.                                 bg: "bg-[#25D366] hover:bg-[#1ebe5d]",
428.                                 icon: <svg className="w-4 h-4"
fill="currentColor" viewBox="0 0 24 24"><path d="M17.472 14.382c-.297-.149-
1.758-.867-2.03-.967-.273-.099-.471-.148-.6715-.197.297-.767.966-.94
1.164-.173.199-.347.223-.644.075-.297-.15-1.255-.463-2.39-1.475-.883-.788-
1.48-1.761-1.653-2.059-.173-.297-.018-.458.13-.606.134-.133.298-.347.446-
.52.149-.174.198-.298.298-.497.099-.198.05-.371-.025-.52-.075-.149-.669-
1.612-.916-2.207-.242-.579-.487-.5-.669-.51-.173-.008-.371-.01-.57-.01-.198
0-.52.074-.792.372-.272.297-1.04 1.016-1.04 2.479 0 1.462 1.065 2.875 1.213
3.074.149.198 2.096 3.2 5.077 4.487.709.306 1.262.489 1.694.625.712.227
1.36.195 1.871.118.571-.085 1.758-.719 2.006-1.413.248-.694.248-1.289.173-
1.413-.074-.124-.272-.198-.57-.347z"/><path d="M11.997 2C6.477 2 6.477 2
11.997c0 1.761.459 3.411 1.26 4.845L2 22.5.27-1.236A9.953 9.953 0 0011.997
22c5.52 0 9.997-4.477 9.997-9.997S17.517 2 11.997 2zm0 18.188a8.176 8.176 0
01-4.165-1.1381-.299-.178-3.128.733.745-3.063-.194-.311a8.188 8.188 0
119.041 3.957z"/></svg>,
429.                         },

```

```

430.          {
431.              name: "LinkedIn",
432.              href:
`https://www.linkedin.com/messaging/compose?message=${encodeURIComponent(me
ssage)}`,
433.              bg: "bg-[#0077B5] hover:bg-[#006399]",
434.              icon: <svg className="w-4 h-4"
fill="currentColor" viewBox="0 0 24 24"><path d="M20.447 20.452h-3.554v-
5.569c0-1.328-.027-3.037-1.852-3.037-1.853 0-2.136 1.445-2.136
2.939v5.667H9.351V9h3.414v1.561h.046c.477-.9 1.637-1.85 3.37-1.85 3.601 0
4.267 2.37 4.267 5.455v6.286zM5.337 7.433a2.062 2.062 0 01-2.063-2.065
2.064 2.064 0 112.063 2.065zm1.782 13.019H3.555V9h3.564v11.452zM22.225
0H1.771C.792 0 0 .774 0 1.729v20.542C0 23.227.792 24 1.771 24h20.451C23.2
24 24 23.227 24 22.271V1.729C24 .774 23.2 0 22.222 0h.003z"/></svg>,
435.          },
436.          {
437.              name: "Email",
438.              href:
`mailto:?body=${encodeURIComponent(message)}`,
439.              bg: "bg-[#EA4335] hover:bg-[#d33b2c]",
440.              icon: <svg className="w-4 h-4"
fill="currentColor" viewBox="0 0 24 24"><path d="M20 4H4c-1.1 0-2 .9-2
2v12c0 1.1.9 2 2 2h16c1.1 0 2-.9 2-2V6c0-1.1-.9-2-2-2zm0 4l-8 5-8-5V6l8 5
8-5v2z"/></svg>,
441.          },
442.      ].map((platform) => (
443.          <a
444.              key={platform.name}
445.              href={platform.href}
446.              target="_blank"
447.              rel="noopener noreferrer"
448.              onClick={(e) => e.stopPropagation()}
449.              className={`flex items-center gap-2 px-3 py-2
rounded-xl text-white text-xs font-semibold font-roboto transition-all
duration-200 hover:scale-105 active:scale-95 shadow-sm ${platform.bg}`}
450.          >
451.              {platform.icon}
452.              {platform.name}
453.          </a>
454.      )})
455.  </div>
456. </div>
457. </div>
458. </div>
459.
460. </div>
461. </div>
462.
463. {/* — Product Description
464.      <div className={`mx-auto max-w-7xl px-5 py-6 rounded-2xl mx-3
my-4
465.      ${isDark ? "bg-gray-800/40 border border-gray-700" : "bg-
gray-50 border border-gray-200"}`}
466.      >
467.          <h2 className={`text-xl font-bold font-roboto mb-3 ${isDark
? "text-white" : "text-gray-900"}`}>
468.              Product Description
469.          </h2>
470.          <p className={`font-roboto text-sm leading-relaxed ${isDark
? "text-gray-300" : "text-gray-700"}`}>

```

```

471.         {isExpened
472.         ? productData?.description
473.         : productData?.description?.toString()?.substring(0,
300)}
474.         {productData?.description?.length > 300 && (
475.         <span
476.             className="text-indigo-500 font-semibold cursor-
pointer ml-1 hover:text-indigo-400 transition-colors"
477.             onClick={() => setIsExpened(!isExpened)}
478.         >
479.             {isExpened ? " Read Less" : " Read More"}
480.         </span>
481.         )}
482.     </p>
483. </div>
484.
485.     {/* — Reviews Section
_____ */}
486.     <div className="review-section px-4 py-6 max-w-7xl mx-auto">
487.         <div className="flex gap-3 justify-between mb-5 items-
center">
488.             <div>
489.                 <h2 className={`text-2xl font-bold font-roboto ${isDark
? "text-white" : "text-gray-900"}`}>
490.                     Customer Reviews
491.                 </h2>
492.                 {reviews.length > 0 && (
493.                     <p className={`text-sm font-roboto mt-1 ${isDark ?
"text-gray-400" : "text-gray-500"}`}>
494.                         {reviews.length} review{reviews.length !== 1 ? "s" :
""} · {avgRating.toFixed(1)} avg rating
495.                     </p>
496.                 )}
497.             </div>
498.             {userDetail && authToken && (
499.                 <button
500.                     className="rounded-xl bg-indigo-600 hover:bg-indigo-
700 px-4 py-2 text-white font-roboto text-sm font-semibold transition-all
duration-200 flex items-center gap-2 hover:scale-[1.02]"
501.                     onClick={handleClick}
502.                 >
503.                     <MdVerified />
504.                     Write a Review
505.                 </button>
506.             )}
507.         </div>
508.
509.         {reviews.length > 0 ? (
510.             <div className="flex gap-4 flex-wrap mobile:flex-col
small-device:flex-row">
511.                 {reviews.map((review, index) => (
512.                     <ReviewCard review={review} key={index} />
513.                 ))}
514.             </div>
515.         ) : (
516.             <div className={`
517.                 flex flex-col items-center justify-center py-12 rounded-
2xl border-2 border-dashed
518.                 ${isDark ? "border-gray-700 text-gray-500" : "border-
gray-300 text-gray-400"}
519.             `}>

```

```

520.         <svg className="w-12 h-12 mb-3 opacity-40" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
521.             <path strokeLinecap="round" strokeLinejoin="round"
strokeWidth={1.5} d="M8 10h.01M12 10h.01M16 10h.01M9 16H5a2 2 0 01-2-2V6a2
2 0 012-2h14a2 2 0 012 2v8a2 2 0 01-2 2h-5l-5 5v-5z" />
522.         </svg>
523.         <p className="text-base font-semibold font-roboto">No
reviews yet</p>
524.         <p className="text-sm font-roboto mt-1">Be the first to
share your experience!</p>
525.     </div>
526.     )}
527. </div>
528.
529.     {/* — Related Products
_____ */}

530.     {relatedProduct.length > 0 && (
531.         <div className={`related-product-section px-4 py-6 mt-2
border-t ${isDark ? "border-gray-800" : "border-gray-100"}`}>
532.             <div className="max-w-7xl mx-auto">
533.                 <div className="flex items-center gap-3 mb-5">
534.                     <div className="w-1 h-7 bg-indigo-500 rounded-full" />
535.                     <h2 className={`font-bold font-roboto text-2xl
mobile:text-xl ${isDark ? "text-white" : "text-gray-900"}`}>
536.                         You May Also Like
537.                     </h2>
538.                 </div>
539.                 <div className="w-full grid grid-cols-5 mobile:grid-
cols-2 small-device:grid-cols-3 tablet:grid-cols-4 laptop:grid-cols-5 gap-3
items-stretch">
540.                     {relatedProduct.map((product) => (
541.                         <div key={product?.id || product?._id}>
542.                             <ProductCard
543.                                 product={product}
544.                                 authToken={authToken}
545.                                 userDetail={userDetail}
546.                             />
547.                         </div>
548.                     ))}
549.                 </div>
550.             </div>
551.         </div>
552.     )}
553.
554.     {/* — Review Popup
_____ */}

555.     {isReviewClicked && (
556.         <ReviewForm
557.             onClose={handleClick}
558.             productId={productData?._id}
559.             setIsRefreshClicked={setIsRefreshClicked}
560.             userDetail={userDetail}
561.             authToken={authToken}
562.         />
563.     )}
564.
565. </div>
566. )
567. );
568. };
569.

```

```
570. export default ProductPage;
571.
```

File Name: frontend\src\Component\Product\index.js

```
1. export { default as ProductCard } from "../ProductCard";
2. export { default as ProductImage } from "../ProductImage";
3. export { default as ProductsContainer } from "../ProductsContainer";
4.
```

File Name: frontend\src\Component\Product\ProductCard.jsx

```
1. import Button from "../../LIBS/Button";
2. import React from "react";
3. import { useNavigate } from "react-router-dom";
4. import { formatNumber } from "../../utility/constant";
5. import {
6.   AddToCartButton,
7.   AddRemoveProductFromWishListButton,
8. } from "../../utility";
9. import { useTheme } from "../../Context/themeContext";
10. import { useAuth } from "../../Context/authContext";
11. import { useUser } from "../../Context/userContext";
12.
13. const ProductCard = ({ product }) => {
14.   const {
15.     image,
16.     name,
17.     price,
18.     mrpPrice,
19.     category,
20.     id,
21.     rating,
22.     stock,
23.     sellingPrice,
24.     ratingNumber,
25.     isAddedToWislist,
26.   } = product;
27.   const { theme } = useTheme();
28.   const { authToken } = useAuth();
29.   const { userDetails } = useUser();
30.   const navigate = useNavigate();
31.   const isDark = theme === "dark";
32.
33.   const handleCardClick = () => {
34.     navigate(`/product/${id}`);
35.   };
36.
37.   const selling = sellingPrice || price;
38.   const discountPct =
39.     mrpPrice && selling && mrpPrice > selling
40.       ? Math.round(((mrpPrice - selling) / mrpPrice) * 100)
41.       : null;
42.
43.   return (
44.     <div
45.       onClick={handleCardClick}
```

```

46.         className={`
47.             group relative flex flex-col rounded-2xl overflow-hidden
cursor-pointer
48.             border transition-all duration-300 ease-out
49.             hover:-translate-y-1 hover:shadow-xl
50.             ${isDark
51.                 ? "bg-gray-800 border-gray-700 hover:border-indigo-500/50
hover:shadow-indigo-900/30"
52.                 : "bg-white border-gray-200 hover:border-indigo-200
hover:shadow-indigo-100/80"}
53.         }
54.     `}
55. >
56.     { /* — Image Section —————
*/}
57.     <div className="relative overflow-hidden aspect-square bg-gray-
100">
58.         <img
59.             src={image || "#"}
60.             alt={name}
61.             className="w-full h-full object-cover object-top transition-
transform duration-500 ease-out group-hover:scale-110"
62.             />
63.
64.         { /* Out of Stock Overlay */}
65.         { !stock && (
66.             <div className="absolute inset-0 bg-black/50 flex items-
center justify-center">
67.                 <span className="text-white text-xs font-bold font-roboto
px-3 py-1.5 bg-red-500 rounded-full tracking-wide">
68.                     OUT OF STOCK
69.                 </span>
70.             </div>
71.         ) }
72.
73.         { /* Discount Badge */}
74.         { discountPct && stock ? (
75.             <span className="absolute top-2.5 left-2.5 bg-green-500 text-
white text-[11px] font-bold font-roboto px-2 py-0.5 rounded-full shadow-
sm">
76.                 {discountPct}% OFF
77.             </span>
78.         ) : null }
79.
80.         { /* Wishlist Button */}
81.         <div className="absolute top-2.5 right-2.5">
82.             <AddRemoveProductFromWishListButton
83.                 authToken={authToken}
84.                 productId={id}
85.                 userDetails={userDetail}
86.                 isAddedToWishlist={isAddedToWishlist}
87.             />
88.         </div>
89.
90.         { /* Category Badge — slides up on hover */}
91.         { category && (
92.             <div className={`
93.                 absolute bottom-0 left-0 right-0
94.                 px-3 py-1.5
95.                 translate-y-full group-hover:translate-y-0
96.                 transition-transform duration-300

```

```

97.         ${isDark ? "bg-gray-900/85" : "bg-white/85"}
98.         backdrop-blur-sm
99.     `}>
100.         <span className={`text-[10px] font-semibold font-roboto
uppercase tracking-widest
101.             ${isDark ? "text-indigo-400" : "text-indigo-500"}}`>
102.             {category?.name}
103.         </span>
104.     </div>
105. })
106. </div>
107.
108. {/* — Info Section _____
*/}
109. <div className="flex flex-col flex-1 p-3 gap-2">
110.
111.     {/* Product Name – fixed 2-line height so all cards align */}
112.     <p className={`
113.         font-semibold font-roboto text-sm leading-snug
114.         line-clamp-2 min-h-[2.5rem]
115.         transition-colors duration-200
116.         ${isDark
117.             ? "text-gray-100 group-hover:text-indigo-300"
118.             : "text-gray-900 group-hover:text-indigo-600"
119.         }
120.     `}>
121.         {name}
122.     </p>
123.
124.     {/* Star Rating – always occupies fixed height (h-5) so cards
stay aligned
125.         whether or not a product has reviews */}
126.     <div className="h-5 flex items-center gap-1.5">
127.         {!!rating && (
128.             <>
129.                 <div className="flex gap-0.5">
130.                     {[1, 2, 3, 4, 5].map((index) => (
131.                         <svg
132.                             key={index}
133.                             className={`w-3.5 h-3.5 ${
134.                                 index <= Math.round(rating)
135.                                 ? "text-yellow-400"
136.                                 : isDark
137.                                 ? "text-gray-600"
138.                                 : "text-gray-300"
139.                             }}`
140.                             fill="currentColor"
141.                             viewBox="0 0 24 24"
142.                         >
143.                             <path d="M12 2l3.09 6.26L22 9.27l-5 4.87L18.18 22
12 18.77 5.82 22 7 14.14l-5-4.87 6.91-1.01z" />
144.                         </svg>
145.                     )}}
146.                 </div>
147.                 {ratingNumber && (
148.                     <span
149.                         className={`text-[11px] font-roboto font-medium ${
150.                             isDark ? "text-gray-400" : "text-gray-500"
151.                         }}`
152.                     >
153.                         ({ratingNumber})

```



```

154.         </span>
155.     )}
156. </>
157. )}
158. </div>
159.
160. {/* Pricing */}
161. <div className="flex items-baseline gap-1.5 flex-wrap">
162.     <span
163.         className={`text-base font-bold font-roboto ${
164.             isDark ? "text-white" : "text-gray-900"
165.         }`}
166.     >
167.         {formatNumber(selling)}
168.     </span>
169.     {mrpPrice && mrpPrice > selling && (
170.         <span
171.             className={`text-xs font-roboto line-through ${
172.                 isDark ? "text-gray-500" : "text-gray-400"
173.             }`}
174.         >
175.             {formatNumber(mrpPrice)}
176.         </span>
177.     )}
178. </div>
179.
180. {/* Add to Cart / Out of Stock Button */}
181. <div className="mt-auto pt-1">
182.     {stock ? (
183.         <AddToCartButton
184.             authToken={authToken}
185.             userDetails={userDetail}
186.             productId={id}
187.             className={`
188.                 w-full py-2 rounded-xl font-roboto font-semibold text-
189.                 xs
190.                 transition-all duration-200
191.                 bg-indigo-600 hover:bg-indigo-700 text-white
192.                 flex items-center justify-center gap-1.5
193.                 hover:shadow-md active:scale-95
194.             `}
195.         ) : (
196.             <Button
197.                 btnText="Out of Stock"
198.                 className="w-full py-2 rounded-xl font-roboto font-
199.                 semibold text-xs text-red-500 bg-red-50 border border-red-200 flex items-
200.                 center justify-center cursor-not-allowed"
201.                 disabled={true}
202.                 onClick={(e) => {
203.                     e.preventDefault();
204.                     e.stopPropagation();
205.                 }}
206.             </div>
207.         </div>
208.     </div>
209. );
210. };
211.

```

```
212. export default ProductCard;
213.
```

File Name: frontend/src/Component/Product/ProductImage.jsx

```
1. import React, { useState } from "react";
2. import { useTheme } from "../../Context/themeContext";
3.
4. const ProductImage = ({ productData }) => {
5.   const [imageIndex, setImageIndex] = useState(0);
6.   const { theme } = useTheme();
7.
8.   const isDark = theme === "dark";
9.
10.  return (
11.    <div className="img-container flex gap-3 m-4 p-4 w-1/3 mobile:w-
full tablet:w-1/2 mobile:m-0 mobile:flex-col-reverse mobile:items-center
small-device:flex-row tablet:flex-row">
12.
13.      {/* Thumbnail Strip */}
14.      <div className="flex flex-col gap-2 overflow-y-auto items-start
justify-start min-h-[400px] min-w-[72px] max-h-[480px] mobile:flex-row
mobile:min-h-20 mobile:overflow-x-auto mobile:w-full mobile:max-h-24 small-
device:w-max small-device:min-h-[400px] small-device:flex-col tablet:flex-
col">
15.        {productData?.image?.map((img, index) => (
16.          <div
17.            key={index}
18.            onClick={() => setImageIndex(index)}
19.            className={`
20.              relative cursor-pointer rounded-lg overflow-hidden
21.              min-h-[60px] min-w-[60px] h-[64px] w-[64px]
22.              mobile:max-w-[60px] mobile:max-h-[60px]
23.              border-2 transition-all duration-200
24.              ${imageIndex === index
25.                ? "border-indigo-500 ring-2 ring-indigo-300 shadow-md"
26.                : isDark
27.                  ? "border-gray-600 hover:border-indigo-400"
28.                  : "border-gray-300 hover:border-indigo-300"}
29.            `}
30.          >
31.            <img
32.              src={img}
33.              className="object-cover w-full h-full transition-
transform duration-200 hover:scale-110"
34.              alt={` ${productData?.name} view ${index + 1}`}
35.            />
36.            {imageIndex === index && (
37.              <div className="absolute inset-0 bg-indigo-500/10
pointer-events-none" />
38.            )}
39.          </div>
40.        </div>
41.      )}
42.    </div>
43.
44.    {/* Main Image with Zoom */}
45.    <div className={`
46.      relative overflow-hidden rounded-2xl flex-1
```

```

47.         border-2
48.         ${isDark ? "border-gray-700" : "border-gray-200"}
49.         shadow-lg max-h-[480px]
50.         group
51.     `}>
52.         <img
53.             src={productData?.image?.[imageIndex]}
54.             className="w-full h-full object-cover object-top aspect-
square max-h-[480px] transition-transform duration-500 ease-out group-
hover:scale-110"
55.             alt={productData?.name}
56.         />
57.         {/* Image count badge */}
58.         {productData?.image?.length > 1 && (
59.             <div className={`
60.                 absolute bottom-3 right-3 text-xs font-semibold px-2 py-1
rounded-full
61.                 ${isDark ? "bg-gray-800/80 text-gray-200" : "bg-white/80
text-gray-700"}
62.                 backdrop-blur-sm shadow
63.             `}>
64.                 {imageIndex + 1} / {productData?.image?.length}
65.             </div>
66.         )}
67.     </div>
68. </div>
69. );
70. };
71.
72. export default ProductImage;
73.

```

File Name: frontend/src/Component/Product/ProductsContainer.jsx

```

1. import axios from "axios";
2. import { useEffect, useState } from "react";
3. import { useLocation } from "react-router-dom";
4. import Productcard from "../ProductCard";
5. import { Button, Loader, ServerError } from "../../LIBS";
6. import { useTheme } from "../../Context/themeContext";
7. import { useAuth } from "../../Context/authContext";
8. import noResultImage from "../../assets/images/noResult.png";
9. import { useUser } from "../../Context/userContext";
10. // import { SERVER_URL } from "../../config";
11. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
12.
13. console.log(SERVER_URL);
14. const ProductsContainer = () => {
15.     const location = useLocation();
16.     const { theme } = useTheme();
17.     const { authToken } = useAuth();
18.     const { userDetail } = useUser();
19.     const [allProduct, SetAllProduct] = useState();
20.     const [allCategories, SetAllCategories] = useState();
21.     const [filteredProduct, SetFilteredProduct] = useState();
22.     const [filteredCategory, SetFilteredCategory] = useState([]);
23.     const [currentPage, setCurrentPage] = useState(1);
24.     const [totalPages, setTotalPages] = useState(1);
25.     const [productCount, setProductCount] = useState(0);

```

```

26.   const [productRange, setProductRange] = useState({});
27.   const [isDataFetched, setIsDataFetched] = useState(false);
28.   const [error, setError] = useState({});
29.   const visiblePages = Array.from(
30.     { length: totalPages },
31.     (_, index) => index + 1,
32.   )
33.     .filter((page) => page >= Math.max(1, currentPage - 1))
34.     .filter((page) => page <= Math.min(totalPages, currentPage + 1));
35.   const searchQuery = new URLSearchParams(location?.search)
36.     ?.get("q")
37.     ?.toString()
38.     ?.trim();
39.   const searchCategory = new URLSearchParams(location?.search)
40.     ?.get("category")
41.     ?.toString()
42.     ?.trim();
43.   const handleSubmit = (e) => {
44.     e.preventDefault();
45.     // const formElement = Array.from(e.target.elements).slice(0, -1);
46.     // console.log(formElement);
47.     // formElement.map((element, index) => {
48.     //   console.log(element.checked);
49.     //   console.log(element.value);
50.     //   if (element.checked) {
51.     //     SetFilteredProduct(
52.     //       allProduct?.filter((item) =>
item.category.includes(element.value))
53.     //     );
54.     //   }
55.     // });
56.     let filter = [];
57.     filteredCategory.forEach((category) => {
58.       const products = allProduct?.filter((item) =>
59.         item.category?.name?.includes(category),
60.       );
61.       filter = [...filter, ...products];
62.     });
63.     SetFilteredProduct(filter);
64.   };
65.   const getAllProduct = () => {
66.     axios
67.       .get(`${SERVER_URL}/api/product/get-all-products`, {
68.         params: {
69.           activeUserId: userDetails?.id?.toString(),
70.           searchQuery,
71.           searchCategory,
72.           page: currentPage,
73.         },
74.       })
75.       .then((response) => {
76.         const { data, status } = response;
77.         if (status === 200) {
78.           SetAllProduct(data?.products);
79.           SetFilteredProduct(data?.products);
80.           setTotalPages(data?.totalPages || 1);
81.           setProductCount(data?.totalProducts || 0);
82.           setProductRange({
83.             startProductIndex: data?.startProductIndex || 0,
84.             endProductIndex: data?.endProductIndex || 0,
85.           });

```

```

86.         }
87.     })
88.     .catch((error) => {
89.         const { message = "" } = error ||
error?.response?.data?.message;
90.
91.         setError({ errorMessage: message });
92.     })
93.     .finally(() => setIsDataFetched(true));
94. };
95.
96. // const getAllCategory = () => {
97. //   try {
98. //     axios
99. //       .get(API_URL + "products/categories")
100. //       .then((response) => SetAllCategories(response.data));
101. //   } catch (error) {
102. //     console.log(error);
103. //   }
104. // };
105.
106. const handleChange = (e) => {
107.   if (e.target.checked) {
108.     SetFilteredCategory((previous) => [...previous,
e.target.value]);
109.   }
110.   // e.target.checked
111.   //   ? SetFilteredProduct(
112.   //     allProduct?.filter((item) =>
item.category?.name?.includes(e.target.value))
113.   //   )
114. };
115.
116. useEffect(() => {
117.   // axios
118.   //   .get(API_URL + "products")
119.   //   .then((response) => {
120.   //     SetAllProduct(response.data);
121.   //     SetFilteredProduct(response.data);
122.   //     setIsDataFetch(true);
123.   //   })
124.   //   .catch((error) => console.log(error));
125.   getAllProduct();
126.   // getAllCategory();
127. }, [userDetail?.id, searchQuery, searchCategory, currentPage]);
128.
129. useEffect(() => {
130.   setCurrentPage(1);
131. }, [searchQuery, searchCategory]);
132.
133. useEffect(() => {
134.   window.scrollTo(0, 0); // Scroll to the top on component mount
135. }, [currentPage]);
136. if (error.errorMessage) {
137.   return <ServerError />;
138. }
139.
140. if (isDataFetched && !filteredProduct?.length > 0 &&
!error.errorMessage)
141.   return (
142.     <div

```

```

143.         className={` w-full h-screen flex justify-center items-center
144.         ${
145.             theme === "dark"
146.             ? " bg-gray-900 text-white"
147.             : "bg-white text-gray-900"
148.         } transition-all duration-300`}
149.     >
150.         <div className=" font-mono flex flex-col gap-3 ">
151.             <img
152.                 src={noResultImage}
153.                 alt="there is not products"
154.                 className="max-w-[350px]"
155.             />
156.             <div className="h-min flex flex-col gap-2 justify-center
157.             items-center">
158.                 <span className="w-max max-w-[250px] text-lg font-
159.                 semibold">
160.                     No Result Found!
161.                 </span>
162.                 <span className="max-w-[250px] text-center">
163.                     We can't find any items matching your Search
164.                 </span>
165.             </div>
166.         </div>
167.     </div>
168. );
169. if (!isDataFetched && !error.errorMessage) return <Loader />;
170. return (
171.     filteredProduct && (
172.         <div
173.             className={`product-container w-full h-full items-center
174.             justify-center pt-8 flex flex-col gap-2 p-4 ${
175.                 theme === "dark"
176.                 ? " bg-gray-900 text-white"
177.                 : "bg-white text-gray-900"
178.             } transition-all duration-300`}
179.         >
180.             {/* <div className="left-section w-2/12 h-full border border-
181.             black bg-gray min-w-[200px] mobile:hidden laptop:block">
182.                 <h2 className="text-center">Filter</h2>
183.                 <div className="p-2 ">
184.                     <form action="" onSubmit={(e) => handleSubmit(e)}>
185.                         {allCategories?.map((category, index) => {
186.                             return (
187.                                 <div className="flex gap-1 items-center"
188.                                 key={index}>
189.                                     <input
190.                                         type="checkbox"
191.                                         name="filter"
192.                                         id={category}
193.                                         onClick={(e) => {
194.                                             handleChange(e);
195.                                         }}
196.                                         value={category}
197.                                     />
198.                                     <label htmlFor={category} className="cursor-
199.                                     pointer">
200.                                         {category}
201.                                     </label>
202.                                 </div>

```

```

197.         );
198.     }}}
199.     <input
200.         type="submit"
201.         className="p-2 w-full my-2 cursor-pointer flex
justify-center items-center bg-blue-900 rounded-md text-white"
202.         value="Apply"
203.     />
204. </form>
205. </div>
206. </div> */}
207.
208.     <div className="right-section grid grid-cols-1 place-items-
center mobile:grid-cols-2 small-device:grid-cols-3 tablet:grid-cols-4
laptop:grid-cols-4 desktop:grid-cols-6 large-device:grid-cols-6 w-10/12
justify-around flex-wrap gap-3 mobile:w-full laptop:w-full">
209.         {filteredProduct &&
210.         filteredProduct.map((product, index) => {
211.             return (
212.                 <div
213.                     className="w-full h-full"
214.                     key={product?.id || product?._id}
215.                 >
216.                     <Productcard
217.                         product={product}
218.                         userDetails={userDetail}
219.                         authToken={authToken}
220.                     />
221.                 </div>
222.             );
223.         }}}
224.     </div>
225.
226.     <div
227.         className={`sticky bottom-0 z-10 mt-4 w-full flex items-
center justify-between gap-4 border-t px-3 py-3 mobile:flex-col small-
device:flex-row ${
228.             theme === "dark"
229.             ? "border-gray-700 bg-gray-900/95"
230.             : "border-gray-200 bg-gray-100/95"
231.         } backdrop-blur`}
232.     >
233.         <p className="text-gray-500">
234.             Showing {productRange?.startProductIndex} to{" "}
235.             {productRange?.endProductIndex} of {productCount} results
236.         </p>
237.         <div className="flex flex-wrap justify-center gap-2">
238.             <Button
239.                 className={`px-3 py-1 rounded-md disabled:opacity-50 ${
240.                     theme === "dark"
241.                     ? "bg-gray-700 text-white"
242.                     : "bg-gray-300 text-gray-900"
243.                 }`}
244.                 btnText="Previous"
245.                 onClick={() =>
246.                     setCurrentPage((prev) => (prev > 1 ? prev - 1 : prev))
247.                 }
248.                 disabled={currentPage === 1}
249.             />
250.
251.             {currentPage > 2 && (

```

```

252.         <>
253.         <Button
254.             className={`px-3 py-1 rounded-md ${
255.                 theme === "dark"
256.                 ? "bg-gray-700 text-white"
257.                 : "bg-gray-300 text-gray-900"
258.             }`}
259.             btnText={1}
260.             onClick={() => setCurrentPage(1)}
261.         />
262.         {currentPage > 3 && <span className="px-1 py-
1">...</span>}
263.     </>
264. )}
265.
266. {visiblePages.map((page) => (
267.     <Button
268.         className={`px-3 py-1 rounded-md ${
269.             page === currentPage
270.             ? "bg-purple-600 text-white"
271.             : theme === "dark"
272.             ? "bg-gray-700 text-white"
273.             : "bg-gray-300 text-gray-900"
274.         }`}
275.         key={page}
276.         btnText={page}
277.         onClick={() => setCurrentPage(page)}
278.     />
279. )})
280.
281. {currentPage < totalPages - 1 && (
282.     <>
283.         {currentPage < totalPages - 2 && (
284.             <span className="px-1 py-1">...</span>
285.         )}
286.         <Button
287.             className={`px-3 py-1 rounded-md ${
288.                 theme === "dark"
289.                 ? "bg-gray-700 text-white"
290.                 : "bg-gray-300 text-gray-900"
291.             }`}
292.             btnText={totalPages}
293.             onClick={() => setCurrentPage(totalPages)}
294.         />
295.     </>
296. )}
297.
298. <Button
299.     className={`px-3 py-1 rounded-md disabled:opacity-50 ${
300.         theme === "dark"
301.         ? "bg-gray-700 text-white"
302.         : "bg-gray-300 text-gray-900"
303.     }`}
304.     btnText="Next"
305.     onClick={() =>
306.         setCurrentPage((prev) => (prev < totalPages ? prev + 1
: prev))
307.     }
308.     disabled={currentPage === totalPages}
309. />
310. </div>

```



```

311.         </div>
312.     </div>
313. )
314. );
315. };
316.
317. export default ProductsContainer;
318.

```

File Name: frontend\src\Component\Seller\AddProductPanel.jsx

```

1. import { useState, useEffect } from "react";
2. import axios from "axios";
3. import { FiImage, FiUploadCloud } from "react-icons/fi";
4. import { BiLoaderAlt } from "react-icons/bi";
5. import { useDropzone } from "react-dropzone";
6. import { useNavigate, useParams } from "react-router-dom";
7. import { useTheme } from "../../Context/themeContext";
8. import { useAuth } from "../../Context/authContext";
9. import { Button, Input, Select, Textarea } from "../../LIBS";
10. import ImagePreview from "../UploadProduct/ImagePreview";
11. import {
12.     checkValidation,
13.     initialProductDetails,
14.     productBrands,
15.     swalWithCustomConfiguration,
16. } from "../../utility/constant";
17. import { useCategory } from "../../Context/categoryContext";
18.
19. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
20. const CLOUD_NAME = import.meta.env.VITE_CLOUDINARY_CLOUD_NAME;
21.
22. export const AddProductPanel = () => {
23.     const [productForm, setProductForm] =
24.     useState(initialProductDetails);
25.     const [error, setError] = useState({});
26.     const [images, setImages] = useState([]);
27.     const [productUploading, setProductUploading] = useState(false);
28.     const [isFetchingInitialData, setIsFetchingInitialData] =
29.     useState(false);
30.     const { theme } = useTheme();
31.     const { authToken } = useAuth();
32.     const { categories } = useCategory();
33.
34.     const dynamicCategories = categories?.filter(c => c.parentCategory
35.     === null).map(mainCat => {
36.         return {
37.             id: mainCat._id,
38.             name: "category",
39.             value: mainCat._id, // Use ID for value
40.             displayName: mainCat.name, // Use name for UI
41.             child: categories.filter(c => c.parentCategory === mainCat._id
42.             || c.parentCategory?._id === mainCat._id).map(c => ({ id: c._id, name:
43.             c.name })))
44.         };
45.     }) || [];
46.
47.     const navigate = useNavigate();
48.     const { id } = useParams();

```

```

44.   const isEditMode = Boolean(id);
45.
46.   useEffect(() => {
47.     if (isEditMode && authToken) {
48.       fetchProductDetails();
49.     }
50.   }, [id, authToken]);
51.
52.   const fetchProductDetails = async () => {
53.     try {
54.       setIsFetchingInitialData(true);
55.       const response = await
axios.get(`${SERVER_URL}/api/product/${id}`);
56.       const productData = response?.data?.product;
57.       if (productData) {
58.         setProductForm({
59.           name: productData.name || "",
60.           mrpPrice: productData.mrpPrice?.toString() || "",
61.           sellingPrice: productData.sellingPrice?.toString() || "",
62.           description: productData.description || "",
63.           category: productData.category?._id || productData.category
|| "",
64.           stock: productData.stock?.toString() || "",
65.           brand: productData.brand || "",
66.           subCategory: productData.subCategory?._id ||
productData.subCategory || "",
67.         });
68.         setImages(productData.image || []);
69.       }
70.     } catch (error) {
71.       console.log(error);
72.       swalWithCustomConfiguration.fire("Error", "Could not fetch
product details", "error");
73.     } finally {
74.       setIsFetchingInitialData(false);
75.     }
76.   };
77.
78.   const onDrop = (acceptedFiles) => {
79.     setImages((prev) => [...prev, ...acceptedFiles]);
80.     setError((prev) => ({ ...prev, image: "" }));
81.   };
82.
83.   const { getRootProps, getInputProps, isDragActive, open } =
useDropzone({
84.     onDrop,
85.     accept: { "image/*": [] },
86.     multiple: true,
87.     noClick: true,
88.   });
89.
90.   const handleChange = (e) => {
91.     const { name, value } = e.target;
92.     setProductForm((prev) => ({
93.       ...prev,
94.       ...(name === "category" ? { subCategory: "" } : {}),
95.       [name]: value,
96.     }));
97.     setError((prev) => ({ ...prev, [name]: "" }));
98.   };
99.

```

```

100. // The client-side Cloudinary logic has been removed. Uploads are
strictly handled by the Backend Node Server.
101.
102. const validateForm = () => {
103.     const requiredFields = [
104.         "name",
105.         "category",
106.         "description",
107.         "stock",
108.         "brand",
109.         "mrpPrice",
110.         "sellingPrice",
111.     ];
112.
113.     const formErrors = checkValidation(productForm, requiredFields) ||
{};
114.
115.     if (!productForm?.subCategory && productForm?.category !==
"others") {
116.         formErrors.subCategory = "Sub Category is required!";
117.     }
118.
119.     if (images.length < 1) {
120.         formErrors.image = "Product Image is required!";
121.     }
122.
123.     if (
124.         Number(productForm?.mrpPrice) > 0 &&
125.         Number(productForm?.sellingPrice) >
Number(productForm?.mrpPrice)
126.     ) {
127.         formErrors.mrpPrice = "MRP Price should be greater than Selling
Price";
128.     }
129.
130.     return formErrors;
131. };
132.
133. const handleSubmit = async (e) => {
134.     e?.preventDefault?.();
135.
136.     const formErrors = validateForm();
137.     if (Object.keys(formErrors).length > 0) {
138.         setError(formErrors);
139.         return;
140.     }
141.
142.     if (!authToken) {
143.         swalWithCustomConfiguration.fire(
144.             "Login required",
145.             "Please login again before uploading a product.",
146.             "warning"
147.         );
148.         return;
149.     }
150.
151.     try {
152.         setProductUploading(true);
153.         setError({});
154.
155.         // Separate Kept Strings and New Files

```

```

156.     const existingImages = images.filter((img) => typeof img ===
"string");
157.     const newImageFiles = images.filter((img) => typeof img !==
"string");
158.
159.     const formData = new FormData();
160.
161.     Object.keys(productForm).forEach((key) => {
162.         formData.append(key, productForm[key]);
163.     });
164.
165.     // Append Kept Images natively
166.     existingImages.forEach((url) => {
167.         formData.append("existingImages", url);
168.     });
169.
170.     // Append New file uploads
171.     newImageFiles.forEach((file) => {
172.         formData.append("image", file);
173.     });
174.
175.     const response = await axios({
176.         method: isEditMode ? "PUT" : "POST",
177.         url: isEditMode ? `${SERVER_URL}/api/seller/product/${id}` :
`${SERVER_URL}/api/product/add-product`,
178.         data: formData,
179.         headers: {
180.             "Content-Type": "multipart/form-data",
181.             Authorization: `Bearer ${authToken}`,
182.         },
183.     });
184.
185.     if (response.status === 201 || response.status === 200) {
186.         await swalWithCustomConfiguration.fire(
187.             isEditMode ? "Product successfully updated" : "Product
successfully uploaded",
188.             isEditMode ? "Your product changes have been saved." : "Your
product has been added.",
189.             "success"
190.         );
191.         setProductForm(initialProductDetails);
192.         setImages([]);
193.         navigate("/seller/products");
194.     }
195. } catch (submitError) {
196.     const status = submitError?.response?.status;
197.     const message =
198.         submitError?.response?.data?.message ||
199.         "Error in uploading product, Please try again later.";
200.
201.     swalWithCustomConfiguration.fire(
202.         status === 401 ? "Unauthorized" : "Error uploading!",
203.         message,
204.         "error"
205.     );
206. } finally {
207.     setProductUploading(false);
208. }
209. };
210.
211. return (

```

```

212.     <div
213.         className={`w-full rounded-[24px] shadow-sm border p-6 md:p-8
214.             transition-all duration-300 font-roboto ${
215.                 theme === "dark" ? "bg-gray-900 border-gray-800 text-white" :
216.                 "bg-white border-gray-200 text-gray-900"
217.             }`}
218.         >
219.             {isFetchingInitialData ? (
220.                 <div className="flex flex-col items-center justify-center py-
221.                 32 opacity-80">
222.                     <BiLoaderAlt className="animate-spin text-5xl mb-4 text-
223.                     indigo-500" />
224.                     <p className="font-bold tracking-widest uppercase text-sm
225.                     text-gray-500">Loading Product Data...</p>
226.                 </div>
227.             ) : (
228.                 <>
229.                     {/* Header keeping the button at the top as originally
230.                     structured */}
231.                     <div className="flex flex-col gap-4 mb-8 small-device:flex-
232.                     row small-device:justify-between small-device:items-center">
233.                         <div>
234.                             <h1 className="text-2xl font-extrabold tracking-tight">
235.                                 {isEditMode ? "Edit Product" : "Add New Product"}
236.                             </h1>
237.                             <p className={`text-sm mt-1 ${theme === 'dark' ? 'text-gray-
238.                             400' : 'text-gray-500'}`}>
239.                                 {isEditMode ? "Modify your selected product listing" :
240.                                 "Create and publish a new product listing"}
241.                             </p>
242.                         </div>
243.                         <div className="w-full small-device:w-auto">
244.                             <Button
245.                                 className="w-full small-device:w-auto min-w-[180px] px-6
246.                                 py-3 bg-indigo-600 font-bold text-white rounded-xl shadow-md hover:bg-
247.                                 indigo-700 hover:-translate-y-0.5 transition-all outline-none
248.                                 disabled:opacity-60 disabled:cursor-not-allowed"
249.                                 btnText={productUploading ? (isEditMode ? "Updating..." :
250.                                 "Publishing...") : (isEditMode ? "Update Product" : "Publish Product")}
251.                                 onClick={handleSubmit}
252.                                 loading={productUploading}
253.                                 disabled={productUploading}
254.                             />
255.                         </div>
256.                     </div>
257.                     {/* Form structure completely reverted to original */}
258.                     <form className="space-y-6" onSubmit={handleSubmit}>
259.                         {/* Row 1: Name and Brand */}
260.                         <div className="grid grid-cols-1 small-device:grid-cols-2 gap-
261.                         6">
262.                             <div className="flex flex-col gap-2">
263.                                 <label htmlFor="name" className={`font-semibold text-sm
264.                                 ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}`}>
265.                                     Product Name <span className="text-red-500">*</span>
266.                                 </label>
267.                                 <Input
268.                                     type="text"
269.                                     id="name"
270.                                     name="name"

```

```

258.         value={productForm?.name}
259.         onChange={handleChange}
260.         placeholder="Product Name"
261.         className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none ${
262.             error.name ? "border-red-500 focus:ring-red-500/20" :
263.             theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
264.         }}
265.     />
266.     {error.name && <p className="text-red-500 text-xs font-
bold">{error.name}</p>}
267. </div>
268.
269.     <div className="flex flex-col gap-2">
270.         <label htmlFor="brand" className={`font-semibold text-sm
${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
271.             Brand <span className="text-red-500">*</span>
272.         </label>
273.         <Select
274.             name="brand"
275.             id="brand"
276.             className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none cursor-pointer ${
277.                 error.brand ? "border-red-500 focus:ring-red-500/20" :
278.                 theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
279.             }}
280.             onChange={handleChange}
281.             value={productForm?.brand}
282.             displayName="Select Brand"
283.             itemArray={productBrands}
284.         />
285.         {error.brand && <p className="text-red-500 text-xs font-
bold">{error.brand}</p>}
286.     </div>
287. </div>
288.
289.     {/* Row 2: Media */}
290.     <div>
291.         <label className={`font-semibold text-sm block mb-2 ${theme
=== 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
292.             Product Media <span className="text-red-500">*</span>
293.         </label>
294.         <div
295.             {...getRootProps()}
296.             className={`w-full p-8 border-2 border-dashed rounded-2xl
flex flex-col items-center justify-center text-center transition-all
duration-300 cursor-pointer ${
297.                 isDragActive
298.                 ? "border-purple-500 bg-purple-50 dark:bg-purple-
900/20"
299.                 : error.image
300.                 ? "border-red-400 bg-red-50 dark:bg-red-900/10"
301.                 : theme === "dark"
302.                 ? "border-gray-600 bg-gray-800/50 hover:bg-gray-800
text-gray-300"
303.                 : "border-gray-300 bg-gray-50 hover:bg-gray-100/50
text-gray-500"

```

```

304.         }` }
305.     >
306.         <input {...getInputProps()} />
307.         <span className="text-4xl text-purple-500 mb-3">
308.             {images.length > 0 ? <FiImage /> : <FiUploadCloud />}
309.         </span>
310.         <p className="font-semibold mb-1">
311.             {isDragActive
312.                 ? "Drop product images here"
313.                 : "Drag and drop your product images here"}
314.         </p>
315.         <button
316.             type="button"
317.             className="text-purple-600 dark:text-purple-400 font-
bold hover:underline cursor-pointer"
318.             onClick={(e) => {
319.                 e.stopPropagation();
320.                 open();
321.             }}
322.         >
323.             Browse Files
324.         </button>
325.     </div>
326.     {error.image && <p className="text-red-500 text-xs font-bold
mt-2">{error.image}</p>}
327.     {images.length > 0 && (
328.         <div className="mt-4 p-4 border rounded-xl dark:border-
gray-800">
329.             <ImagePreview
330.                 image={images}
331.                 onRemove={(indexToRemove) => setImages(prev =>
prev.filter((_, idx) => idx !== indexToRemove))}
332.             />
333.         </div>
334.     )}
335. </div>
336.
337.     {/* Row 3: Prices & Stock */}
338.     <div className="grid grid-cols-1 small-device:grid-cols-3 gap-
6">
339.         <div className="flex flex-col gap-2">
340.             <label htmlFor="mrpPrice" className={`font-semibold text-
sm ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
341.                 MRP Price <span className="text-red-500">*</span>
342.             </label>
343.             <Input
344.                 type="number"
345.                 id="mrpPrice"
346.                 name="mrpPrice"
347.                 value={productForm?.mrpPrice}
348.                 onChange={handleChange}
349.                 placeholder="MRP Price"
350.                 className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none ${
351.                     error.mrpPrice ? "border-red-500 focus:ring-red-
500/20" :
352.                     theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
353.                 }` }
354.             />

```

```

355.         {error.mrpPrice && <p className="text-red-500 text-xs
font-bold">{error.mrpPrice}</p>}
356.     </div>
357.
358.     <div className="flex flex-col gap-2">
359.         <label htmlFor="sellingPrice" className={`font-semibold
text-sm ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
360.             Selling Price <span className="text-red-500">*</span>
361.         </label>
362.         <Input
363.             type="number"
364.             id="sellingPrice"
365.             name="sellingPrice"
366.             value={productForm?.sellingPrice}
367.             onChange={handleChange}
368.             placeholder="Selling Price"
369.             className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none ${
370.                 error.sellingPrice ? "border-red-500 focus:ring-red-
500/20" :
371.                 theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
372.             }}`>
373.         </>
374.         {error.sellingPrice && <p className="text-red-500 text-xs
font-bold">{error.sellingPrice}</p>}
375.     </div>
376.
377.     <div className="flex flex-col gap-2">
378.         <label htmlFor="stock" className={`font-semibold text-sm
${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
379.             Stock Quantity <span className="text-red-500">*</span>
380.         </label>
381.         <Input
382.             type="number"
383.             id="stock"
384.             name="stock"
385.             value={productForm?.stock}
386.             onChange={handleChange}
387.             placeholder="Stock"
388.             className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none ${
389.                 error.stock ? "border-red-500 focus:ring-red-500/20" :
390.                 theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
391.             }}`>
392.         </>
393.         {error.stock && <p className="text-red-500 text-xs font-
bold">{error.stock}</p>}
394.     </div>
395. </div>
396.
397.     {/* Row 4: Description */}
398.     <div className="w-full flex flex-col gap-2">
399.         <label htmlFor="description" className={`font-semibold text-
sm ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
400.             Description <span className="text-red-500">*</span>
401.         </label>
402.         <TextArea

```



```

403.         name="description"
404.         id="description"
405.         value={productForm?.description}
406.         onChange={handleChange}
407.         placeholder="Description"
408.         className={`resize-y w-full min-h-[150px] p-3 border-2
rounded-xl focus:ring-4 transition-all outline-none ${
409.             error.description ? "border-red-500 focus:ring-red-
500/20" :
410.             theme === 'dark' ? "bg-gray-800 text-white border-gray-
700 focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 text-
gray-900 border-gray-200 focus:border-purple-500 focus:ring-purple-500/20"
411.         }}
412.     />
413.     {error.description && <p className="text-red-500 text-xs
font-bold">{error.description}</p>}
414. </div>
415.
416.     {/* Row 5: Categories */}
417.     <div className="grid grid-cols-1 small-device:grid-cols-2 gap-
6">
418.         <div className="flex flex-col gap-2">
419.             <label htmlFor="category" className={`font-semibold text-
sm ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
420.                 Category <span className="text-red-500">*</span>
421.             </label>
422.             <Select
423.                 name="category"
424.                 id="category"
425.                 className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none cursor-pointer ${
426.                     error.category ? "border-red-500 focus:ring-red-
500/20" :
427.                     theme === 'dark' ? "bg-gray-800 border-gray-700
focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50 border-
gray-200 focus:border-purple-500 focus:ring-purple-500/20"
428.                 }}
429.                 onChange={handleChange}
430.                 value={productForm?.category}
431.                 itemArray={dynamicCategories}
432.                 displayName="Select Category"
433.             />
434.             {error.category && <p className="text-red-500 text-xs
font-bold">{error.category}</p>}
435.         </div>
436.
437.         <div className="flex flex-col gap-2">
438.             <label htmlFor="subCategory" className={`font-semibold
text-sm ${theme === 'dark' ? 'text-gray-200' : 'text-gray-800'}}`>
439.                 Sub Category <span className="text-red-500">*</span>
440.             </label>
441.             <select
442.                 name="subCategory"
443.                 className={`p-3 border-2 rounded-xl focus:ring-4
transition-all outline-none disabled:opacity-50 disabled:cursor-not-allowed
444.                     error.subCategory ? "border-red-500 focus:ring-red-
500/20" :
445.                     theme === 'dark' ? "bg-gray-800 text-white border-
gray-700 focus:border-purple-500 focus:ring-purple-500/20" : "bg-gray-50

```

```

text-gray-900 border-gray-200 focus:border-purple-500 focus:ring-purple-
500/20"
446.             `}`
447.             id="subCategory"
448.             disabled={
449.                 productForm?.category === "others" ||
!productForm?.category
450.             }
451.             onChange={handleChange}
452.             value={productForm?.subCategory}
453.         >
454.             <option value="">Select Sub Category</option>
455.             {dynamicCategories?.find(item => item.value ===
productForm?.category)?.child?.map((subItem) => (
456.                 <option key={subItem.id} value={subItem.id}>
457.                     {subItem.name.toCapitalize()}
458.                 </option>
459.             )}}
460.         </select>
461.         {error.subCategory && <p className="text-red-500 text-xs
font-bold">{error.subCategory}</p>}
462.     </div>
463. </div>
464. </form>
465. </>
466. })
467. </div>
468. );
469. };
470.
471. export default AddProductPanel;
472.

```

File Name: frontend/src/Component/Seller/ProductsPanel.jsx

```

1. import axios from "axios";
2. import { Button, Input, SectionTitle, Select, ToggleSwitch } from
"../../LIBS";
3. import { useEffect, useState } from "react";
4. import { Link } from "react-router-dom";
5. import { BiLoaderAlt } from "react-icons/bi";
6. import { FaTrash, FaSearch, FaBoxOpen, FaChevronLeft, FaChevronRight,
FaPen } from "react-icons/fa";
7. import { useTheme } from "../../Context/themeContext";
8. import { useAuth } from "../../Context/authContext";
9. import { formatNumber, swalWithCustomConfiguration } from
"../../utility/constant";
10. import { useCategory } from "../../Context/categoryContext";
11.
12. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
13.
14. export const ProductsPanel = () => {
15.     const [searchQuery, setSearchQuery] = useState("");
16.     const [productArr, setProductArr] = useState([]);
17.     const [currentPage, setCurrentPage] = useState(1);
18.     const [totalPages, setTotalPages] = useState(1);
19.     const [productCount, setProductCount] = useState(null);
20.     const [productRange, setProductRange] = useState({});
21.     const [isDataFetching, setIsDataFetching] = useState(false);

```

```

22.   const [deletingProductId, setDeletingProductId] = useState("");
23.   const [filters, setFilters] = useState({});
24.   const { theme } = useTheme();
25.   const { authToken } = useAuth();
26.   const { categories } = useCategory();
27.
28.   const dynamicCategories = categories?.filter(c => c.parentCategory
=== null).map(mainCat => {
29.     return { id: mainCat._id, value: mainCat._id, displayName:
mainCat.name };
30.   }) || [];
31.
32.   let tempSearchQuery = "";
33.   const visiblePages = Array.from({ length: totalPages }, (_, index) =>
index + 1)
34.     .filter((page) => page >= Math.max(1, currentPage - 1))
35.     .filter((page) => page <= Math.min(totalPages, currentPage + 1));
36.
37.   const getAllProduct = async () => {
38.     setIsDataFetching(true);
39.     try {
40.       const response = await axios.get(
41.         `${SERVER_URL}/api/seller/get-all-product`,
42.         {
43.           headers: {
44.             "Content-Type": "application/json; charset=UTF-8",
45.             Authorization: `Bearer ${authToken}`,
46.           },
47.           params: { filters, page: currentPage, search: searchQuery },
48.         }
49.       );
50.       setProductArr(response?.data?.allProduct);
51.       setTotalPages(response?.data?.totalPages);
52.       setProductCount(response?.data?.totalProducts);
53.       setProductRange({
54.         startProductIndex: response?.data?.startProductIndex,
55.         endProductIndex: response?.data?.endProductIndex,
56.       });
57.     } catch (error) {
58.       console.error(error?.response?.data?.message || error.message);
59.     } finally {
60.       setIsDataFetching(false);
61.     }
62.   };
63.
64.   const handleFiltersChange = (e) => {
65.     const { name, value } = e.target || {};
66.     setFilters((prev) => {
67.       return {
68.         ...prev,
69.         [name]: value?.toString()?.trim(),
70.       };
71.     });
72.     setCurrentPage(1);
73.   };
74.
75.   useEffect(() => {
76.     getAllProduct();
77.   }, [authToken, JSON.stringify(filters), currentPage, searchQuery]);
78.
79.   const handleDeleteProduct = async (productId) => {

```

```

80.     const response = await swalWithCustomConfiguration.fire({
81.         title: "Delete product?",
82.         text: "This product will be permanently removed from your
inventory.",
83.         icon: "warning",
84.         showCancelButton: true,
85.         confirmButtonText: "Delete",
86.         cancelButtonText: "Cancel",
87.         reverseButtons: true,
88.     });
89.
90.     if (!response.isConfirmed) return;
91.
92.     try {
93.         setDeletingProductId(productId);
94.         const deleteResponse = await axios.delete(
95.             `${SERVER_URL}/api/seller/product/${productId}`,
96.             {
97.                 headers: {
98.                     "Content-Type": "application/json; charset=UTF-8",
99.                     Authorization: `Bearer ${authToken}`,
100.                },
101.            }
102.        );
103.
104.        setProductArr((prev) => prev.filter((product) => product?._id
!== productId));
105.        setProductCount((prev) => (typeof prev === "number" ?
Math.max(prev - 1, 0) : prev));
106.
107.        await swalWithCustomConfiguration.fire(
108.            "Deleted",
109.            deleteResponse?.data?.message || "Product deleted
successfully",
110.            "success"
111.        );
112.        await getAllProduct();
113.    } catch (error) {
114.        swalWithCustomConfiguration.fire(
115.            "Delete failed",
116.            error?.response?.data?.message || "Unable to delete product
right now",
117.            "error"
118.        );
119.    } finally {
120.        setDeletingProductId("");
121.    }
122. };
123.
124. return (
125.     <div className={`w-full max-w-7xl mx-auto overflow-x-hidden
${theme === "dark" ? "bg-gray-900 text-white" : "bg-gray-100 text-gray-
900"} h-full transition-all duration-300 pb-12`} >
126.         <main className="px-2 tablet:px-6 mt-2">
127.
128.             {/* Header */}
129.             <div className="flex flex-col tablet:flex-row justify-between
items-start tablet:items-center mb-8 gap-4">
130.                 <div>
131.                     <h1 className="text-3xl font-extrabold tracking-tight mb-
2">Inventory</h1>

```

```

132.         <p className={` ${theme === 'dark' ? 'text-gray-400' :
'text-gray-500'} `}>Manage your product catalog</p>
133.     </div>
134.     <Link to={"add"}>
135.         <Button
136.             className="px-6 py-3 font-bold bg-indigo-600 text-white
rounded-xl shadow-lg shadow-indigo-500/30 hover:bg-indigo-700 transition-
colors w-full tablet:w-auto"
137.             btnText="+ Add New Product"
138.         />
139.     </Link>
140. </div>
141.
142.     { /* Search & Filter Bar */ }
143.     <div className={` p-4 tablet:p-5 rounded-2xl border shadow-sm
flex flex-col tablet:flex-row items-center gap-4 mb-8 ${theme === 'dark' ?
'bg-gray-800/50 border-gray-800' : 'bg-white border-gray-200'} `}>
144.         <div className="relative w-full tablet:flex-1">
145.             <FaSearch className="absolute left-4 top-1/2 -translate-y-
1/2 text-gray-400" />
146.             <Input
147.                 type="text"
148.                 placeholder="Search Products by name or ID..."
149.                 className={` w-full pl-11 pr-4 py-3 rounded-xl border-2
font-medium focus:ring-4 outline-none transition-all ${
150.                     theme === "dark"
151.                     ? "bg-gray-900 text-white border-gray-700
focus:border-indigo-500 focus:ring-indigo-500/20"
152.                     : "bg-gray-50 text-gray-900 border-gray-200
focus:border-indigo-500 focus:ring-indigo-500/20"
153.                 } `>
154.                 value={searchQuery}
155.                 onChange={(e) => { tempSearchQuery = e?.target?.value;
}}
156.                 onKeyDown={(e) => {
157.                     if (e?.key?.toString().toLowerCase() === "enter") {
158.                         setSearchQuery(tempSearchQuery);
159.                     }
160.                 }}
161.             />
162.         </div>
163.
164.         <Select
165.             name="category"
166.             itemArray={dynamicCategories}
167.             className={` w-full tablet:w-[200px] px-4 py-3 rounded-xl
border-2 font-medium cursor-pointer focus:ring-4 outline-none transition-
all flex-shrink-0 ${
168.                 theme === "dark"
169.                 ? "bg-gray-900 text-white border-gray-700
focus:border-indigo-500 focus:ring-indigo-500/20"
170.                 : "bg-gray-50 text-gray-900 border-gray-200
focus:border-indigo-500 focus:ring-indigo-500/20"
171.             } `>
172.             displayName="All Categories"
173.             onChange={handleFiltersChange}
174.         />
175.         <Select
176.             name="productStockStatus"
177.             itemArray={[
178.                 { id: 1, value: "in stock" },

```

```

179.             { id: 2, value: "low stock" },
180.             { id: 3, value: "out of stock" },
181.         ]}
182.         className={`w-full tablet:w-[200px] px-4 py-3 rounded-xl
border-2 font-medium cursor-pointer focus:ring-4 outline-none transition-
all flex-shrink-0 ${
183.             theme === "dark"
184.             ? "bg-gray-900 text-white border-gray-700
focus:border-indigo-500 focus:ring-indigo-500/20"
185.             : "bg-gray-50 text-gray-900 border-gray-200
focus:border-indigo-500 focus:ring-indigo-500/20"
186.         }`}
187.         onChange={handleFiltersChange}
188.         displayName="All Status"
189.     />
190. </div>
191.
192.     {/* Product Roster TABLE */}
193.     <div className="relative min-h-[50vh]">
194.         {isDataFetching ? (
195.             <div className="flex flex-col items-center justify-center
p-20 opacity-50">
196.                 <BiLoaderAlt className="animate-spin text-4xl mb-4
text-indigo-500" />
197.                 <p className="font-bold tracking-widest
uppercase">Fetching Inventory...</p>
198.             </div>
199.         ) : productArr?.length === 0 ? (
200.             <div className={`rounded-3xl border p-16 flex flex-col
items-center justify-center text-center shadow-sm transition-colors ${theme
=== "dark" ? "border-gray-800 bg-gray-900 text-gray-400" : "border-gray-200
bg-white text-gray-500"}`}>
201.                 <div className="p-5 bg-indigo-500/10 rounded-full mb-
6">
202.                     <FaBoxOpen className="text-6xl text-indigo-500
opacity-50 relative top-1" />
203.                 </div>
204.                 <h3 className="text-xl font-bold mb-2">No Products
Found</h3>
205.                 <span className="text-sm">Adjust your filters or add a
new product to your store.</span>
206.             </div>
207.         ) : (
208.             <div className={`w-full max-w-full overflow-hidden
rounded-2xl border shadow-lg ${theme === 'dark' ? 'bg-gray-900 border-gray-
800' : 'bg-white border-gray-200'}`}>
209.                 <div className="w-full overflow-x-auto fancy-scroll">
210.                     <table className="w-full text-left border-collapse
min-w-[900px]">
211.                         <thead className={`text-[11px] uppercase tracking-
widest font-bold border-b transition-colors ${theme === 'dark' ? 'bg-gray-
800/50 text-gray-500 border-gray-800' : 'bg-gray-50 text-gray-500 border-
gray-100'}`}>
212.                             <tr>
213.                                 <th className="p-4 pl-6">Product</th>
214.                                 <th className="p-4">Category</th>
215.                                 <th className="p-4">Price</th>
216.                                 <th className="p-4">Stock</th>
217.                                 <th className="p-4 text-center">Visibility</th>
218.                                 <th className="p-4 pr-6 text-right">Actions</th>
219.                             </tr>

```

```

220.                 </thead>
221.                 <tbody className={`divide-y transition-colors
${theme === 'dark' ? 'divide-gray-800/60' : 'divide-gray-100'}}`>
222.                     {productArr.map((product) => (
223.                         <tr key={product?._id} className={`transition-
colors hover:bg-black/5 dark:hover:bg-white/5`}>
224.                             <td className="p-4 pl-6 flex items-center gap-
4">
225.                                 <img
226.                                     src={product?.image?.[0]}
227.                                     alt={product?.name}
228.                                     className={`w-14 h-14 rounded-lg object-
cover shadow-sm border ${theme === 'dark' ? 'border-gray-700 bg-gray-800' :
'border-gray-200 bg-white'}}`>
229.                                 />
230.                                 <div className="min-w-0">
231.                                     <p className={`font-bold leading-tight
line-clamp-1 max-w-[250px] ${theme === 'dark' ? 'text-gray-200' : 'text-
gray-800'}}`>
232.                                         {product?.name}
233.                                     </p>
234.                                     <p className="text-[11px] mt-1 text-gray-
500 line-clamp-1 max-w-[250px]" title={product?.description}>
235.                                         {product?.description}
236.                                     </p>
237.                                 </div>
238.                             </td>
239.                             <td className="p-4">
240.                                 <span className={`px-3 py-1 rounded-full
text-[10px] font-black uppercase tracking-widest ${theme === 'dark' ? 'bg-
gray-800 text-gray-300' : 'bg-gray-100 text-gray-600'}}`>
241.                                     {product?.category?.name?.toCapitalizе()}
242.                                 </span>
243.                             </td>
244.                             <td className={`p-4 font-black ${theme ===
'dark' ? 'text-white' : 'text-gray-900'}}`>
245.                                 {formatNumber(product?.price ||
product?.mrpPrice)}
246.                             </td>
247.                             <td className="p-4">
248.                                 <span className={`text-[11px] font-bold
uppercase tracking-widest ${product?.stock > 20 ? "text-green-500" :
product?.stock > 0 ? "text-yellow-500" : "text-red-500"}}`>
249.                                     {product?.stock > 20 ? `${product.stock}
In Stock` : product?.stock > 0 ? `${product.stock} Low Stock` : "Out of
Stock"}
250.                                 </span>
251.                             </td>
252.                             <td className="p-4">
253.                                 <div className="flex justify-center scale-
90">
254.                                     <ToggleSwitch
255.                                         visibilityState={!product?.isHide}
256.                                         onToggle={async (state) => {
257.                                             try {
258.                                                 await axios.patch(
259.                                                     `${SERVER_URL}/api/seller/product/visibility-toggle/${product?._id}`,
260.                                                     { state },
261.                                                     { headers: { "Content-Type":
"application/json; charset=UTF-8", Authorization: `Bearer ${authToken}` } }

```

```

262.                );
263.                } catch (error) { console.log(error);
264.            }
265.            }
266.            </div>
267.        </td>
268.        <td className="p-4 pr-6 text-right">
269.            <div className="flex items-center justify-
end gap-2">
270.                <Link
to={`\seller/products/edit/${product?._id}`}>
271.                    <Button
272.                        className={`p-2.5 rounded-lg border-2
inline-flex items-center justify-center transition-all duration-300 ${
273.                            theme === "dark" ? "border-indigo-
500/20 text-indigo-400 hover:bg-indigo-500/20 hover:border-indigo-500 bg-
indigo-500/10"
274.                            : "border-indigo-
200 text-indigo-600 hover:bg-indigo-50 hover:border-indigo-300 bg-white"
275.                        }`>
276.                        btntext=""
277.                        icon={ <FaPen className="text-base" /> }
278.                    </Button>
279.                </Link>
280.
281.                <Button
282.                    className={`p-2.5 rounded-lg border-2
inline-flex items-center justify-center transition-all duration-300
disabled:opacity-50 ${
283.                        theme === "dark" ? "border-red-500/20
text-red-400 hover:bg-red-500/20 hover:border-red-500 bg-red-500/10"
284.                        : "border-red-200
text-red-600 hover:bg-red-50 hover:border-red-300 bg-white"
285.                    }`>
286.                    btntext=""
287.                    icon={ deletingProductId ===
product?._id ? <BiLoaderAlt className="animate-spin text-base" /> :
<FaTrash className="text-base" /> }
288.                    onClick={() =>
handleDeleteProduct(product?._id)}
289.                    disabled={deletingProductId ===
product?._id}
290.                </Button>
291.            </div>
292.        </td>
293.    </tr>
294.    </tbody>
295. </table>
296. </div>
297. </div>
298. </div>
299. )}
300.
301.    <\/* Pagination Component */>
302.    {productArr?.length > 0 && !isDataFetching && (
303.        <div className={`mt-8 flex flex-col tablet:flex-row items-
center justify-between gap-4 border p-4 rounded-2xl shadow-xl ${theme ===
"dark" ? "border-gray-700 bg-gray-900/95" : "border-gray-200 bg-
white/95"}`}>

```



```

348.             )}
349.
350.             <Button
351.                 className={`px-4 py-2 font-bold tracking-wide
rounded-lg disabled:opacity-50 transition-colors flex items-center gap-2
${theme === "dark" ? "bg-gray-800 text-white hover:bg-gray-700" : "bg-gray-
100 text-gray-900 hover:bg-gray-200"}`
352.                 btnText="Next"
353.                 icon={<FaChevronRight className="text-[10px]" />}
354.                 iconPosition="right"
355.                 onClick={() => { setCurrentPage((prev) => prev <
totalPages ? prev + 1 : prev); }}
356.                 disabled={currentPage === totalPages}
357.             />
358.         </div>
359.     </div>
360.     )}
361. </div>
362. </main>
363. </div>
364. );
365. };
366.
367. export default ProductsPanel;
368.

```

File Name: frontend\src\Component\Seller\UploadProduct\ImagePreview.jsx

```

1. import React, { useState, useEffect } from "react";
2. import { FiChevronLeft, FiChevronRight, FiTrash2 } from "react-
icons/fi";
3.
4. const ImagePreview = ({ image, onRemove }) => {
5.     const [currentIndex, setCurrentIndex] = useState(0);
6.
7.     const handlePrev = (e) => {
8.         e.preventDefault();
9.         setCurrentIndex((prev) => (prev > 0 ? prev - 1 : image.length - 1));
10.    };
11.
12.    const handleNext = (e) => {
13.        e.preventDefault();
14.        setCurrentIndex((prev) => (prev + 1) % image.length);
15.    };
16.
17.    if (!image || image.length === 0) return null;
18.
19.    const safeIndex = currentIndex >= image.length ? Math.max(0,
image.length - 1) : currentIndex;
20.
21.    const handleDelete = (e) => {
22.        e.preventDefault();
23.        if (onRemove) {
24.            onRemove(safeIndex);
25.            // Automatically adjust index backwards if we delete the last
item
26.            if (safeIndex >= image.length - 1) {
27.                setCurrentIndex(Math.max(0, image.length - 2));
28.            }

```

```

29.     }
30.   };
31.
32.   return (
33.     <div className="w-full flex flex-col gap-4 bg-white dark:bg-gray-
900 rounded-xl">
34.       {/* Main Large Preview Area */}
35.       <div className="relative w-full h-[280px] bg-gray-100 dark:bg-
gray-800 rounded-xl overflow-hidden shadow-inner group border border-gray-
200 dark:border-gray-700">
36.         <img
37.           src={typeof image[safeIndex] === 'string' ? image[safeIndex]
: URL.createObjectURL(image[safeIndex])}
38.           className="w-full h-full object-contain transition-transform
duration-300"
39.           alt="Product Preview"
40.         />
41.
42.         {/* Delete Button overlaid top-left */}
43.         <button
44.           onClick={handleDelete}
45.           title="Remove this image"
46.           className="absolute top-3 left-3 bg-red-500/90 text-white p-2
rounded-lg shadow-md backdrop-blur-md opacity-0 group-hover:opacity-100
transition-all hover:bg-red-600 hover:scale-105 active:scale-95"
47.         >
48.           <FiTrash2 size={18} />
49.         </button>
50.
51.         {/* Carousel Overlay Nav - Only show if > 1 image */}
52.         {image.length > 1 && (
53.           <>
54.             <button
55.               onClick={handlePrev}
56.               disabled={safeIndex === 0}
57.               className={`absolute left-2 top-1/2 -translate-y-1/2 p-2
rounded-full backdrop-blur-md shadow-md transition-all ${
58.                 safeIndex === 0
59.                 ? "bg-white/40 text-gray-400 cursor-not-allowed
hidden"
60.                 : "bg-white/80 text-gray-800 hover:bg-white
hover:scale-110 active:scale-95"
61.               }}
62.             >
63.               <FiChevronLeft size={24} />
64.             </button>
65.             <button
66.               onClick={handleNext}
67.               disabled={safeIndex === image.length - 1}
68.               className={`absolute right-2 top-1/2 -translate-y-1/2 p-2
rounded-full backdrop-blur-md shadow-md transition-all ${
69.                 safeIndex === image.length - 1
70.                 ? "bg-white/40 text-gray-400 cursor-not-allowed
hidden"
71.                 : "bg-white/80 text-gray-800 hover:bg-white
hover:scale-110 active:scale-95"
72.               }}
73.             >
74.               <FiChevronRight size={24} />
75.             </button>
76.

```

```

77.             {/* Image Counter Badge */}
78.             <div className="absolute top-3 right-3 bg-black/60
backdrop-blur-md text-white text-[11px] font-bold tracking-widest px-3 py-1
rounded-full shadow-lg">
79.                 {safeIndex + 1} / {image.length}
80.             </div>
81.         </>
82.     )}
83. </div>
84.
85.     {/* Thumbnails Row */}
86.     {image.length > 1 && (
87.         <div className="flex gap-3 overflow-x-auto pb-2 fancy-scroll
px-1">
88.             {image.map((file, idx) => (
89.                 <button
90.                     key={idx}
91.                     type="button"
92.                     onClick={e => {
93.                         e.preventDefault();
94.                         setCurrentIndex(idx);
95.                     }}
96.                     className={`relative flex-shrink-0 w-16 h-16 rounded-lg
overflow-hidden border-2 transition-all duration-200 ${
97.                         safeIndex === idx
98.                         ? "border-purple-600 shadow-
[0_0_2px_rgba(147,51,234,0.3)] scale-105"
99.                         : "border-transparent opacity-60 hover:opacity-100
hover:border-gray-300 dark:hover:border-gray-600"
100.                     }`}
101.                 >
102.                     <img
103.                         src={typeof file === 'string' ? file :
URL.createObjectURL(file)}
104.                         alt={`Thumbnail ${idx}`}
105.                         className="w-full h-full object-cover"
106.                     />
107.                     <div
108.                         className={`absolute inset-0 bg-purple-600/20 pointer-
events-none transition-opacity ${
109.                             safeIndex === idx ? "opacity-100" : "opacity-0"
110.                         }`}
111.                     />
112.                 </button>
113.             )})
114.         </div>
115.     )}
116. </div>
117. );
118. };
119.
120. export default ImagePreview;
121.

```

File Name: frontend\src\Component\Seller\UploadProduct\ProductUpload.jsx

```

1. import React, { useState } from "react";
2. // import Input from "../UI/Input";
3. // import Button from "../UI/Button";

```

```

4. import { Button, Input, TextArea } from "../../../../../LIBS";
5. import axios from "axios";
6. import { FiUpload } from "react-icons/fi";
7. import { useDropzone } from "react-dropzone";
8. import ImagePreview from "../ImagePreview";
9. import {
10.   productBrands,
11.   swalWithCustomConfiguration,
12. } from "../../../../../utility/constant";
13. import { useCategory } from "../../../../../Context/categoryContext";
14. import Select from "../../../../../LIBS/Select";
15. import { useTheme } from "../../../../../Context/themeContext";
16. // import cloudinary from "../../cloudinary.config";
17. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
18. const API_KEY = import.meta.env.VITE_CLOUDINARY_API_KEY;
19. const CLOUD_NAME = import.meta.env.VITE_CLOUDINARY_CLOUD_NAME;
20. const ProductUpload = ({ authToken }) => {
21.   const [productUploading, setProductUploading] = useState(false);
22.   const { theme } = useTheme();
23.   const { categories } = useCategory();
24.
25.   const dynamicCategories = categories?.filter(c => c.parentCategory
=== null).map(mainCat => {
26.     return {
27.       id: mainCat._id,
28.       name: "category",
29.       value: mainCat._id, // Use ID for value
30.       displayName: mainCat.name, // Use name for UI
31.       child: categories.filter(c => c.parentCategory === mainCat._id
|| c.parentCategory?._id === mainCat._id).map(c => ({ id: c._id, name:
c.name }))
32.     };
33.   }) || [];
34.   const [initialProductDetail] = useState({
35.     name: "",
36.     price: "",
37.     image: [],
38.     category: "",
39.     subCategory: "",
40.     description: "",
41.     stock: "",
42.     brand: "",
43.     mrpPrice: "",
44.     sellingPrice: "",
45.   });
46.   const [productDetails, setProductDetails] =
useState(initialProductDetail);
47.   const [image, setImage] = useState([]);
48.   const [error, setError] = useState({
49.     name: "",
50.     price: "",
51.     image: "",
52.     category: "",
53.     subCategory: "",
54.     description: "",
55.     stock: "",
56.     brand: "",
57.     mrpPrice: "",
58.     sellingPrice: "",
59.   });
60.

```

```

61.   const onDrop = (accepted) => {
62.     setImage((prev) => [...prev, ...accepted]);
63.   };
64.   const { getRootProps, getInputProps, isDragActive } = useDropzone({
65.     onDrop,
66.     accept: "image/*",
67.     multiple: true,
68.   });
69.   const handleChange = (e) => {
70.     const { name, value } = e.target;
71.     console.log(name, value);
72.     setProductDetails({ ...productDetails, [name]: value });
73.   };
74.
75.   const handleFileChange = (e) => {
76.     const files = Array.from(e.target.files);
77.     setImage((prev) => [...prev, ...files]);
78.   };
79.   const saveImageToCloudinary = async () => {
80.     try {
81.       const uploadPromises = image.map(async (file) => {
82.         const formData = new FormData();
83.         formData.append("file", file);
84.         formData.append("upload_preset", "mycloud");
85.         formData.append("cloud_name", CLOUD_NAME);
86.         const response = await axios({
87.           method: "POST",
88.           url:
89.             `https://api.cloudinary.com/v1_1/${CLOUD_NAME}/image/upload`,
90.           data: formData,
91.           headers: {
92.             "Content-Type": "application/x-www-form-urlencoded",
93.           },
94.         });
95.         console.log(response?.data?.secure_url?.toString());
96.         return response?.data?.secure_url?.toString();
97.       });
98.       const imageUrl = await Promise.all(uploadPromises);
99.       console.log(imageUrl);
100.      const updatedProductDetail = {
101.        ...productDetails,
102.        image: [...productDetails?.image, ...imageUrl],
103.      };
104.      setProductDetails(updatedProductDetail);
105.      return updatedProductDetail;
106.    } catch (error) {
107.      console.log(error);
108.    }
109.  };
110.  const uploadProduct = async (detail) => {
111.    axios({
112.      method: "POST",
113.      url: `${SERVER_URL}/api/product/add-product`,
114.      data: detail,
115.      headers: {
116.        "Content-Type": "application/json",
117.        Authorization: `Bearer ${authToken}`,
118.      },
119.    })
120.      .then((response) => {

```

```

121.     const { data, status } = response;
122.     if (status === 201) {
123.       swalWithCustomConfiguration
124.         ?.fire(
125.           "Product successfully uploaded",
126.           "your product is added",
127.           "success"
128.         )
129.         .then(() => {
130.           setProductDetails(initialProductDetail);
131.           setImage(() => []);
132.         });
133.     }
134.     setProductUploading(false);
135.     // setProductDetails({});
136.     // setError({});
137.     // setImage([]);
138.   })
139.   .catch((error) => {
140.     const { data, status } = error?.response;
141.     setProductUploading(false);
142.     if (status === 500) {
143.       swalWithCustomConfiguration.fire(
144.         "Error uploading!",
145.         data?.message,
146.         "error"
147.       );
148.     } else if (status === 400) {
149.       swalWithCustomConfiguration?.fire("Error uploading!",
data?.message);
150.     }
151.     console.log("Error uploading product", error);
152.     console.log(productDetails);
153.   });
154. };
155.
156. const handleSubmit = async (e) => {
157.   e.preventDefault();
158.   setProductUploading((prev) => !prev);
159.   const updatedProductDetail = await saveImageToCloudinary();
160.   if (updatedProductDetail) {
161.     await uploadProduct(updatedProductDetail);
162.   }
163. };
164.
165. return (
166.   <div className="w-11/12 p-3">
167.     <form
168.       action=""
169.       className="h-full flex flex-col gap-3 overflow-y-scroll"
170.       onSubmit={handleSubmit}
171.       method="POST"
172.     >
173.       <div
174.         {...getRootProps()}
175.         className={`w-full min-h-[200px] flex flex-col justify-
center items-center border-2 ${
176.           image.length > 0 ? "hidden" : "flex"
177.         } ${
178.           isDragActive
179.             ? "border-blue-500 border-solid"

```

```

180.         : "border-gray-500 border-dashed"
181.     }   rounded gap-3 overflow-scroll`}
182. >
183.     {isDragActive ? (
184.         <span>Drop Here</span>
185.     ) : (
186.         <div className="flex flex-col items-center justify-center
gap-2">
187.             <span>
188.                 <FiUpload className="text-3xl" />
189.             </span>
190.             <span className="font-roboto">
191.                 <span>Drag & Drop or </span>
192.                 <span>
193.                     <label
194.                         htmlFor="image"
195.                         className="text-blue-500 underline cursor-pointer"
196.                     >
197.                         Choose file
198.                     </label>
199.                     <Input
200.                         type={"file"}
201.                         id={"image"}
202.                         name={"image"}
203.                         // value={productDetails.image}
204.                         className={"hidden"}
205.                         multiple={true}
206.                         accept={"image/*"}
207.                         onChange={handleFileChange}
208.                     />
209.                 </span>
210.                 <span> to upload</span>
211.             </span>
212.         </div>
213.     )}
214. </div>
215. {/* Image Preview Section */}
216. {image.length > 0 && <ImagePreview image={image} />}
217.
218. <div className=" product-detail-container grid items-stretch
grid-cols-2 gap-5">
219.     {/* <div className="w-full flex gap-3"> */}
220.     {/* Product Name Section */}
221.     <div className=" flex flex-col gap-3">
222.         <label htmlFor="name">
223.             Product Name <span className="required">*</span>
224.         </label>
225.         <Input
226.             type={"text"}
227.             id={"name"}
228.             name={"name"}
229.             value={productDetails?.name}
230.             onChange={handleChange}
231.             placeholder={"Product Name"}
232.             className="p-2 bg-gray-100 border-2 rounded border-
gray-300"
233.         />
234.         {error.name && <p>{error.name}</p>}
235.     </div>
236.
237.     {/* Brand Section */}

```



```

238. <div className=" flex flex-col gap-3">
239.   <label htmlFor="stock">
240.     Brand <span className="required">*</span>
241.   </label>
242.   <Select
243.     name="brand"
244.     id="brand"
245.     className="p-2 bg-gray-100 border-2 rounded border-gray-
300"
246.     onChange={handleChange}
247.     value={productDetails?.brand}
248.     displayName="Select Brand"
249.     itemArray={productBrands}
250.   />
251.   {/* <select
252.     name="brand"
253.     id="brand"
254.     className="p-2 bg-gray-100 border-2 rounded "
255.     onChange={handleChange}
256.     value={productDetails?.brand}
257.   >
258.     <option value="">Select Brand</option>
259.     {productBrands?.map((item) => {
260.       return (
261.         <option key={item?.id} value={item?.value}>
262.           {item.value.toCapitalize()}
263.         </option>
264.       );
265.     })}
266.   </select> */}
267. </div>
268. {/* Category Section */}
269. <div className=" flex flex-col gap-3 ">
270.   <label htmlFor="category">
271.     Category <span className="required">*</span>
272.   </label>
273.   <Select
274.     name="category"
275.     id="category"
276.     className="p-2 bg-gray-100 border-2 rounded border-gray-
300"
277.     onChange={handleChange}
278.     value={productDetails?.category}
279.     itemArray={dynamicCategories}
280.     displayName={"Select Category"}
281.   />
282.   {/* <select
283.     name="category"
284.     id="category"
285.     className="p-2 bg-gray-100 border-2 rounded border-gray-
300 outline-gray-400 outline-2"
286.     onChange={handleChange}
287.     value={productDetails?.category}
288.   >
289.     <option value="">Select Category</option>
290.     {productCategory?.map((item) => {
291.       return (
292.         <option key={item?.id} value={item?.value}>
293.           {item.value.toCapitalize()}
294.         </option>
295.       );

```

```

296.         }}}
297.     </select> */}
298.     {error.category && <p>{error.category}</p>}
299. </div>
300.
301.     {/* Sub Category Section */}
302.     <div className=" flex flex-col gap-3">
303.         <label htmlFor="subCategory">
304.             Sub Category <span className="required">*</span>
305.         </label>
306.         {/* <Select
307.             name="subCategory"
308.             className="p-2 bg-gray-100 border-2 rounded border-gray-
300 outline-gray-400 outline-2"
309.             id="subCategory"
310.             disabled={
311.                 productDetails?.category === "others" ||
312.                 !productDetails?.category
313.             }
314.             onChange={handleChange}
315.             value={productDetails?.subCategory}
316.             displayName={"Select Sub Category"}
317.             itemArray={productCategory}
318.         /> */}
319.         <select
320.             name="subCategory"
321.             className={`p-2 bg-gray-100 border-2 rounded border-
gray-300 outline-gray-400 outline-0 transition-all duration-300 ${
322.                 theme === "dark"
323.                 ? "bg-gray-700 text-white border-gray-600
focus:border-gray-300"
324.                 : "text-gray-900 bg-gray-100 border-gray-300
focus:border-gray-600"
325.             } `}
326.             id="subCategory"
327.             disabled={
328.                 productDetails?.category === "others" ||
329.                 !productDetails?.category
330.             }
331.             onChange={handleChange}
332.             value={productDetails?.subCategory}
333.         >
334.             <option value="">Select Sub Category</option>
335.             {dynamicCategories?.find(item => item.value ===
productDetails?.category)?.child?.map((subItem) => (
336.                 <option key={subItem.id} value={subItem.id}>
337.                     {subItem.name.toCapitalized()}
338.                 </option>
339.             )
)}}
340.         </select>
341.     </div>
342.     {/* // )} */}
343.     {/* </div> */}
344.     {/* <div className="w-full flex gap-3"> */}
345.
346.     {/* MRP Section */}
347.     <div className=" flex flex-col gap-3">
348.         <label htmlFor="mrpPrice">
349.             MRP Price <span className="required">*</span>
350.         </label>
351.         <Input

```

```

352.         type={"number"}
353.         id={"mrpPrice"}
354.         name={"mrpPrice"}
355.         value={productDetails?.mrpPrice}
356.         onChange={handleChange}
357.         placeholder={"MRP Price"}
358.         className={"p-2 bg-gray-100 border-2 rounded border-
gray-300"}
359.     />
360.     {error.price && <p>{error.mrpPrice}</p>}
361. </div>
362.
363.     {/* Discounted Price Section */}
364.     <div className="flex flex-col gap-3">
365.         <label htmlFor="discountedPrice">
366.             Selling Price <span className="required">*</span>
367.         </label>
368.         <Input
369.             type={"number"}
370.             id={"sellingPrice"}
371.             name={"sellingPrice"}
372.             value={productDetails?.sellingPrice}
373.             onChange={handleChange}
374.             placeholder={"Selling Price"}
375.             className={"p-2 bg-gray-100 border-2 rounded border-
gray-300"}
376.         />
377.         {error.price && <p>{error.sellingPrice}</p>}
378.     </div>
379.     {/* </div> */}
380.     {/* <div className="w-full flex gap-3"> */}
381.     <div className="flex flex-col gap-3">
382.         <label htmlFor="stock">
383.             Stock <span className="required">*</span>
384.         </label>
385.         <Input
386.             type={"number"}
387.             id={"stock"}
388.             name={"stock"}
389.             value={productDetails?.stock}
390.             onChange={handleChange}
391.             placeholder={"Stock"}
392.             className={"p-2 bg-gray-100 border-2 rounded border-
gray-300"}
393.         />
394.         {error.quantity && <p>{error.stock}</p>}
395.     </div>
396.     {/* </div> */}
397.
398.     <div className="w-full col-span-2 flex flex-col gap-3">
399.         <div className="w-full flex flex-col gap-3">
400.             <label htmlFor="description">
401.                 Description <span className="required">*</span>
402.             </label>
403.             <TextArea
404.                 name="description"
405.                 id={"description"}
406.                 value={productDetails?.description}
407.                 onChange={handleChange}
408.                 placeholder="Description"

```

```

409.             className={`resize-none w-full min-h-[150px] p-2
border-2 outline-none rounded ${
410.                 theme === "dark"
411.                 ? "bg-gray-700 text-white border-gray-600
focus:border-gray-300"
412.                 : "text-gray-900 bg-gray-100 border-gray-300
focus:border-gray-600"
413.             }}
414.         />
415.         {/* <textarea
416.             name="description"
417.             id={"description"}
418.             value={productDetails?.description}
419.             onChange={handleChange}
420.             placeholder="Description"
421.             className={`resize-none w-full min-h-[150px] p-2
border-2 outline-none rounded ${
422.                 theme === "dark"
423.                 ? "bg-gray-700 text-white border-gray-600
focus:border-gray-300"
424.                 : "text-gray-900 bg-gray-100 border-gray-300
focus:border-gray-600"
425.             }}
426.             ></textarea> */}
427.         {error.description && <p>{error.description}</p>}
428.     </div>
429. </div>
430. </div>
431. <div className="">
432.     <Button
433.         btnText={"Upload"}
434.         loading={productUploading}
435.         className={"w-full text-white bg-indigo-600 p-2 rounded"}
436.     />
437. </div>
438. </form>
439. </div>
440. );
441. };
442.
443. export default ProductUpload;
444.

```

File Name: frontend\src\Context\authContext.jsx

```

1. import { createContext, useState, useContext } from "react";
2. const authContext = createContext();
3. const useAuth = () => useContext(authContext);
4. const AuthProvider = ({ children }) => {
5.     // const [theme, setTheme] = useState(() => {
6.     //     return localStorage.getItem("theme") || "dark";
7.     // });
8.     // const toggleTheme = () => {
9.     //     // setTheme(theme === "light"? "dark" : "light");
10.    //     setTheme((prevState) => (prevState === "light" ? "dark" :
"light"));
11.    // };
12.    // useEffect(() => {
13.    //     localStorage.setItem("theme", theme);

```

```

14.    //    }, [theme]);
15.
16.    const [authToken, setAuthToken] = useState(
17.      () => localStorage.getItem("AuthToken") || null
18.    );
19.    return (
20.      <authContext.Provider value={{ authToken, setAuthToken }}>
21.        {children}
22.      </authContext.Provider>
23.    );
24.  };
25.
26. export { AuthProvider, authContext, useAuth };
27.

```

File Name: frontend\src\Context\cartContext.jsx

```

1. // const { createContext, useState, useContext } = require("react");
2. import { createContext, useState, useContext } from "react";
3.
4. const CartContext = createContext();
5. const useCart = () => useContext(CartContext);
6. const CartProvider = ({ children }) => {
7.   const [cartCount, setCartCount] = useState(0);
8.   //   const getCartItem = () => {
9.     //     axios({
10.      //       method: "POST",
11.      //       url: `${SERVER_URL}/api/user/get-cart-items`,
12.      //       headers: { Authorization: `Bearer ${authToken}` },
13.      //       data: { userId: userDetails?.id },
14.      //     })
15.      //     .then((response) => {
16.      //       setCartCount(response?.data?.cartCount);
17.      //       // setCartItem(response.data);
18.      //     })
19.      //     .catch((error) => {
20.      //       console.error(error);
21.      //       setError(() => {
22.      //         return {
23.      //           message: "Failed to fetch cart items",
24.      //           status: error?.response?.status,
25.      //         };
26.      //       });
27.      //     });
28.      //   };
29.      //   useEffect(() => {
30.      //     getCartItem();
31.      //   }, []);
32.      return (
33.        <CartContext.Provider value={{ cartCount, setCartCount }}>
34.          {children}
35.        </CartContext.Provider>
36.      );
37.    };
38.
39. export { CartProvider, CartContext, useCart };
40.

```

File Name: frontend\src\main.jsx

```
1. import { StrictMode } from "react";
2. import { createRoot } from "react-dom/client";
3. import "../index.css";
4. import App from "../Component/App/App.jsx";
5. import { CartProvider } from "../Context/cartContext.jsx";
6. import { ThemeProvider } from "../Context/themeContext.jsx";
7. import { AuthProvider } from "../Context/authContext.jsx";
8. import { UserProvider } from "../Context/userContext.jsx";
9. import { CategoryProvider } from "../Context/categoryContext.jsx";
10. // import PwaManager from "../Component/PWA/PwaManager.jsx";
11.
12. // Suppress Chrome extension errors silently
13. try {
14.   const chromeApi = globalThis.chrome;
15.   if (chromeApi?.runtime?.onMessage) {
16.     chromeApi.runtime.onMessage.addListener(() => true);
17.   }
18. } catch {
19.   // Silently ignore chrome API errors
20. }
21.
22. createRoot(document.getElementById("root")).render(
23.   <StrictMode>
24.     <ThemeProvider>
25.       <AuthProvider>
26.         <UserProvider>
27.           <CartProvider>
28.             <CategoryProvider>
29.               <App />
30.               { /* <PwaManager /> */ }
31.             </CategoryProvider>
32.           </CartProvider>
33.         </UserProvider>
34.       </AuthProvider>
35.     </ThemeProvider>
36.   </StrictMode>,
37. );
38.
```

File Name: frontend\src\utility\AddRemoveProductFromWishListButton.jsx

```
1. import axios from "axios";
2. import React, { useEffect, useState } from "react";
3. import { FaHeart, FaRegHeart } from "react-icons/fa";
4. import { customToast } from "../constant";
5. import { useTheme } from "../Context/themeContext";
6.
7. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
8.
9. const AddRemoveProductFromWishListButton = ({
10.   authToken,
11.   productId,
12.   userDetails,
13.   isAddedToWishlist = false, // initial state from product data
14. }) => {
15.   const { theme } = useTheme();
```

```

16. // Initialize from the prop so the heart is filled on first render
17. // if the product is already in the user's wishlist
18. const [isSaved, setIsSaved] = useState(isAddedToWishlist);
19. const [isLoading, setIsLoading] = useState(false);
20.
21. // Sync if the prop changes (e.g. product list re-fetched after
login)
22. useEffect(() => {
23.   setIsSaved(isAddedToWishlist);
24. }, [isAddedToWishlist]);
25.
26. const toggleWishList = async (e) => {
27.   e.preventDefault();
28.   e.stopPropagation();
29.
30.   if (!authToken || !userDetail) return;
31.   if (isLoading) return;
32.
33.   // Optimistic UI update
34.   setIsSaved((prev) => !prev);
35.   setIsLoading(true);
36.
37.   try {
38.     const response = await axios.post(
39.       `${SERVER_URL}/api/user/add-to-wishlist`,
40.       { productId },
41.       {
42.         headers: {
43.           "Content-type": "application/json; charset=UTF-8",
44.           Authorization: `Bearer ${authToken}`,
45.         },
46.       }
47.     );
48.
49.     const { data, status } = response;
50.
51.     if (status === 201) {
52.       // Newly added
53.       setIsSaved(true);
54.       customToast(theme).fire({
55.         title: "Added to Wishlist",
56.         icon: "success",
57.       });
58.     } else if (status === 200) {
59.       // Toggled by backend (removed or re-added)
60.       const isNowSaved = !data?.isRemoved;
61.       setIsSaved(isNowSaved);
62.       customToast(theme).fire({
63.         title: data?.message || (isNowSaved ? "Added to Wishlist" :
"Removed from Wishlist"),
64.         icon: "success",
65.       });
66.     }
67.   } catch (error) {
68.     // Revert optimistic update on failure
69.     setIsSaved((prev) => !prev);
70.     console.log("Wishlist toggle error:", error);
71.   } finally {
72.     setIsLoading(false);
73.   }
74. };

```

```

75.
76.   return (
77.     <span
78.       className={`
79.         h-8 w-8 flex items-center justify-center rounded-full shadow-md
80.         bg-white/90 backdrop-blur-sm
81.         transition-all duration-200
82.         ${isLoading ? "opacity-50 cursor-wait" : "hover:scale-110
active:scale-95 cursor-pointer"}
83.         ${!authToken ? "hidden" : ""}
84.       `}
85.       onClick={toggleWishList}
86.       title={isSaved ? "Remove from Wishlist" : "Add to Wishlist"}
87.     >
88.       {isSaved ? (
89.         <FaHeart className="text-red-500 w-4 h-4" />
90.       ) : (
91.         <FaRegHeart className="text-gray-400 w-4 h-4" />
92.       )}
93.     </span>
94.   );
95. };
96.
97. export default AddRemoveProductFromWishListButton;
98.

```

File Name: frontend\src\utility\AddToCartButton.jsx

```

1. import React, { useState } from "react";
2. import { Button } from "../LIBS";
3. import { useCart } from "../Context/cartContext";
4. import { useNavigate } from "react-router-dom";
5. import Swal from "sweetalert2";
6. import { swalWithCustomConfiguration } from "./constant";
7. import axios from "axios";
8. const SERVER_URL = import.meta.env.VITE_SERVER_URL;
9. const AddToCartButton = ({ userDetails, productId, authToken, className
}) => {
10.   const [isProductAdding, setIsProductAdding] = useState(false);
11.   const { setCartCount } = useCart();
12.   const navigate = useNavigate();
13.
14.   const addToCart = async (e) => {
15.     e.preventDefault();
16.     e.stopPropagation();
17.     if (!productId) {
18.       return;
19.     }
20.
21.     if (!userDetails || !authToken) {
22.       await swalWithCustomConfiguration.fire({
23.         title: "Error!",
24.         text: "Please login first!",
25.         icon: "warning",
26.         // allowOutsideClick: false,
27.       });
28.       return;
29.     }
30.     setIsProductAdding(true);

```



```

31.     try {
32.         const response = await axios.post(
33.             `${SERVER_URL}/api/user/add-to-cart`,
34.             {
35.                 userId: userDetails?.id,
36.                 productId: productId,
37.                 quantity: 1,
38.             },
39.             {
40.                 headers: {
41.                     "Content-type": "application/json; charset=UTF-8",
42.                     Authorization: `Bearer ${authToken}`,
43.                 },
44.             }
45.         );
46.
47.         const { status, data } = response;
48.         if (status === 200) {
49.             setIsProductAdding(false);
50.             setCartCount(() => data?.cartCount);
51.             let shouldNavigateToCart = false;
52.             await swalWithCustomConfiguration.fire({
53.                 title: "Product Added!",
54.                 text: "Your product has been added to the cart. You can
proceed to checkout.",
55.                 icon: "success",
56.                 confirmButtonText: "View Cart",
57.                 cancelButtonText: "Continue Shopping",
58.                 showCancelButton: true,
59.                 reverseButtons: true,
60.                 preConfirm: () => {
61.                     shouldNavigateToCart = true;
62.                 },
63.                 didClose: () => {
64.                     if (shouldNavigateToCart) {
65.                         navigate("/cart");
66.                     }
67.                 },
68.             });
69.         } else return;
70.     } catch (error) {
71.         console?.log(error);
72.         if (error?.message === "Network Error") {
73.             alert("Please check your Internet Connection");
74.             return;
75.         }
76.         const { status, data } = error?.response;
77.         const errorMsg = data?.message || "Something went wrong.";
78.         const errorTitle =
79.             status === 401
80.                 ? "Unauthorized!"
81.                 : status === 404
82.                 ? "Not Found!"
83.                 : status === 500
84.                 ? "Server Error!"
85.                 : "Error!";
86.         await Swal.fire({
87.             title: errorTitle,
88.             text: errorMsg,
89.             icon: "error",
90.         });

```

```

91.     } finally {
92.         setIsProductAdding(false);
93.     }
94. };
95.
96. return (
97.     <Button
98.         btnText={`${isProductAdding ? "Adding..." : "Add to Cart"}}`}
99.         className={className || "btn btn-primary"}
100.        disabled={isProductAdding}
101.        // loading={isProductAdding}
102.        onClick={addToCart}
103.    />
104. );
105. };
106.
107. export default AddToCartButton;
108.

```

File Name: backend/src/app.js

```

1. import express from "express";
2. import bodyParser from "body-parser";
3. import cors from "cors";
4.
5. import productRoutes from "../routes/productRoutes.js";
6. import userRoutes from "../routes/userRoutes.js";
7. import orderRoutes from "../routes/orderRoutes.js";
8. import sellerRoutes from "../routes/sellerRoutes.js";
9. import authRoutes from "../routes/authRoute.js";
10. import adminRoutes from "../routes/adminRoutes.js";
11. import categoryRoutes from "../routes/categoryRoutes.js";
12. import newsletterRoutes from "../routes/newsletterRoutes.js";
13.
14. const app = express();
15. const allowedOrigins = ["http://localhost:3000",
    "http://localhost:3001", "http://localhost:5173",
    process.env.CORS_ORIGIN_URL];
16. app.use(
17.     cors({
18.         origin: function (origin, callback) {
19.             // Allow requests with no origin (like mobile apps or curl
            requests)
20.             if (!origin) return callback(null, true);
21.             if (allowedOrigins.includes(origin) ||
                allowedOrigins.includes("*")) {
22.                 return callback(null, true);
23.             }
24.             return callback(new Error("CORS policy violation"), false);
25.         },
26.         methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
27.         credentials: true,
28.     })
29. );
30. app.options("*", cors());
31. // app.use(express.json());
32. app.use(express.urlencoded({ extended: true }));
33. app.use(bodyParser.json());
34. app.use("/api/product", productRoutes);

```

```
35. app.use("/api/user", userRoutes);
36. app.use("/api/order", orderRoutes);
37. app.use("/api/seller", sellerRoutes);
38. app.use("/api/auth", authRoutes);
39. app.use("/api/admin", adminRoutes);
40. app.use("/api/category", categoryRoutes);
41. app.use("/api/newsletter", newsletterRoutes);
42.
43. app.get("/", function (req, res) {
44.   res.send("Hello, bro");
45. });
46.
47. export default app;
48.
```

File Name: backend\src\config\db-connect.js

```
1. import mongoose from "mongoose";
2.
3. let cachedConnection = null;
4. let cachedPromise = null;
5.
6. mongoose.set("strictQuery", true);
7.
8. const getMongoUri = () => {
9.   const DB_URL = process.env.DB_URL;
10.  const DB_NAME = process.env.DB_NAME;
11.
12.  if (!DB_URL) {
13.    throw new Error("DB_URL is not configured");
14.  }
15.
16.  return DB_NAME ? `${DB_URL}/${DB_NAME}` : DB_URL;
17. };
18.
19. const connectDB = async () => {
20.  if (cachedConnection && mongoose.connection.readyState === 1) {
21.    return cachedConnection;
22.  }
23.
24.  if (cachedPromise) {
25.    return cachedPromise;
26.  }
27.
28.  const mongoUri = getMongoUri();
29.
30.  cachedPromise = mongoose
31.    .connect(mongoUri, {
32.      serverSelectionTimeoutMS: 10000,
33.    })
34.    .then((mongooseInstance) => {
35.      cachedConnection = mongooseInstance;
36.      console.log("Database connected...");
37.      return mongooseInstance;
38.    })
39.    .catch((error) => {
40.      cachedPromise = null;
41.      cachedConnection = null;
42.      console.error("Database connection failed:", error.message);
```

```

43.         throw error;
44.     });
45.
46.     return cachedPromise;
47. };
48.
49. export default connectDB;
50.

```

File Name: backend\src\controllers\adminController\getProducts.js

```

1. import Product from "../../model/productSchema.js";
2. import Category from "../../model/categorySchema.js";
3. import SellerUser from "../../model/sellerUserSchema.js";
4.
5. const getProducts = async (req, res) => {
6.     try {
7.         const page = parseInt(req.query.page) || 1;
8.         const limit = parseInt(req.query.limit) || 10;
9.         const skip = (page - 1) * limit;
10.
11.         const query = {};
12.         if (req.query.search) {
13.             query.$or = [
14.                 { name: { $regex: req.query.search, $options: "i" } },
15.                 { description: { $regex: req.query.search, $options: "i" } }
16.             ];
17.         }
18.
19.         const totalProducts = await Product.countDocuments(query);
20.         const products = await Product.find(query)
21.             .populate("category subCategory")
22.             .populate({
23.                 path: "sellerId",
24.                 model: SellerUser,
25.                 select: "businessName emailAddress"
26.             })
27.             .sort({ createdAt: -1, _id: -1 })
28.             .skip(skip)
29.             .limit(limit);
30.
31.         res.status(200).json({
32.             success: true,
33.             products,
34.             totalPages: Math.ceil(totalProducts / limit),
35.             currentPage: page,
36.             totalProducts
37.         });
38.     } catch (error) {
39.         res.status(500).json({ success: false, message: "Server error",
40.             error: error.message });
41.     }
42. };
43. export default getProducts;
44.

```

File Name: backend\src\controllers\authController\forgetPassword.js

```

1. import User from "../../model/userSchema.js";
2. import sendOtpEmail from "../../services/otpService.js";
3. const forgetPassword = async (req, res, next) => {
4.   try {
5.     const { email } = req?.body;
6.     const existingUser = await User.findOne({ email });
7.     if (!existingUser) {
8.       return res
9.         .status(404)
10.        .json({ success: false, message: "You are not a Registered
User!" });
11.     }
12.     const otp = await sendOtpEmail(email, "Your OTP for Password
Reset");
13.
14.     if (otp) {
15.       existingUser.otp = otp;
16.       existingUser.otpExpiresAt = new Date(Date.now() + 10 * 60 * 1000);
17.       existingUser.save();
18.       return res.status(200).json({
19.         success: true,
20.         message:
21.           "Your OTP has been sent! Kindly check your inbox or spam
folder.",
22.       });
23.     }
24.     return res.status(500).json({
25.       success: false,
26.       message: "Unable to send OTP. Please try again shortly.",
27.     });
28.   } catch (error) {
29.     res
30.       .status(500)
31.       .json({ success: false, message: "Error sending email", error });
32.   }
33. };
34.
35. export default forgetPassword;
36.

```

File Name: backend\src\controllers\authController\googleAuth.js

```

1. import { google } from "googleapis";
2.
3. /**
4.  * GET /api/auth/google
5.  * Creates a fresh oauth2Client per request and returns the Google OAuth
URL.
6.  */
7. const googleAuth = (req, res) => {
8.   try {
9.     // ☒ New client per request – safe for serverless
10.    const oauth2Client = new google.auth.OAuth2(
11.      process.env.GOOGLE_CLIENT_ID,
12.      process.env.GOOGLE_CLIENT_SECRET,
13.      process.env.GOOGLE_CALLBACK_URL
14.    );

```

```

15.
16.     const scopes = [
17.         "https://www.googleapis.com/auth/userinfo.profile",
18.         "https://www.googleapis.com/auth/userinfo.email",
19.     ];
20.
21.     const authUrl = oauth2Client.generateAuthUrl({
22.         access_type: "offline",
23.         scope: scopes,
24.         prompt: "select_account",
25.     });
26.
27.     return res.status(200).json({ success: true, url: authUrl });
28. } catch (error) {
29.     console.error("Google Auth URL Error:", error);
30.     return res
31.         .status(500)
32.         .json({ success: false, message: "Failed to generate Google login
URL" });
33. }
34. };
35.
36. export default googleAuth;
37.

```

File Name: backend/src/controllers/authController/googleCallback.js

```

1. import { google } from "googleapis";
2. import User from "../../model/userSchema.js";
3.
4. /**
5.  * POST /api/auth/google/callback
6.  * Creates a FRESH oauth2Client per request to avoid shared-state
7.  * issues on Vercel serverless cold starts.
8.  */
9. const googleCallback = async (req, res) => {
10.     const { code } = req.body;
11.
12.     if (!code) {
13.         return res
14.             .status(400)
15.             .json({ success: false, message: "Authorization code is
required." });
16.     }
17.
18.     try {
19.         // ☒ New client per request – safe for serverless
20.         const oauth2Client = new google.auth.OAuth2(
21.             process.env.GOOGLE_CLIENT_ID,
22.             process.env.GOOGLE_CLIENT_SECRET,
23.             process.env.GOOGLE_CALLBACK_URL
24.         );
25.
26.         // 1. Exchange code for tokens
27.         const { tokens } = await oauth2Client.getToken(code);
28.         oauth2Client.setCredentials(tokens);
29.
30.         // 2. Fetch user info from Google

```

```

31.     const oauth2 = google.oauth2({ version: "v2", auth: oauth2Client
32. });
33.     const { data: googleUser } = await oauth2.userinfo.get();
34.     const { id: googleId, email, name, picture } = googleUser;
35.
36.     if (!email) {
37.         return res.status(400).json({
38.             success: false,
39.             message: "Could not retrieve email from Google. Please try
again.",
40.         });
41.     }
42.
43.     // 3. Find existing user by email
44.     let user = await User.findOne({ email: email.trim().toLowerCase()
45. });
46.     if (user) {
47.         // — Existing user —
48.         if (user.status === "blocked") {
49.             return res.status(403).json({
50.                 success: false,
51.                 message: "Your account has been suspended by the
administrator.",
52.             });
53.         }
54.
55.         // Link googleId if they previously registered with
email/password
56.         if (!user.googleId) {
57.             user.googleId = googleId;
58.             await user.save();
59.         }
60.     } else {
61.         // — New user — register via Google —
62.         user = new User({
63.             name: name || email.split("@")[0],
64.             email: email.trim().toLowerCase(),
65.             googleId,
66.             profilePic: picture || null,
67.             isVerified: true,
68.             hasPassword: false,
69.             TandC: true,
70.         });
71.         await user.save();
72.     }
73.
74.     // 4. Generate JWT and respond
75.     const authToken = user.generateToken();
76.     return res.status(200).json({
77.         success: true,
78.         message: "Google login successful.",
79.         AuthToken: authToken,
80.     });
81. } catch (error) {
82.     console.error("Google Callback Error:", error);
83.
84.     if (error.code === 11000) {
85.         return res.status(400).json({
86.             success: false,

```

```

87.         message: "An account with this email already exists.",
88.     });
89. }
90.
91.     return res.status(500).json({
92.         success: false,
93.         message: "Google authentication failed. Please try again.",
94.         error: error.message,
95.     });
96. }
97. };
98.
99. export default googleCallback;
100.

```

File Name: backend\src\controllers\authController\login.js

```

1. import sendOtpEmail from "../services/otpService.js";
2. import User from "../model/userSchema.js";
3.
4. const login = async (req, res, next) => {
5.     const { email, password } = req.body;
6.     // Validate email and password
7.     try {
8.         if (!email || !password) {
9.             return res
10.                .status(400)
11.                .json({ success: false, message: "All fields are required" });
12.         }
13.
14.         const existingUser = await User.findOne({ email: email?.trim() });
15.         if (!existingUser) {
16.             return res
17.                .status(404)
18.                .json({ success: false, message: "User Not Found" });
19.         }
20.
21.         // --- ADMIN ENFORCEMENT: Block Suspended Users Instantly ---
22.         if (existingUser.status === "blocked") {
23.             return res.status(403).json({
24.                 success: false,
25.                 message: "Your account has been suspended by the
administrator.",
26.             });
27.         }
28.         // -----
29.
30.
31.         // --- GOOGLE-ONLY ACCOUNT GUARD ---
32.         // If the user signed up via Google and has never set a password
33.         if (!existingUser.password || existingUser.hasPassword === false) {
34.             return res.status(400).json({
35.                 success: false,
36.                 message:
37.                     "This account uses Google Sign-In. Please log in with Google,
or set a password from your Profile settings.",
38.             });
39.         }
40.         // -----

```



```

41.
42.     const validPassword = await existingUser.passwordCompare(password);
43.     if (!validPassword) {
44.         return res
45.             .status(401)
46.             .json({ success: false, message: "Invalid Credentials" });
47.     }
48.     if (!existingUser?.isVerified) {
49.         const hashedOtp = await sendOtpEmail(
50.             existingUser?.email,
51.             "Your Registration OTP"
52.         );
53.
54.         if (!hashedOtp) {
55.             return res.status(500).json({
56.                 success: false,
57.                 message: "Failed to send OTP. Please try again.",
58.             });
59.         }
60.
61.         const otpExpires = new Date(Date.now() + 10 * 60 * 1000);
62.
63.         // console.log("=== LOGIN OTP GENERATION ===");
64.         // console.log("OTP Expiry Set To:", otpExpires);
65.         // console.log("Current Time:", new Date());
66.         // console.log("=====");
67.
68.         existingUser.otp = hashedOtp;
69.         existingUser.otpExpireAt = otpExpires;
70.         await existingUser.save();
71.
72.         return res.status(403).json({
73.             success: false,
74.             message: "Email not verified. OTP sent to your email!",
75.             requiresVerification: true,
76.             email: existingUser.email,
77.         });
78.     }
79.     return res.status(200).json({
80.         success: true,
81.         message: "User logged in successfully",
82.         AuthToken: await existingUser.generateToken(),
83.     });
84. } catch (err) {
85.     res?.status(500).json({
86.         success: false,
87.         message: "Internal Server Error",
88.         error: err.message,
89.         rowError: err,
90.     });
91. }
92. };
93. export default login;
94.

```

File Name: backend\src\controllers\authController\register.js

```

1. import sendOtpEmail from "../../services/otpService.js";
2. import User from "../../model/userSchema.js";

```

```

3. import bcrypt from "bcrypt";
4. const register = async (req, res) => {
5.   const { name, email, password, TandC, confirmPassword, phoneNumber } =
6.     req.body;
7.   try {
8.     if (!name || !email || !password || !confirmPassword ||
!phoneNumber) {
9.       return res
10.        .status(400)
11.        .json({ success: false, message: "All fields are required" });
12.     }
13.     const existingUser = await User.findOne({ email });
14.     if (existingUser)
15.       return res.status(400).json({
16.         success: false,
17.         message: "You're already registered. Try logging in.",
18.       });
19.     if (password !== confirmPassword)
20.       return res?.status(400)?.json({
21.         success: false,
22.         message: "Both password and confirm password must be same",
23.       });
24.     const hashedOtp = await sendOtpEmail(email, "Verify Your
Registration OTP");
25.     const otpExpireAt = new Date(Date.now() + 10 * 60 * 1000);
26.
27.     if (!hashedOtp) {
28.       return res?.status(500)?.json({
29.         success: false,
30.         message: "Failed to send OTP. Please try again later.",
31.       });
32.     }
33.
34.     // console.log("=== REGISTRATION OTP GENERATION ===");
35.     // console.log("OTP Expiry Set To:", otpExpireAt);
36.     // console.log("Current Time:", new Date());
37.     // console.log("=====");
38.
39.     const newUser = new User({
40.       name: name.trim(),
41.       email: email.trim(),
42.       password: password,
43.       TandC: TandC || false,
44.       otp: hashedOtp,
45.       otpExpireAt,
46.       phoneNumber: phoneNumber,
47.     });
48.     await newUser.save();
49.
50.     res.status(200).json({
51.       success: true,
52.       message:
53.         "Your OTP has been sent! Kindly check your inbox or spam
folder.",
54.       email,
55.     });
56.   } catch (error) {
57.     if (error.code === 11000) {
58.       const field = Object?.keys(error?.keyPattern)[0];
59.       const value = error?.keyValue[field];
60.       return res.status(400).json({

```

```

61.         success: false,
62.         message: `${field} ${value} already exists.`,
63.     });
64.     }
65.     console.error(error);
66.     return res.status(500).json({
67.         success: false,
68.         message: "Internal server error. Please try again later.",
69.     });
70.     }
71. };
72.
73. export default register;
74.

```

File Name: backend\src\controllers\authController\resendOtp.js

```

1. import sendOtpEmail from "../../services/otpService.js";
2. import User from "../../model/userSchema.js";
3.
4. const resendOtp = async (req, res) => {
5.     const { email } = req.body;
6.
7.     try {
8.         if (!email) {
9.             return res.status(400).json({
10.                 success: false,
11.                 message: "Email is required",
12.             });
13.         }
14.
15.         const user = await User.findOne({ email: email?.trim() });
16.
17.         if (!user) {
18.             return res.status(404).json({
19.                 success: false,
20.                 message: "User not found",
21.             });
22.         }
23.
24.         if (user.isVerified) {
25.             return res.status(400).json({
26.                 success: false,
27.                 message: "Email is already verified",
28.             });
29.         }
30.
31.         // Generate new OTP
32.         const hashedOtp = await sendOtpEmail(email, "Verify Your
Registration OTP");
33.
34.         if (!hashedOtp) {
35.             return res.status(500).json({
36.                 success: false,
37.                 message: "Failed to send OTP. Please try again later.",
38.             });
39.         }
40.
41.         const otpExpireAt = new Date(Date.now() + 10 * 60 * 1000);

```

```

42.
43.     // console.log("=== RESEND OTP ===");
44.     // console.log("Email:", email);
45.     // console.log("OTP Expiry Set To:", otpExpireAt);
46.     // console.log("Current Time:", new Date());
47.     // console.log("=====");
48.
49.     user.otp = hashedOtp;
50.     user.otpExpireAt = otpExpireAt;
51.     await user.save();
52.
53.     res.status(200).json({
54.         success: true,
55.         message: "OTP has been resent! Please check your email.",
56.         email,
57.     });
58. } catch (error) {
59.     console.error("Resend OTP Error:", error);
60.     res.status(500).json({
61.         success: false,
62.         message: "Internal server error. Please try again later.",
63.         error: error.message,
64.     });
65. }
66. };
67.
68. export default resendOtp;
69.

```

File Name: backend\src\controllers\authController\resetPassword.js

```

1. import sendEmail from "../../services/emailService.js";
2. import User from "../../model/userSchema.js";
3. import jwt from "jsonwebtoken";
4. import bcrypt from "bcrypt";
5.
6. const resetPassword = async (req, res) => {
7.     try {
8.         const { resetToken, newPassword } = req.body;
9.
10.         if (!resetToken || !newPassword) {
11.             return res.status(400).json({
12.                 success: false,
13.                 message: "Token and new password are required!",
14.             });
15.         }
16.
17.         let decoded = jwt.verify(resetToken, process.env.JWT_SECRET_KEY);
18.         if (!decoded) {
19.             return res
20.                 .status(400)
21.                 .json({ success: false, message: "Invalid or expired token!"
22.             });
23.         }
24.         const user = await User.findOne({ email: decoded.email });
25.
26.         if (!user) {
27.             return res

```

```

28.         .status(404)
29.         .json({ success: false, message: "You are not a Registered
User!" });
30.     }
31.     user.password = newPassword;
32.     user.otp = null;
33.     user.expireAt = null;
34.     await user.save();
35.     const htmlContent = `
36.         <div style="font-family: Arial, sans-serif; text-align: center;
background-color: #f4f4f4; padding: 20px;">
37.             <div style="max-width: 600px; margin: auto; background:
#ffffff; padding: 20px; border-radius: 8px; box-shadow: 0 2px 10px rgba(0,
0, 0, 0.1);">
38.                 <h2 style="background: #007bff; color: #ffffff; padding:
15px; border-radius: 8px 8px 0 0;">Password Reset Successful</h2>
39.                 <p>Hello <strong>${user.name || user.email}</strong></p>
40.                 <p>Your password has been successfully changed.</p>
41.                 <p>If you did not make this request, please contact our
support team immediately.</p>
42.                 <a href="https://browsemart.vercel.app/login"
style="display: inline-block; padding: 10px 20px; background-color:
#007bff; color: #ffffff; text-decoration: none; border-radius: 5px; font-
size: 16px; margin-top: 20px;">Login to Your Account</a>
43.                 <p style="margin-top: 20px; font-size: 14px; color:
#666;">Thank you,<br> The BrowseMart Team</p>
44.             </div>
45.         </div>
46.     `;
47.     await sendEmail(user?.email, "Your Password Has Been Changed",
htmlContent);
48.     res.status(200).json({
49.         success: true,
50.         message:
51.             "Password reset successful! You can now log in with your new
password.",
52.     });
53. } catch (error) {
54.     console.error("Password Reset Error:", error);
55.     res
56.         .status(500)
57.         .json({ success: false, message: "Error resetting password" });
58. }
59. };
60.
61. export default resetPassword;
62.

```

File Name: backend\src\controllers\authController\verifyOtp.js

```

1. import User from "../../model/userSchema.js";
2. import jwt from "jsonwebtoken";
3. const verifyOtp = async (req, res) => {
4.     try {
5.         const foundedUser = req?.user;
6.         // const { email, otp } = req.body;
7.
8.         // if (!email || !otp) {
9.             //     return res

```

```

10.      //      .status(400)
11.      //      .json({ success: false, message: "Email and OTP are
required!" });
12.      // }
13.
14.      // const user = await User.findOne({ email });
15.
16.      // if (!user) {
17.      //   return res
18.      //     .status(404)
19.      //     .json({ success: false, message: "You are not a Registered
User!" });
20.      // }
21.
22.      // if (user.otp?.toString() !== otp) {
23.      //   return res
24.      //     .status(400)
25.      //     .json({ success: false, message: "Invalid OTP! Please try
again." });
26.      // }
27.
28.      // if (new Date() > user.otpExpireAt) {
29.      //   return res.status(400).json({
30.      //     success: false,
31.      //     message: "OTP has expired! Request a new one.",
32.      //   });
33.      // }
34.
35.      const resetToken = jwt.sign(
36.        { email: foundedUser?.email, isOtpVerified: true },
37.        process.env.JWT_SECRET_KEY,
38.        { expiresIn: "10m" }
39.      );
40.      res.status(200).json({
41.        success: true,
42.        message: "OTP verified successfully!",
43.        resetToken,
44.      });
45.    } catch (error) {
46.      res.status(500).json({ message: "Error verifying OTP", error });
47.    }
48.  };
49.
50. export default verifyOtp;
51.

```

File Name: backend\src\controllers\authController\verifyOTPForRegistration.js

```

1. import User from "../../model/userSchema.js";
2. const verifyOTPForRegistration = async (req, res) => {
3.   try {
4.     const foundedUser = req?.user;
5.     // const user = await User.findOne({ email });
6.     // if (!user) return res.status(400).json({ message: "User not
found" });
7.
8.     // if (user.otp !== otp || new Date() > user.otpExpireAt) {
9.     //   return res.status(400).json({
10.      //     success: false,

```

```

11.      //      message: "Invalid or Expired OTP! Please try again.",
12.      //    });
13.      //  }
14.      // // ✓ OTP is correct, so clear OTP fields
15.      // user.otp = undefined;
16.      // user.otpExpireAt = undefined;
17.      // user.isVerified = true;
18.      // await user.save();
19.      // ✓ Generate JWT Token
20.      const authToken = foundedUser?.isVerified
21.        ? await foundedUser?.generateToken()
22.        : null;
23.
24.      res
25.        .status(201)
26.        .json({ message: "Account successfully registered", authToken });
27.    } catch (error) {
28.      console.error(error);
29.      res.status(500).json({ message: "Server error" });
30.    }
31.  };
32.
33. export default verifyOTPForRegistration;
34.

```

File Name: backend\src\controllers\orderController\getAllOrderByUserId.js

```

1. import Order from "../../model/orderSchema.js";
2. import Product from "../../model/productSchema.js";
3.
4. const getAllOrderByUserId = async (req, res, next) => {
5.   try {
6.     const activeUser = req?.user;
7.     const { page = 1, limit = 5 } = req?.query;
8.
9.     // Validate input data
10.    if (!activeUser) {
11.      return res.status(400).json({ message: "Invalid user or order
data" });
12.    }
13.
14.    const parsedPage = Number(page) > 0 ? Number(page) : 1;
15.    const parsedLimit = Number(limit) > 0 ? Number(limit) : 5;
16.    const skip = (parsedPage - 1) * parsedLimit;
17.
18.
19.    const query = { customerId: activeUser?._id };
20.    const totalCount = await Order.countDocuments(query);
21.    const totalPages = Math.ceil(totalCount / parsedLimit);
22.
23.    const paginatedOrders = await Order.find(query)
24.      .sort({ createdAt: -1, _id: -1 })
25.      .skip(skip)
26.      .limit(parsedLimit)
27.      .lean();
28.
29.    // Fetch and process current page orders
30.    const ordersArr = await Promise.all(
31.      paginatedOrders?.map(async (filteredOrderArr) => {

```

```

32.         try {
33.             if (!filteredOrderArr) return null;
34.
35.             // Process each order
36.             const orderDetails = {
37.                 orderStatus: filteredOrderArr?.orderStatus,
38.                 id: filteredOrderArr?._id,
39.                 orderId: filteredOrderArr?.orderId,
40.                 orderDate: filteredOrderArr?.orderDate,
41.                 orderItems: await Promise.all(
42.                     filteredOrderArr?.orderItems?.map(async (item) => {
43.                         const productData = await
Product?.findById(item?.productId);
44.                         const resolvedPrice = Number(
45.                             item?.sellingPrice ?? item?.price ?? item?.mrpPrice
?? 0,
46.                             );
47.                         const resolvedQuantity = Number(item?.quantity ?? 0);
48.
49.                         return {
50.                             ...item,
51.                             quantity: resolvedQuantity,
52.                             price: resolvedPrice,
53.                             sellingPrice: Number(item?.sellingPrice ??
resolvedPrice),
54.                             mrpPrice: Number(item?.mrpPrice ?? resolvedPrice),
55.                             productImage: productData?.image?.[0],
56.                             productDescription: productData?.description,
57.                         };
58.                     })),
59.                 ),
60.                 totalAmount:
61.                     filteredOrderArr?.totalAmount ||
62.                     filteredOrderArr?.grandTotal ||
63.                     0,
64.             };
65.
66.             return orderDetails;
67.         } catch (err) {
68.             console.error("Error processing order item:", err);
69.             return null;
70.         }
71.     })),
72. );
73.
74. // Filter out any null values in case of missing orders or errors
75. const validOrders = ordersArr.filter((order) => order !== null);
76. const start = totalCount === 0 ? 0 : skip + 1;
77. const end = Math.min(skip + validOrders.length, totalCount);
78.
79. // Return the processed orders
80. res.status(200).json({
81.     message: "All orders fetched successfully!",
82.     ordersArr: validOrders,
83.     totalOrders: totalCount,
84.     page: parsedPage,
85.     totalPages,
86.     startOrderIndex: start,
87.     endOrderIndex: end,
88. });
89. } catch (err) {

```



```

90.     console.error("Error fetching all orders:", err);
91.     res.status(500).json({ message: "Server error while fetching
orders" });
92.   }
93. };
94.
95. export default getAllOrderByUserId;
96.

```

File Name: backend\src\controllers\orderController\getOrderById.js

```

1. import Order from "../../model/orderSchema.js";
2. import Product from "../../model/productSchema.js";
3. import User from "../../model/userSchema.js";
4. const getOrderById = async (req, res) => {
5.   try {
6.     const orderId = req?.body?.orderId;
7.     if (!orderId) {
8.       return res.status(400).json({ msg: "No orderId provided" });
9.     }
10.
11.     const orderDetails = await Order?.findOne({ orderId: orderId });
12.
13.     if (orderDetails?.customerId?.toString() !==
req?.user?._id?.toString()) {
14.       return res.status(403).json({
15.         success: false,
16.         message: "You are not authorized to view this order",
17.       });
18.     }
19.     if (!orderDetails) {
20.       return res.status(404).json({ msg: "Order not found" });
21.     }
22.     const customerDetails = await
User?.findById(orderDetails?.customerId);
23.
24.     // Resolve promises for all orderItems
25.     const resolvedOrderItems = await Promise.all(
26.       orderDetails?.orderItems?.map(async (orderItem) => {
27.         const itemDetails = await Product?.findById(
28.           orderItem?.productId || orderItem?.id || orderItem?._id
29.         );
30.         return {
31.           ...orderItem?._doc,
32.           productImage: itemDetails?.image?.[0],
33.           productDescription: itemDetails?.description,
34.         };
35.       })
36.     );
37.
38.     const filteredOrderDetails = {
39.       ...orderDetails?._doc,
40.       orderItems: resolvedOrderItems,
41.       customerDetail: {
42.         customerName: customerDetails?.name,
43.         customerEmail: customerDetails?.email,
44.         customerPhoneNumber: customerDetails?.phoneNumber,
45.         customerAddress: customerDetails?.address,
46.       },

```

```

47.     };
48.
49.     res
50.         .status(200)
51.         .json({ message: "Order fetched successfully!",
filteredOrderDetails });
52.     } catch (err) {
53.         console.error(err);
54.         res.status(500).send("Server Error");
55.     }
56. };
57.
58. export default getOrderById;
59.

```

File Name: backend\src\controllers\orderController\index.js

```

1. export { default as submitOrder } from "./submitOrder";
2.

```

File Name: backend\src\controllers\orderController\submitOrder.js

```

1. import Order from "../../model/orderSchema.js";
2. import Product from "../../model/productSchema.js";
3.
4. import { generateOrderId } from "../../utility/constraint.js";
5. import sendOrderConfirmationEmail from
"./../services/sendOrderConfirmationEmail.js";
6.
7. const submitOrder = async (req, res, next) => {
8.     try {
9.         const activeUser = req.user;
10.        const productCard = req?.body?.cartProduct;
11.        const shippingAddress = req?.body?.shippingAddress;
12.        const { methodName, methodDetail } = req?.body?.paymentData;
13.        const orderIds = [];
14.
15.        const orderDate = new Date(); // UTC timestamp from server
16.        const saveDefaultShippingAddress = (shippingAddress, activeUser) =>
{
17.            try {
18.                if (!activeUser) return;
19.                if (!shippingAddress?.isDefaultShippingAddress) return;
20.                if (!shippingAddress) return "Shipping Address is required";
21.                const normalizedAddress = {
22.                    addressLine1: shippingAddress?.addressLine1?.trim(),
23.                    addressLine2:
24.                        shippingAddress?.addressLine2?.trim() ||
25.                        shippingAddress?.addressLine2?.trim() ||
26.                        "",
27.                    state: shippingAddress?.state?.trim(),
28.                    city: shippingAddress?.city?.trim(),
29.                    pinCode: shippingAddress?.pinCode?.trim(),
30.                    country: shippingAddress?.country?.trim(),
31.                };
32.                activeUser.shippingAddress = normalizedAddress;
33.                return true;

```

```

34.     } catch (error) {
35.         console.error(" line 29", error);
36.     }
37. };
38. // decrease product quantity
39. const decreaseProductQuantity = async (productId, quantity) => {
40.     try {
41.         const product = await Product.findById(productId);
42.
43.         if (!product) throw new Error("Product not found");
44.
45.         const quantityToOrder = Math?.min(product?.stock, quantity);
46.         product.stock -= quantityToOrder;
47.         await product.save();
48.         return quantityToOrder;
49.     } catch (error) {
50.         console.error("Line No 58", error.message);
51.     }
52. };
53.
54. const filteredProduct = await Promise.all(
55.     productCard?.map(async (item) => {
56.         const orderedQuantity = await decreaseProductQuantity(
57.             item?.id || item?._id,
58.             item?.quantity
59.         );
60.         console.log(" Line No 120orderedQuantity ", orderedQuantity);
61.         if (orderedQuantity > 0)
62.             return {
63.                 productId: item?.id || item?._id,
64.                 productName: item?.name,
65.                 price: item?.price || item?.sellingPrice,
66.                 mrpPrice: item?.mrpPrice || null,
67.                 sellingPrice: item?.sellingPrice || null,
68.                 sellerId: item?.userId || item?.userID || item?.sellerId,
69.                 quantity: orderedQuantity, // resolved quantity from
decreaseProductQuantity
70.             };
71.         })
72. );
73. const isDefaultShippingAddressSaved = saveDefaultShippingAddress(
74.     shippingAddress,
75.     activeUser
76. );
77. if (isDefaultShippingAddressSaved === false) {
78.     return res
79.         .status(400)
80.         .json({ message: "Failed to save default shipping address" });
81. }
82.
83. const groupProduct = filteredProduct?.reduce((acc, item) => {
84.     const sellerId = item?.sellerId;
85.     if (!acc[sellerId]) {
86.         acc[sellerId] = [];
87.     }
88.     acc[sellerId].push(item);
89.     return acc;
90. }, {});
91.
92. for (const sellerId in groupProduct) {
93.     const filteredProduct = groupProduct[sellerId];

```

```

94.      // console.log("Active user", activeUser);
95.      if (filteredProduct[0]?.quantity === 0 || !filteredProduct[0]) {
96.          console.log("Line No 158 No products available for order");
97.          continue;
98.      }
99.      const totalMrpPrice = filteredProduct.reduce((total, product) =>
100.      {
101.          return (
102.              total + (product?.mrpPrice || product?.price) *
103.              product?.quantity
104.          );
105.      }, 0);
106.      const totalSellingPrice = filteredProduct.reduce((total,
107.      product) => {
108.          return (
109.              total + (product?.sellingPrice || product?.price) *
110.              product?.quantity
111.          );
112.      }, 0);
113.      const totalDiscount = totalMrpPrice - totalSellingPrice;
114.      const shippingCharges =
115.          totalSellingPrice < 1000 ? totalSellingPrice * 0.1 : 0;
116.
117.      const order = new Order({
118.          orderId: generateOrderId(orderDate.toLocaleString()),
119.          customerId: activeUser?._id,
120.          orderItems: filteredProduct,
121.          shippingAddress: {
122.              addressLine1: shippingAddress?.addressLine1?.trim(),
123.              addressLine2: shippingAddress?.addressLine2?.trim() || "",
124.              state: shippingAddress?.state?.trim(),
125.              city: shippingAddress?.city?.trim(),
126.              pinCode: shippingAddress?.pinCode?.trim(),
127.              country: shippingAddress?.country?.trim(),
128.          },
129.          paymentMethod: methodName,
130.          paymentStatus: methodName?.toString() !== "cod" ? "success" :
131.          "pending",
132.          paymentMethodDetail: { ...methodDetail },
133.          // totalAmount: filteredProduct?.reduce((totalAmount, product)
134.          // => {
135.          //     return product?.quantity > 0
136.          //         ? totalAmount +
137.          //             (product?.price || product?.sellingPrice) *
138.          //             product?.quantity
139.          //         : totalAmount;
140.          // }, 0),
141.          orderDate: orderDate,
142.          sellerId: sellerId,
143.          totalMrpPrice: totalMrpPrice,
144.          totalSellingPrice: totalSellingPrice,
145.          totalDiscount: totalDiscount,
146.          grandTotal: totalSellingPrice + shippingCharges,
147.          shippingCharge: shippingCharges,
148.      });
149.      await order.save();
150.      const { _id, orderId } = order;
151.      orderIds?.push(orderId);
152.      activeUser.order.push({
153.          orderId: _id,
154.      });

```

```

148.         console.log("Line No 193", order);
149.         await sendOrderConfirmationEmail(activeUser?.email, order);
150.     }
151.
152.     // console.log("Line No 193", orderIds);
153.     if (!orderIds.length > 0) {
154.         return res.status(400).json({ message: "Failed to submit order"
155.     });
156.     }
157.     await activeUser.save();
158.
159.     res?.status(201)?.json({
160.         message: "Order submitted successfully",
161.         orderIds,
162.         ...(isDefaultShippingAddressSaved === true && {
163.             updatedShippingAddress: activeUser.shippingAddress,
164.         }),
165.     });
166.     } catch (error) {
167.         console.error("Line No 203", error);
168.         res.status(500).json({ message: "Server Error" });
169.     }
170.
171. export default submitOrder;
172.

```

File Name: backend\src\controllers\productController\addProduct.js

```

1. import Product from "../..model/productSchema.js";
2. import { uploadImageToCloudinary } from "../..utility/cloudinary.js";
3.
4. const addProduct = async (req, res) => {
5.     try {
6.         const {
7.             name,
8.             sellingPrice,
9.             mrpPrice,
10.            description,
11.            category,
12.            stock,
13.            brand,
14.            subCategory,
15.        } = req.body;
16.
17.
18.        const foundedUser = req.user;
19.        if (foundedUser?.userType !== "seller") {
20.            return res.status(401).json({
21.                success: false,
22.                message: "unauthorized access you don't have seller account",
23.            });
24.        }
25.
26.        // `req.files` will be populated by the multer array middleware
27.        const files = req.files;
28.
29.        if (
30.            !name ||

```

```

31.         !mrpPrice ||
32.         !sellingPrice ||
33.         !description ||
34.         !category ||
35.         !stock ||
36.         !brand ||
37.         !files ||
38.         files.length === 0
39.     ) {
40.         return res.status(400).json({
41.             success: false,
42.             message: "Please fill all fields and upload at least one
image.",
43.         });
44.     }
45.
46.     // Process all incoming memory buffers to Cloudinary securely in
parallel
47.     const uploadPromises = files.map((file) =>
uploadImageToCloudinary(file.buffer));
48.     const imageUrls = await Promise.all(uploadPromises);
49.
50.     const newProduct = new Product({
51.         name: name,
52.         mrpPrice: Number(mrpPrice),
53.         sellingPrice: Number(sellingPrice),
54.         description: description,
55.         image: imageUrls,
56.         category: category,
57.         stock: Number(stock),
58.         sellerId: foundedUser?.sellerId,
59.         brand: brand,
60.         subCategory: subCategory || "",
61.     });
62.
63.     await newProduct.save();
64.
65.     return res.status(201).json({
66.         success: true,
67.         message: "Data Saved and Uploaded Sucessfully!",
68.     });
69. } catch (error) {
70.     console.error("Add Product Error:", error);
71.     res?.status(500)?.json({
72.         success: false,
73.         message: "Error in uploading product, Please try again later.",
74.     });
75. }
76. };
77.
78. export default addProduct;
79.

```

File Name: backend\src\controllers\productController\getAllProduct.js

```

1. import Product from "../../model/productSchema.js";
2. import User from "../../model/userSchema.js";
3. import Category from "../../model/categorySchema.js";
4. import mongoose from "mongoose";

```

```

5. const getAllProduct = async (req, res) => {
6.   try {
7.     const {
8.       activeUserId,
9.       searchQuery,
10.      searchCategory,
11.      page = 1,
12.      limit = 24,
13.    } = req?.query;
14.    const activeUser = activeUserId ? await User.findById(activeUserId)
: null;
15.    // const isAddedToWislist = await activeUser?.wishlist?.some(
16.      //   (item) => item?.productId?.toString() ===
"670c1ca6dc874450c9d49338"
17.    // );
18.    // console.log("line 9", isAddedToWislist);
19.    const parsedPage = Number(page) > 0 ? Number(page) : 1;
20.    const parsedLimit = Number(limit) > 0 ? Number(limit) : 24;
21.    const skip = (parsedPage - 1) * parsedLimit;
22.
23.    const query = {
24.      isHide: { $ne: true },
25.    };
26.
27.    if (searchQuery) {
28.      query.$or = [
29.        { name: { $regex: searchQuery?.toString(), $options: "i" } },
30.        {
31.          description: { $regex: searchQuery?.toString(), $options: "i"
},
32.        },
33.      ];
34.    } else if (searchCategory) {
35.      if (mongoose.Types.ObjectId.isValid(searchCategory)) {
36.        query.$or = [
37.          { category: searchCategory },
38.          { subCategory: searchCategory },
39.        ];
40.      } else {
41.        // Find categories matching the name to get their IDs
42.        const matchedCategories = await Category.find({
43.          name: { $regex: searchCategory?.toString(), $options: "i" },
44.        }).select("_id");
45.
46.        const categoryIds = matchedCategories.map(c => c._id);
47.
48.        if (categoryIds.length > 0) {
49.          query.$or = [
50.            { category: { $in: categoryIds } },
51.            { subCategory: { $in: categoryIds } },
52.          ];
53.        } else {
54.          // If no category found by name, ensure no products are
returned if that was the only filter
55.          query.category = new mongoose.Types.ObjectId();
56.        }
57.      }
58.    }
59.
60.    const totalCount = await Product.countDocuments(query);
61.    const totalPages = Math.ceil(totalCount / parsedLimit) || 1;

```

```

62.
63.     const products = await Product.find(query)
64.       .populate("category subCategory")
65.       .sort({ _id: -1 })
66.       .skip(skip)
67.       .limit(parsedLimit);
68.
69.     if (products.length === 0)
70.       return res?.status(200).json({ message: "Product not found" });
71.
72.     const modifiedProducts = products?.map((product) => {
73.       let totalStarRating = 0;
74.       product?.review?.map((review) => {
75.         totalStarRating += review?.rating;
76.       });
77.       return {
78.         id: product?._id,
79.         name: product?.name,
80.         price: product?.price,
81.         description: product?.description,
82.         image: product?.image?.[0],
83.         category: product?.category,
84.         stock: product?.stock,
85.         rating: product?.review?.length
86.           ? Number(totalStarRating / product?.review?.length)
87.           : 0,
88.         ratingNumber: product?.review?.length || null,
89.         mrpPrice: product?.mrpPrice,
90.         sellingPrice: product?.sellingPrice,
91.         isAddedToWishlist: activeUserId
92.           ? activeUser?.wishlist?.some(
93.             (item) =>
94.               item?.productId?.toString() ===
product?._id?.toString(),
95.           )
96.           : false,
97.       };
98.     });
99.
100.    const start = totalCount === 0 ? 0 : skip + 1;
101.    const end = Math.min(skip + modifiedProducts.length, totalCount);
102.
103.    return res?.status(200).json({
104.      message: "data fetched",
105.      sucess: true,
106.      products: modifiedProducts,
107.      totalProducts: totalCount,
108.      page: parsedPage,
109.      totalPages,
110.      startProductIndex: start,
111.      endProductIndex: end,
112.    });
113.  } catch (error) {
114.    res?.status(500).json({ message: "Internal Server Error" });
115.    console.error(error);
116.  }
117. };
118. export default getAllProduct;
119.

```

File Name: backend\src\controllers\productController\getProductById.js

```
1. import Product from "../../model/productSchema.js";
2. import Category from "../../model/categorySchema.js";
3. const getProductById = async (req, res) => {
4.   try {
5.     const id = req?.params?.id;
6.     if (!id) return res.status(400).json({ message: "Product ID is required" });
7.     const product = await Product.findById(id).populate("category subCategory");
8.     if (product?.isHide)
9.       return res.status(404).json({
10.         success: false,
11.         message: "Product not found",
12.       });
13.     if (!product) return res.status(404).json({ message: "Product not found" });
14.     res.status(200).json({ success: true, message: "data returned", product });
15.   } catch (error) {
16.     console.error(error);
17.     res.status(500).send("Server Error");
18.   }
19.   // res.json({
20.   //   _id: req.params.id,
21.   //   name: "Sumit Gandhi",
22.   //   price: 1000,
23.   //   description: "this is a product",
24.   //   image:
25.   //     "https://images.pexels.com/photos/674010/pexels-photo-674010.jpeg?cs=srgb&dl=pexels-anjana-c-169994-674010.jpg&fm=jpg",
26.   //   category: "more pankh",
27.   //   stock: 1000,
28.   // });
29. };
30. export default getProductById;
31.
```

File Name: backend\src\controllers\productController\getRelatedProduct.js

```
1. import mongoose from "mongoose";
2. import Product from "../../model/productSchema.js";
3. import Category from "../../model/categorySchema.js";
4. import Order from "../../model/orderSchema.js";
5.
6. const getRelatedProduct = async (req, res) => {
7.   try {
8.     const { category, productId } = req?.query;
9.     if (!mongoose.Types.ObjectId.isValid(productId)) {
10.       return res.status(400).json({ message: "Invalid product ID" });
11.     }
12.     const product = await Product.find({
13.       category: category,
14.       _id: { $ne: productId },
15.       isHide: false,
16.     }).populate("category subCategory").limit(5);
17.
```

```

18.     const modifiedProducts = product
19.       // ?.filter((product) => !product?.isHide)
20.       ?.map((product) => {
21.         let totalStarRating = 0;
22.         product?.review?.map((review) => {
23.           totalStarRating += review?.rating;
24.         });
25.         return {
26.           id: product?._id,
27.           name: product?.name,
28.           price: product?.price,
29.           mrpPrice: product?.mrpPrice,
30.           sellingPrice: product?.sellingPrice,
31.           description: product?.description,
32.           image: product?.image?.[0],
33.           category: product?.category,
34.           stock: product?.stock,
35.           rating: Number(totalStarRating / product?.review?.length),
36.         };
37.       });
38.     return res.json({ message: "data fetched", product:
modifiedProducts });
39.   } catch (error) {
40.     console.error(error?.message);
41.     return res
42.       .status(500)
43.       .json({ message: "Internal server error", error: error.message
});
44.   }
45. };
46.
47. export default getRelatedProduct;
48.

```

File Name: backend\src\controllers\sellerController\deleteProduct.js

```

1. import Product from "../../model/productSchema.js";
2. import { deleteImageFromCloudinary } from "../../utility/cloudinary.js";
3.
4. const deleteProduct = async (req, res) => {
5.   try {
6.     const productId = req?.params?.id;
7.     const sellerId = req?.user?.sellerId?.toString();
8.
9.     const product = await Product.findById(productId);
10.
11.     if (!product) {
12.       return res.status(404).json({
13.         success: false,
14.         message: "Product not found",
15.       });
16.     }
17.
18.     if (product?.sellerId?.toString() !== sellerId) {
19.       return res.status(403).json({
20.         success: false,
21.         message: "Unauthorized access",
22.       });
23.     }

```

```

24.
25.     // Securely wipe all linked images from Cloudinary buckets to
prevent ghost files
26.     if (product.image && Array.isArray(product.image)) {
27.         const destructionPromises = product.image.map((url) =>
deleteImageFromCloudinary(url));
28.         await Promise.all(destructionPromises);
29.     }
30.
31.     await Product.findByIdAndDelete(productId);
32.
33.     return res.status(200).json({
34.         success: true,
35.         message: "Product deleted successfully",
36.     });
37. } catch (error) {
38.     console.log(error);
39.     return res.status(500).json({
40.         success: false,
41.         message: "Unable to delete product right now",
42.     });
43. }
44. };
45.
46. export default deleteProduct;
47.

```

File Name: backend\src\controllers\sellerController\getAllProductUsingSellerId.js

```

1. import Product from "../../model/productSchema.js";
2. import Category from "../../model/categorySchema.js";
3. import mongoose from "mongoose";
4.
5. const getAllProductUsingSellerId = async (req, res) => {
6.     try {
7.         const { filters = {}, page = 1, limit = 10 } = req?.query;
8.         const { category, productStockStatus } = filters;
9.         const sellerId = req?.user?.sellerId;
10.        const skip = (Number(page) - 1) * Number(limit);
11.        const query = { sellerId };
12.        if (productStockStatus === "in stock") {
13.            query.stock = { $gt: 20 };
14.        } else if (productStockStatus === "low stock") {
15.            query.stock = { $gt: 0, $lte: 20 };
16.        } else if (productStockStatus === "out of stock") {
17.            query.stock = 0;
18.        }
19.        if (category) {
20.            if (mongoose.Types.ObjectId.isValid(category)) {
21.                query.category = category;
22.            } else {
23.                const matchedCategory = await Category.findOne({
24.                    name: { $regex: category?.toString(), $options: "i" },
25.                }).select("_id");
26.                if (matchedCategory) {
27.                    query.category = matchedCategory._id;
28.                } else {
29.                    query.category = new mongoose.Types.ObjectId();
30.                }

```

```

31.     }
32.   }
33.   const totalCount = await Product.countDocuments(query);
34.   const totalPages = Math.ceil(totalCount / Number(limit));
35.   const allProduct = await Product?.find(query)
36.     .populate("category subCategory")
37.     .sort({ _id: -1 })
38.     .skip(skip)
39.     .limit(limit)
40.     .select("-review -__v");
41.   const start = totalCount === 0 ? 0 : skip + 1;
42.   const end = Math.min(skip + allProduct.length, totalCount);
43.   // console?.log(allProduct);
44.
45.   //   const modifiedProducts = allProduct?.map((product) => {
46.   //     return {
47.   //       id: product?._id,
48.   //       name: product?.name,
49.   //       price: product?.price,
50.   //       image: product?.image?.[0],
51.   //       category: product?.category,
52.   //       stock: product?.stock > 0,
53.   //       mrpPrice: product?.mrpPrice,
54.   //       sellingPrice: product?.sellingPrice,
55.   //     };
56.   //   });
57.
58.   return res?.status(200)?.json({
59.     sucess: true,
60.     message:
61.       allProduct.length > 0
62.       ? "All products fetched successfully"
63.       : "No Product found",
64.     allProduct,
65.     totalProducts: totalCount,
66.     page: Number(page),
67.     totalPages,
68.     startProductIndex: start,
69.     endProductIndex: end,
70.   });
71.   } catch (error) {
72.     console.log(error);
73.   }
74. };
75.
76. export default getAllProductUsingSellerId;
77.

```

File Name: backend\src\controllers\sellerController\productVisibilityToggle.js

```

1. import Product from "../..model/productSchema.js";
2. const productVisibilityToggle = async (req, res) => {
3.   const productId = req?.params?.id;
4.   const sellerId = req?.user?.sellerId?.toString();
5.   const product = await Product.findById(productId);
6.
7.   if (!product) {
8.     return res.status(404).json({ message: "Product not found" });
9.   }

```

```

10.   if (product?.sellerId?.toString() !== sellerId) {
11.       return res.status(403).json({ message: "Unauthorized access" });
12.   }
13.   product.isHide = !req?.body?.state;
14.   const updatedProduct = await product.save();
15.
16.   res?.status(200).json({
17.       message: "Product visibility toggled successfully",
18.       product: {
19.           ...updatedProduct._doc,
20.           isHide: !updatedProduct.isHide,
21.       },
22.   });
23. };
24.
25. export default productVisibilityToggle;
26.

```

File Name: backend\src\controllers\sellerController\updateProduct.js

```

1. import Product from "../../model/productSchema.js";
2. import { uploadImageToCloudinary, deleteImageFromCloudinary } from
   "../../utility/cloudinary.js";
3.
4. const updateProduct = async (req, res) => {
5.   try {
6.       const productId = req.params.id;
7.       const sellerId = req.user?.sellerId?.toString();
8.
9.       // 1. Find product
10.      const product = await Product.findById(productId);
11.      if (!product) {
12.          return res.status(404).json({ success: false, message: "Product
not found" });
13.      }
14.
15.      // 2. Validate ownership
16.      if (product.sellerId?.toString() !== sellerId) {
17.          return res.status(403).json({ success: false, message:
"Unauthorized access" });
18.      }
19.
20.      const {
21.          name,
22.          sellingPrice,
23.          mrpPrice,
24.          description,
25.          category,
26.          stock,
27.          brand,
28.          subCategory,
29.          existingImages, // String of array or Array of strings sent by
frontend containing kept URLs
30.      } = req.body;
31.
32.      // 3. Parse existing images kept by user
33.      let keptImages = [];
34.      if (existingImages) {
35.          try {

```

```

36.          // Axios FormData might send arrays as JSON stringified if we
stringified it, or just multiple same-named fields.
37.          keptImages = typeof existingImages === 'string' ?
JSON.parse(existingImages) : existingImages;
38.          } catch (err) {
39.              keptImages = Array.isArray(existingImages) ? existingImages :
[existingImages];
40.          }
41.      }
42.
43.      // 4. Determine which images were DELETED by the user locally so we
can obliterate them from Cloudinary
44.      const currentDbImages = product.image || [];
45.      const imagesToDestroy = currentDbImages.filter((dbUrl) =>
!keptImages.includes(dbUrl));
46.
47.      // Destroy the removed frames in parallel
48.      if (imagesToDestroy.length > 0) {
49.          const destructionPromises = imagesToDestroy.map((url) =>
deleteImageFromCloudinary(url));
50.          await Promise.all(destructionPromises);
51.      }
52.
53.      // 5. Upload any completely NEW images sent via req.files
54.      const newFiles = req.files || [];
55.      let newImageUrls = [];
56.      if (newFiles.length > 0) {
57.          const uploadPromises = newFiles.map((file) =>
uploadImageToCloudinary(file.buffer));
58.          newImageUrls = await Promise.all(uploadPromises);
59.      }
60.
61.      // Merge the kept existing images + the newly generated URLs
62.      const finalImagesArray = [...keptImages, ...newImageUrls];
63.
64.      if (finalImagesArray.length === 0) {
65.          return res.status(400).json({ success: false, message: "Product
must have at least one image." });
66.      }
67.
68.      // 6. Update the product in MongoDB
69.      const updatedProduct = await Product.findByIdAndUpdate(
70.          productId,
71.          {
72.              name,
73.              mrpPrice: Number(mrpPrice),
74.              sellingPrice: Number(sellingPrice),
75.              description,
76.              image: finalImagesArray,
77.              category,
78.              stock: Number(stock),
79.              brand,
80.              subCategory: subCategory || "",
81.          },
82.          { new: true } // Return updated doc
83.      );
84.
85.      return res.status(200).json({
86.          success: true,
87.          message: "Product updated successfully!",
88.          product: updatedProduct,

```

```

89.     });
90.   } catch (error) {
91.     console.error("Update Product Error:", error);
92.     res?.status(500)?.json({
93.       success: false,
94.       message: "Error in updating product, Please try again later.",
95.     });
96.   }
97. };
98.
99. export default updateProduct;
100.

```

File Name: backend\src\controllers\userController\addRemoveItemtoWishList.js

```

1. import User from "../..model/userSchema.js";
2. import Product from "../..model/productSchema.js";
3. import mongoose from "mongoose";
4. const addRemoveItemtoWishList = async (req, res) => {
5.   try {
6.     const activeUser = req?.user;
7.     const { productId } = req?.body;
8.     const isProductExist = await Product?.findById(
9.       new mongoose.Types.ObjectId(productId)
10.    );
11.    if (!isProductExist) {
12.      return res?.status(404)?.json({
13.        success: false,
14.        message: "Product No Found",
15.      });
16.    }
17.    const isAlreadyExist = await activeUser?.wishlist?.some(
18.      (item) => item?.productId?.toString() === productId?.toString()
19.    );
20.    if (isAlreadyExist) {
21.      activeUser.wishlist = await activeUser?.wishlist?.filter(
22.        (item) => item?.productId?.toString() !== productId?.toString()
23.      );
24.      await activeUser.save();
25.      return res?.status(200)?.json({
26.        success: true,
27.        isRemoved: true,
28.        message: "Item Removed from Shopping List!",
29.      });
30.    }
31.    await activeUser.wishlist.push({ productId });
32.    await activeUser.save();
33.
34.    res?.status(201)?.json({
35.      success: true,
36.      isItem: true,
37.      message: "Added Successfully!",
38.    });
39.  } catch (error) {
40.    console.log(error);
41.    return res?.status(500)?.json({
42.      success: false,
43.      message: "Internal Server Error",
44.    });

```

```
45.   }
46. };
47. export default addRemoveItemToWishList;
48.
```

File Name: backend\src\controllers\userController\addToCart.js

```
1. import Product from "../../model/productSchema.js";
2. const addToCart = async (req, res, next) => {
3.   try {
4.     // const { _id } = req.user;
5.     // console.log(_id);
6.     // const orders = await Order.find({ userId: _id })
7.     //   .populate("userId", "name email") // Populate customer data
8.     //   .exec();
9.     // console.log(orders);
10.    const { productId, quantity } = req.body;
11.    const user = req.user;
12.    // console.log(user);
13.    // user.cart = [];
14.    const product = await Product.findById(productId);
15.    if (!product) {
16.      return res.status(404).json({ message: "Product not found." });
17.    }
18.    const existingProduct = user.cart.find(
19.      (item) => item?.productId?.toString() === productId?.toString()
20.    );
21.    if (existingProduct) {
22.      existingProduct.quantity += quantity;
23.    } else {
24.      user.cart.push({ productId, quantity });
25.    }
26.    await user.save();
27.    await res
28.      ?.status(200)
29.      ?.json({ message: "Product Added!", cartCount: user?.cart?.length
30.    });
31.  } catch (error) {
32.    res.status(500).json({ message: "Internal Server Error" });
33.    console.log(error);
34.  }
35. };
36. export default addToCart;
37.
```

File Name: backend\src\controllers\userController\changePassword.js

```
1. import User from "../../model/userSchema.js";
2.
3. /**
4.  * POST /api/user/change-password
5.  * Requires: userAuthentication middleware (req.user populated)
6.  * Body: { currentPassword, newPassword, confirmPassword }
7.  */
8. const changePassword = async (req, res) => {
9.   const { currentPassword, newPassword, confirmPassword } = req.body;
```



```
10.   const userId = req.user._id;
11.
12.   try {
13.     if (!currentPassword || !newPassword || !confirmPassword) {
14.       return res.status(400).json({
15.         success: false,
16.         message: "All fields are required.",
17.       });
18.     }
19.
20.     if (newPassword.length < 6) {
21.       return res.status(400).json({
22.         success: false,
23.         message: "New password must be at least 6 characters.",
24.       });
25.     }
26.
27.     if (newPassword !== confirmPassword) {
28.       return res.status(400).json({
29.         success: false,
30.         message: "New password and confirm password do not match.",
31.       });
32.     }
33.
34.     const user = await User.findById(userId);
35.     if (!user) {
36.       return res.status(404).json({ success: false, message: "User not
found." });
37.     }
38.
39.     // Block Google-only users (no password set) from this route
40.     if (!user.password || user.hasPassword === false) {
41.       return res.status(400).json({
42.         success: false,
43.         message: "No password is set on this account. Use 'Set
Password' instead.",
44.       });
45.     }
46.
47.     const isMatch = await user.passwordCompare(currentPassword);
48.     if (!isMatch) {
49.       return res.status(401).json({
50.         success: false,
51.         message: "Current password is incorrect.",
52.       });
53.     }
54.
55.     user.password = newPassword;
56.     await user.save();
57.
58.     return res.status(200).json({
59.       success: true,
60.       message: "Password changed successfully!",
61.     });
62.   } catch (error) {
63.     console.error("Change Password Error:", error);
64.     return res.status(500).json({
65.       success: false,
66.       message: "Internal server error. Please try again.",
67.     });
68.   }
```

```
69. };
70.
71. export default changePassword;
72.
```

File Name: backend\src\controllers\userController\getAllWishList.js

```
1. import Product from "../../model/productSchema.js";
2. const getAllWishList = async (req, res) => {
3.   try {
4.     const currentPage = Math.max(
5.       1,
6.       Number.parseInt(req?.body?.page || req?.query?.page || "1", 10)
7.     );
8.     const limit = Math.max(
9.       1,
10.      Number.parseInt(req?.body?.limit || req?.query?.limit || "10",
10)
11.    );
12.    const wishList = await req.user?.wishlist;
13.
14.    if (wishList?.length === 0) {
15.      return res.status(204).json({
16.        success: true,
17.        message: "WishList is empty",
18.      });
19.    }
20.
21.    const totalWishListProducts = wishList.length;
22.    const totalPages = Math.max(1, Math.ceil(totalWishListProducts /
limit));
23.    const safePage = Math.min(currentPage, totalPages);
24.    const skip = (safePage - 1) * limit;
25.    const paginatedWishList = wishList.slice(skip, skip + limit);
26.    const productIds = paginatedWishList.map((item) =>
item?.productId);
27.
28.    const products = await Product.find({
29.      _id: { $in: productIds },
30.    }).populate("category", "name");
31.
32.    const productMap = new Map(
33.      products.map((product) => [product?._id?.toString(), product])
34.    );
35.
36.    const wishListProducts = paginatedWishList
37.      .map((item) => {
38.        const product = productMap.get(item?.productId?.toString());
39.        if (!product) {
40.          return null;
41.        }
42.
43.        return {
44.          id: product?._id || item?.productId || item?.id,
45.          name: product?.name,
46.          price: product?.price,
47.          description: product?.description,
48.          image: product?.image?.[0],
49.          category: product?.category,
```

```

50.         stock: product?.stock,
51.         mrpPrice: product?.mrpPrice,
52.         sellingPrice: product?.sellingPrice,
53.     };
54. })
55.     .filter(Boolean);
56.
57.     return res.status(200).json({
58.         success: true,
59.         message: "WishList Products",
60.         wishListProducts,
61.         totalWishListProducts,
62.         totalPages,
63.         currentPage: safePage,
64.         startProductIndex: totalWishListProducts > 0 ? skip + 1 : 0,
65.         endProductIndex: Math.min(skip + paginatedWishList.length,
totalWishListProducts),
66.     });
67. } catch (error) {
68.     console.log(error);
69.     return res.status(500).json({
70.         success: false,
71.         message: "Internal Server Error",
72.     });
73. }
74. };
75.
76. export default getAllWishList;
77.

```

File Name: backend\src\controllers\userController\getCartItem.js

```

1. import Product from "../../model/productSchema.js";
2. const getCartItem = async (req, res, next) => {
3.     try {
4.         const cartProduct = [];
5.         const cartItem = await req?.user?.cart;
6.         for (let i = 0; i < cartItem.length; i++) {
7.             const product = await Product?.findById({ _id:
cartItem[i]?.productId });
8.             // console.log(product);
9.             const filteredProductData = {
10.                 id: product?._id,
11.                 name: product?.name,
12.                 price: product?.price,
13.                 description: product?.description,
14.                 image: [product?.image?.[0]],
15.                 category: product?.category,
16.                 // stock: product?.stock,
17.                 sellerId: product?.userId || product?.userID ||
product?.sellerId,
18.                 sellingPrice: product?.sellingPrice || null,
19.                 mrpPrice: product?.mrpPrice || null,
20.             };
21.             // console.log(filteredProduct);
22.             cartProduct?.push({
23.                 // item: { ...filteredProductData },
24.                 ...filteredProductData,
25.                 quantity: cartItem[i]?.quantity,

```

```

26.     });
27.   }
28.   // console.log(cartProduct);
29.   // cartItem?.map(async (item, inde) => {
30.   //   const product = await Product.findById(item.productId);
31.   //   cartProduct.push({ ...product, quantity: item.quantity });
32.   //   // console.log(product);
33.   // });
34.   // console.log(cartItem);
35.   res?.status(200)?.json({
36.     cartProduct,
37.     message: "Data Fetch Successfully!",
38.     cartCount: cartProduct.length,
39.   });
40. } catch (error) {
41.   // console.log(req.user);
42.   console.log(error);
43. }
44. };
45.
46. export default getCartItems;
47.

```

File Name: backend\src\controllers\userController\setPassword.js

```

1. import User from "../../model/userSchema.js";
2.
3. /**
4.  * POST /api/user/set-password
5.  * Allows Google-login users (who have no password) to set one.
6.  * Requires: userAuthentication middleware (req.user is populated).
7.  */
8. const setPassword = async (req, res) => {
9.   const { newPassword, confirmPassword } = req.body;
10.   const userId = req.user._id;
11.
12.   try {
13.     if (!newPassword || !confirmPassword) {
14.       return res.status(400).json({
15.         success: false,
16.         message: "Both password fields are required.",
17.       });
18.     }
19.
20.     if (newPassword.length < 6) {
21.       return res.status(400).json({
22.         success: false,
23.         message: "Password must be at least 6 characters long.",
24.       });
25.     }
26.
27.     if (newPassword !== confirmPassword) {
28.       return res.status(400).json({
29.         success: false,
30.         message: "Passwords do not match.",
31.       });
32.     }
33.
34.     const user = await User.findById(userId);

```

```

35.     if (!user) {
36.         return res
37.             .status(404)
38.             .json({ success: false, message: "User not found." });
39.     }
40.
41.     // Set the password – the pre('save') bcrypt hook will hash it
42.     user.password = newPassword;
43.     user.hasPassword = true;
44.     await user.save();
45.
46.     return res.status(200).json({
47.         success: true,
48.         message:
49.             "Password set successfully! You can now log in with email &
password.",
50.         hasPassword: true,
51.     });
52. } catch (error) {
53.     console.error("Set Password Error:", error);
54.     return res.status(500).json({
55.         success: false,
56.         message: "Internal server error. Please try again.",
57.     });
58. }
59. };
60.
61. export default setPassword;
62.

```

File Name: backend\src\controllers\userController\updateCart.js

```

1. import Product from "../../model/productSchema.js";
2.
3. const updateCart = async (req, res, next) => {
4.     try {
5.         const { productId, quantity } = req.body;
6.         const foundedUser = await req.user;
7.
8.         if (quantity < 0) {
9.             return res?.status(400)?.json({ message: "Invalid quantity" });
10.        }
11.
12.        const item = foundedUser?.cart?.find(
13.            (item) => item.productId?.toString() === productId?.toString()
14.        );
15.
16.        if (!item) {
17.            return res?.status(404)?.json({ message: "Item not found" });
18.        }
19.
20.        if (quantity === 0) {
21.            foundedUser.cart = foundedUser?.cart?.filter(
22.                (item) => item?.productId?.toString() !== productId?.toString()
23.            );
24.            await foundedUser?.save();
25.
26.            return res?.status(200)?.json({
27.                message: "Item removed from cart",

```

```

28.         cartCount: foundedUser?.cart?.length,
29.         removedProductId: productId,
30.     });
31.     }
32.
33.     item.quantity = quantity;
34.     await foundedUser?.save();
35.
36.     const product = await Product.findById(productId);
37.
38.     if (!product) {
39.         return res?.status(404)?.json({ message: "Product not found" });
40.     }
41.
42.     const updatedCartItem = {
43.         id: product?._id,
44.         name: product?.name,
45.         price: product?.price,
46.         description: product?.description,
47.         image: [product?.image?.[0]],
48.         category: product?.category,
49.         sellerId: product?.userId || product?.userID ||
product?.sellerId,
50.         sellingPrice: product?.sellingPrice || null,
51.         mrpPrice: product?.mrpPrice || null,
52.         quantity: item.quantity,
53.     };
54.
55.     return res?.status(200)?.json({
56.         message: "Cart updated successfully",
57.         cartCount: foundedUser?.cart?.length,
58.         updatedCartItem,
59.     });
60.     } catch (error) {
61.         console.log(error);
62.         return res?.status(500)?.json({
63.             message: "Unable to update cart right now",
64.         });
65.     }
66. };
67. export default updateCart;
68.

```

File Name: backend\src\controllers\userController\updateProfile.js

```

1. import User from "../..model/userSchema.js";
2.
3. const updateProfile = async (req, res, next) => {
4.     try {
5.         const { email, name, phoneNumber, address } = req?.body;
6.         const foundedUser = req?.user;
7.         const user = req?.body;
8.         const emailExist = await User?.findOne({ email: email });
9.         if (
10.             emailExist &&
11.             emailExist?._id?.toString() !== foundedUser?._id?.toString()
12.         ) {
13.             return res?.status(400)?.json({ message: "Email already exists"
});

```

```

14.     }
15.     foundedUser.name = name || foundedUser?.name;
16.     foundedUser.email = email || foundedUser?.email;
17.     foundedUser.phoneNumber = phoneNumber || foundedUser?.phoneNumber;
18.     foundedUser.address = address || foundedUser?.address;
19.     await foundedUser?.save();
20.     console?.log(foundedUser);
21.     res?.json({
22.         message: "Hello",
23.         updatedUser: {
24.             id: foundedUser?._id,
25.             name: foundedUser?.name,
26.             email: foundedUser?.email,
27.             phoneNumber: foundedUser?.phoneNumber,
28.             address: foundedUser?.address,
29.             userType: foundedUser?.userType,
30.         },
31.     });
32. } catch (error) {
33.     console?.log("Can't get user Detail", error);
34.     return res?.status(500)?.json({ message: "Internal Server Error"
35. });
36. };
37.
38. export default updateProfile;
39.

```

File Name: backend\src\controllers\userController\updateShippingAddress.js

```

1. const normalizeAddress = (shippingAddress = {}) => ({
2.     addressLine1: shippingAddress?.addressLine1?.trim() || "",
3.     addressLine2:
4.         shippingAddress?.addressLine2?.trim() ||
5.         shippingAddress?.addressLine2?.trim() ||
6.         "",
7.     state: shippingAddress?.state?.trim() || "",
8.     city: shippingAddress?.city?.trim() || "",
9.     pinCode: shippingAddress?.pinCode?.trim() || "",
10.    country: shippingAddress?.country?.trim() || "",
11. });
12.
13. const isAddressValid = (shippingAddress = {}) =>
14.     shippingAddress?.addressLine1 &&
15.     shippingAddress?.state &&
16.     shippingAddress?.city &&
17.     shippingAddress?.pinCode &&
18.     shippingAddress?.country;
19.
20. const updateShippingAddress = async (req, res) => {
21.     try {
22.         const activeUser = req.user;
23.         const { shippingAddress } = req.body;
24.
25.         const normalizedAddress = normalizeAddress(shippingAddress);
26.
27.         if (!isAddressValid(normalizedAddress)) {
28.             return res
29.                 .status(400)

```

```

30.         .json({ message: "Complete shipping address is required" });
31.     }
32.
33.     activeUser.shippingAddress = normalizedAddress;
34.
35.     await activeUser.save();
36.
37.     return res.status(200).json({
38.         message: "Shipping address updated successfully",
39.         shippingAddress: activeUser.shippingAddress,
40.     });
41. } catch (error) {
42.     console.log("Failed to update shipping address", error);
43.     return res.status(500).json({ message: "Internal Server Error" });
44. }
45. };
46.
47. export default updateShippingAddress;
48.

```

File Name: backend\src\controllers\userController\userProfile.js

```

1. import User from "../model/userSchema.js";
2. import jwt from "jsonwebtoken";
3.
4. const userProfile = async (req, res, next) => {
5.     try {
6.         const foundedUser = req.user;
7.         const userDetails = {
8.             id: foundedUser?._id,
9.             name: foundedUser?.name,
10.            email: foundedUser?.email,
11.            phoneNumber: foundedUser?.phoneNumber,
12.            profilePic: foundedUser?.profilePic,
13.            userType: foundedUser?.userType,
14.            address: foundedUser?.address,
15.            shippingAddress: foundedUser?.shippingAddress,
16.            cartCount: foundedUser?.cart?.length || 0,
17.            hasPassword: foundedUser?.hasPassword ?? true,
18.            googleId: foundedUser?.googleId || null,
19.        };
20.        return res.status(200).json({ message: "data Found", userDetails });
21.    } catch (error) {
22.        console.log("Can't get user Detail", error);
23.        return res.status(500).json({ message: "Internal Server Error" });
24.    }
25. };
26.
27. export default userProfile;
28.

```

File Name: backend\src\middleware\adminAuthentication.js

```

1. const adminAuthentication = (req, res, next) => {
2.     const foundedUser = req?.user;
3.
4.     if (foundedUser?.userType !== "admin") {

```



```

5.     return res.status(403).json({
6.         success: false,
7.         message: "Access Denied. Administrator privileges required.",
8.     });
9. }
10.
11. next();
12. };
13.
14. export default adminAuthentication;
15.

```

File Name: backend\src\middleware\sellerAuthentication.js

```

1. const sellerAuthentication = (req, res, next) => {
2.     const foundedUser = req?.user;
3.     if (foundedUser?.userType !== "seller" || !foundedUser?.sellerId) {
4.         return res?.status(401)?.json({
5.             success: false,
6.             message: "You don't access of seller feature kindly resgiser as
Seller",
7.         });
8.     }
9.     next();
10. };
11.
12. export default sellerAuthentication;
13.

```

File Name: backend\src\middleware\userAuthentication.js

```

1. import User from "../model/userSchema.js";
2. import jwt from "jsonwebtoken";
3.
4. const userAuthentication = async (req, res, next) => {
5.     const authHeader = req?.headers?.authorization;
6.     const token = (authHeader && authHeader.split(" ")[1]) || "";
7.     try {
8.         if (!token) {
9.             return res
10.                .status(401)
11.                .json({ message: "Access denied. No token provided." });
12.         }
13.
14.         const decoded = jwt.verify(token, process.env.JWT_SECRET_KEY);
15.
16.         // console.log(decoded, "Hello");
17.
18.         if (!decoded) {
19.             return res.status(401).json({ message: "Access denied. Invalid
token." });
20.         }
21.
22.         const user = await User.findById(decoded?.userId);
23.
24.         if (!user) {
25.             return res.status(404).json({ message: "User not found." });

```

```

26.     }
27.
28.     // return res.status(200).json({ message: "data found", user });
29.     req.user = user;
30.     next();
31. } catch (err) {
32.     console.log(err?.message);
33.     res.status(401).json({ message: "Token expired" });
34. }
35. };
36. export default userAuthentication;
37.

```

File Name: backend\src\model\productSchema.js

```

1. import mongoose from "mongoose";
2.
3. const productSchema = new mongoose.Schema(
4.   {
5.     name: { type: String, required: true },
6.     price: { type: Number, required: false },
7.     mrpPrice: { type: Number, required: false, default: null },
8.     sellingPrice: { type: Number, required: false, default: null },
9.     description: { type: String, required: true },
10.    image: [],
11.    category: { type: mongoose.Schema.Types.ObjectId, ref: "Category",
required: true },
12.    subCategory: { type: mongoose.Schema.Types.ObjectId, ref:
"Category", required: false },
13.    brand: { type: String, required: false },
14.    stock: { type: Number, required: false },
15.    inStock: { type: Boolean, required: false },
16.    isHide: { type: Boolean, required: false, default: false },
17.    review: [
18.      {
19.        rating: { type: Number, required: true },
20.        title: { type: String, required: true },
21.        message: { type: String, required: true },
22.        userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
23.        userName: { type: String, required: true },
24.        createdAt: { type: Date, default: Date.now },
25.      },
26.    ],
27.    userID: {
28.      type: mongoose.Schema.Types.ObjectId,
29.      required: false,
30.      ref: "User",
31.    },
32.    sellerId: { type: mongoose.Schema.Types.ObjectId, ref: "SellerUser"
},
33.    visibilityState: { type: Boolean, default: true },
34.    // productId: {
35.    //   type: mongoose.Schema.Types.ObjectId,
36.    //   required: true,
37.    //   unique: true,
38.    //   index: true,
39.    //   auto: true,
40.    //   default: mongoose.Types.ObjectId(),
41.    // },

```

```

42.     // _id: false,
43.   },
44.   { timeStamps: true }
45. );
46.
47. // productSchema.method.generateToken = function () {
48. //   console.log(this);
49. //   const token = jwt.sign(
50. //     { id: this._id, email: this.email, name: this.name },
51. //     process.env.JWT_SECRET_KEY
52. //   );
53. //   return token;
54. // };
55. const Product = mongoose.model("Product", productSchema);
56.
57. export default Product;
58.

```

File Name: backend\src\routes\authRoute.js

```

1. import express from "express";
2. const router = express.Router();
3.
4. import register from "../controllers/authController/register.js";
5. import login from "../controllers/authController/login.js";
6. import forgetPassword from
7. "../controllers/authController/forgetPassword.js";
8. import verifyOtp from "../controllers/authController/verifyOtp.js";
9. import resetPassword from
10. "../controllers/authController/resetPassword.js";
11. import verifyOTPForRegistration from
12. "../controllers/authController/verifyOTPForRegistration.js";
13. import resendOtp from "../controllers/authController/resendOtp.js";
14. import otpVerification from "../middleware/otpVerification.js";
15. import googleAuth from "../controllers/authController/googleAuth.js";
16. import googleCallback from
17. "../controllers/authController/googleCallback.js";
18.
19. router.route("/register").post(register);
20. router
21.   .route("/email-verification")
22.   .post(otpVerification, verifyOTPForRegistration);
23. router.route("/login").post(login);
24. router.route("/resend-otp").post(resendOtp);
25. router.route("/forget-password").post(forgetPassword);
26. router.route("/verify-otp").post(otpVerification, verifyOtp);
27. router.route("/set-password").post(resetPassword);
28. router.route("/google").get(googleAuth);
29. router.route("/google/callback").post(googleCallback);
30. export default router;
31.

```

File Name: backend\src\routes\productRoutes.js

```

1. import express from "express";
2. import multer from "multer";
3.

```

```

4. const router = express.Router();
5.
6. // Memory storage keeps the files as Buffer objects directly in RAM
7. const upload = multer({ storage: multer.memoryStorage() });
8.
9. import addProduct from "../controllers/productController/addProduct.js";
10. import getAllProduct from
    "../controllers/productController/getAllProduct.js";
11. import getProductById from
    "../controllers/productController/getProductById.js";
12. import getRelatedProduct from
    "../controllers/productController/getRelatedProduct.js";
13.
14. import submitReview from
    "../controllers/reviewController/submitReview.js";
15.
16. import userAuthentication from "../middleware/userAuthentication.js";
17.
18. // Add multer middleware handling mutiple files under the field name
    'image'
19. router.route("/add-product").post(userAuthentication,
    upload.array("image"), addProduct);
20.
21. router.route("/get-all-products").get(getAllProduct);
22. router.route("/get-related-product").get(getRelatedProduct);
23. router.route("/:id").get(getProductById);
24.
25. router.route("/submit-review").post(userAuthentication, submitReview);
26.
27. export default router;
28.

```

File Name: backend/src/server.js

```

1. import "dotenv/config";
2.
3. import app from "./app.js";
4. import connectDB from "./config/db-connect.js";
5.
6. const PORT = process.env.PORT || 8000;
7. const isVercel = process.env.VERCEL === "1";
8.
9. await connectDB();
10.
11. if (!isVercel) {
12.   app.listen(PORT, () => {
13.     console.log(`Server is running on port ${PORT}`);
14.   });
15. }
16.
17. export default app;
18.

```